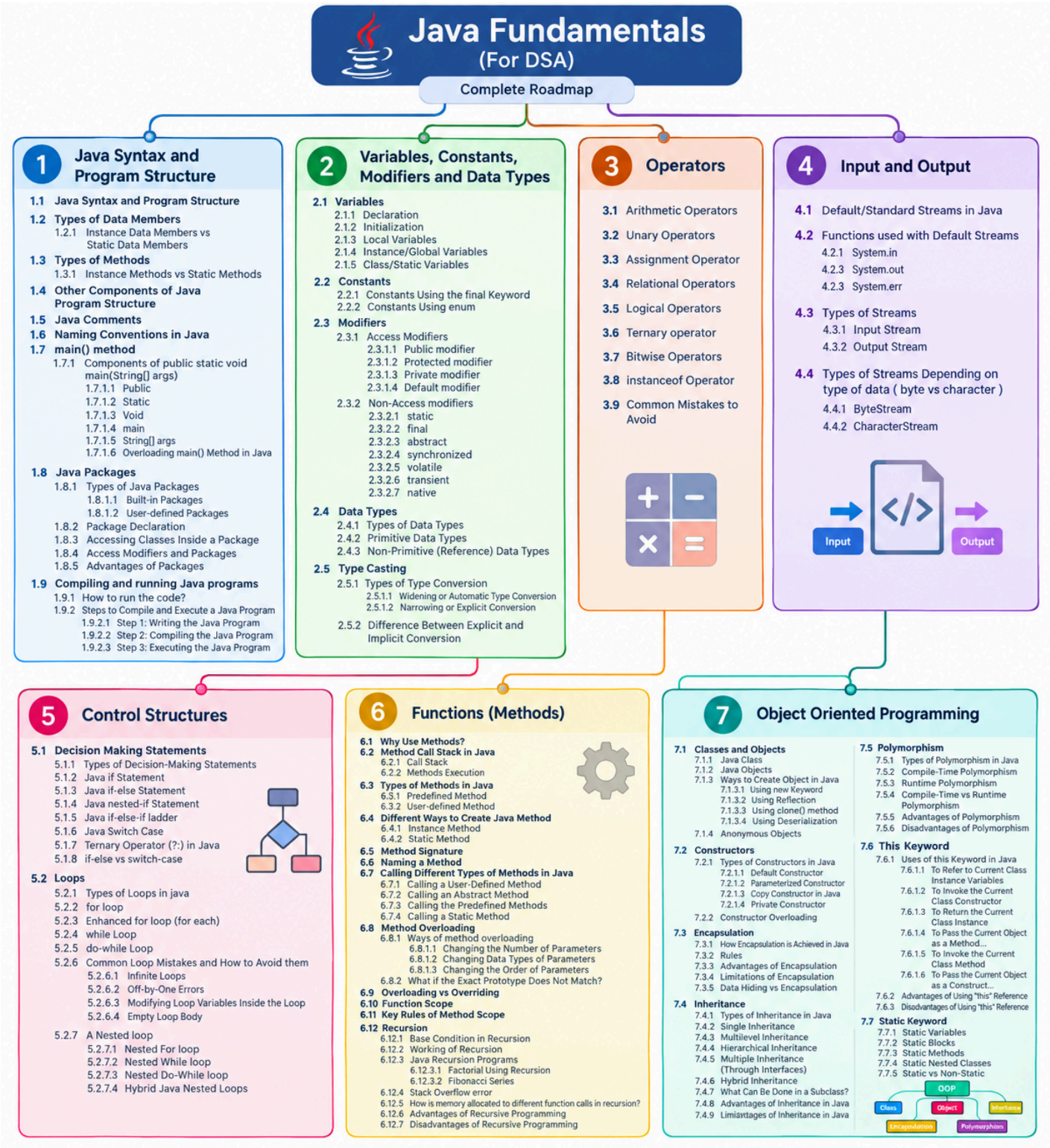


Java Fundamentals for Data Structures and Algorithms

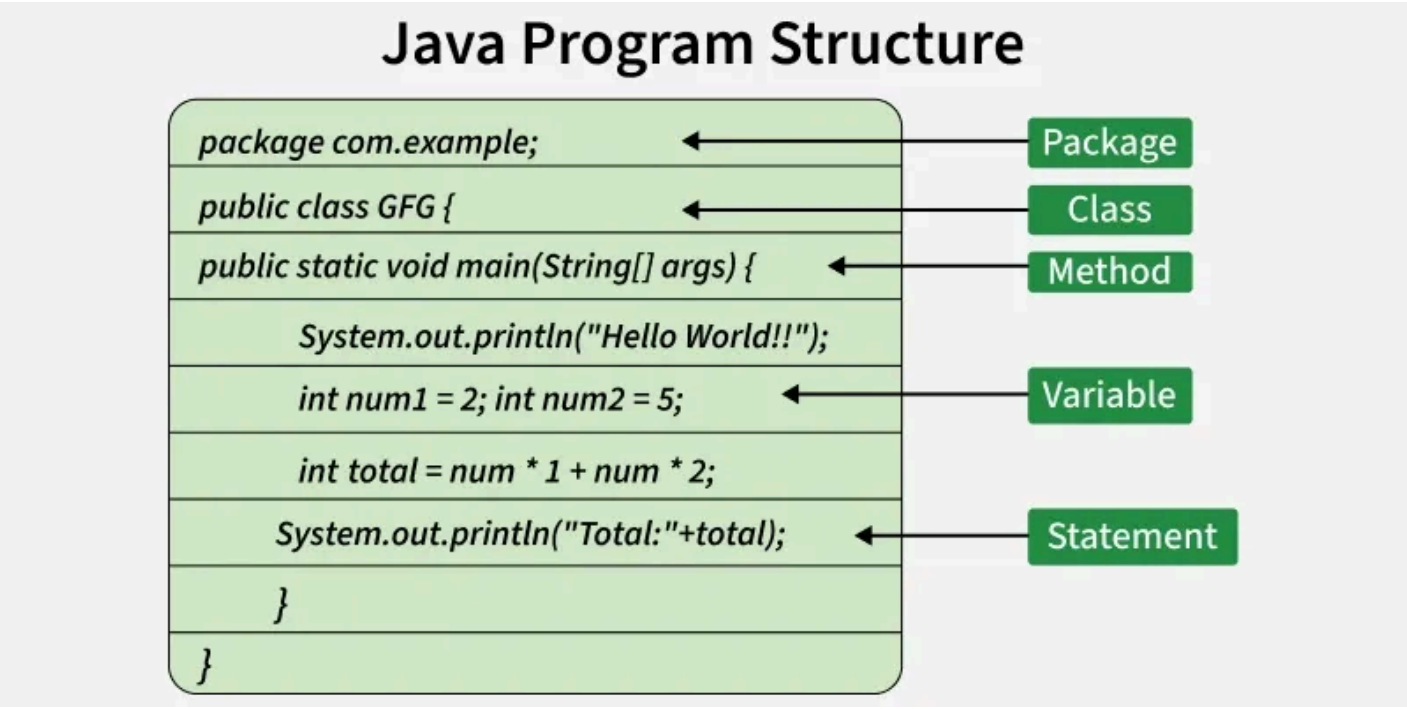
0. Overview



1. Java Syntax and Program Structure

1.1 Java Syntax and Program Structure

In Java, every program is structured into classes, with one containing the `main()` method as the entry point. Classes include data members to store information and methods to define actions on that data.



- **Package:** Groups related classes and interfaces, helping organize code and prevent naming conflicts.
- **Class:** Defines variables and methods, serving as a blueprint for creating objects in Java.
- **Method:** A block of code that performs a specific task; the `main()` method acts as the program's entry point.
- **Variable:** A named memory location used to store and manipulate data during program execution.
- **Statement:** An instruction that performs an action, such as computation or displaying output.

Example Code:-

```
public class GFG{
    public static void main(String[] args) {
        System.out.println("Hello World!!");

        int num1 = 2;
        int num2 = 5;

        int total = num1 * 1 + num2 * 2;

        System.out.println("Total: " + total);
    }
}
```

Output:-

```
Hello World!!
Total: 12
```

1.1.1 Types of Data Members

Java Class is a collection of data members and functions. Any Java program may contain two types of data members.

- Instance or non-static data members
- Static or class data members

A. Instance Data Members vs Static Data Members

The table below demonstrates the difference between Instance data member and Static data member.

Instance Data Members	Static Data Members
Memory is allocated every time an object is created.	Memory is allocated only once when the class is loaded.
Each object has its own copy of instance variables.	All objects share a single copy of static variables
Declared without the static keyword	Declared using the static keyword.

Instance Data Members	Static Data Members
Accessed using the object name	Accessed using the class name
It is known as Object-level data members.	It is Known as Class-level data members.
Value can be different for each object.	Value is common for all objects of the class.

1.1.2 Types of Methods

In Java programs generally, we may define two types of methods apart from the constructor.

- Instance or non-static methods
- Static or class methods

B. Instance Methods vs Static Methods

The table below demonstrates the difference between Instance methods and Static methods.

Instance Methods	Static Methods
Used to perform tasks related to individual objects, such as reading records from a file or processing object-specific data.	Used to perform operations that are related to the class as a whole, such as utility functions or managing shared resources.
Do not require the static keyword in their definition. Syntax: void net_salary(parameters) { statements; }	Must include the static keyword in their definition. Syntax: static void basic_salary(parameters) { statements; }
Accessed through an object of the class	Accessed through the class name.
Can access both instance variables and static variable	Cannot access instance variables or methods without an object reference.

Example Code:-

```
class Student {
    String name;           // Instance variable
    static String college; // Static variable

    void showName() {      // Instance method
        System.out.println(name);
    }

    static void showCollege() { // Static method
        System.out.println(college);
    }
}
```

C. Other Components of Java Program Structure

Concept	One-line definition
Object	An instance of a class
Keyword	A reserved word with a predefined meaning in Java
Identifier	A name given to classes, methods, variables, etc.

1.2 Java Comments

In Java, comments are non-executable statements that explain code and improve readability. They are ignored by the compiler and do not affect program execution.

- Enhance code readability and maintainability.
- Useful for debugging and documenting logic.

Java supports three main types of comments:

Type	Syntax	Purpose
Single-line	//	Short comment

Type	Syntax	Purpose
Multi-line	<code>/* ... */</code>	Multiple-line explanation
Documentation	<code>/** ... */</code>	Generates Javadoc documentation

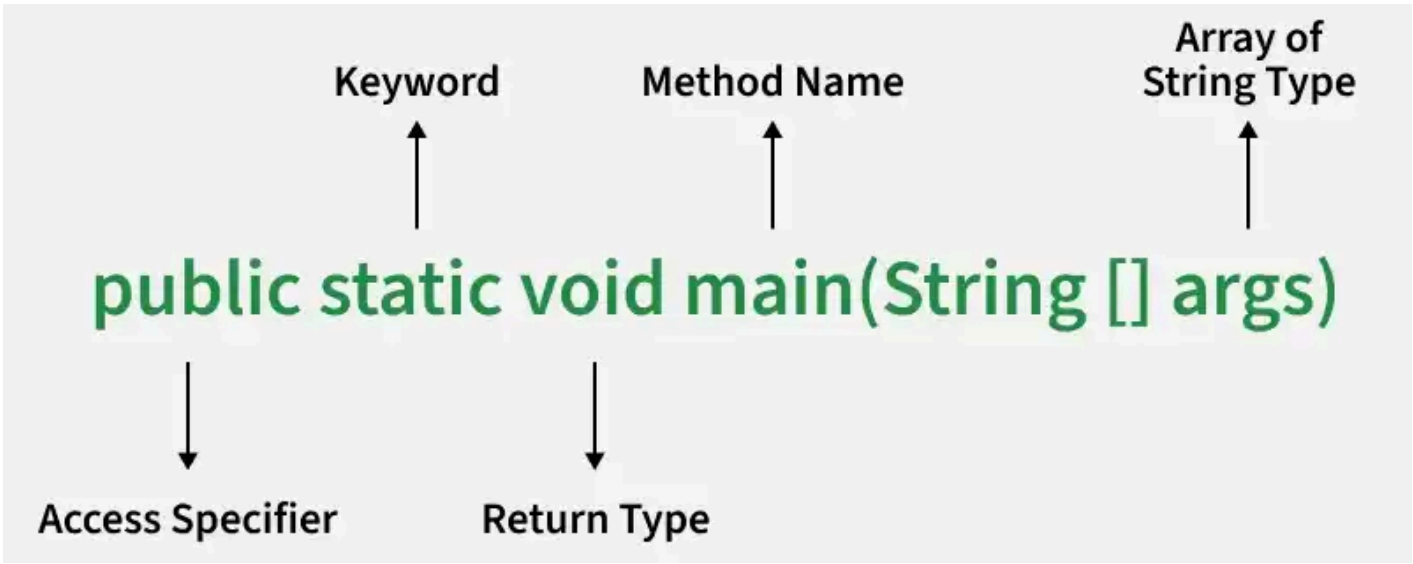
1.3 Naming Conventions in Java

Java Element	Naming Convention	Example	Key Rule
Class	PascalCase	<code>StudentDetails</code>	Start each word with uppercase; generally use nouns
Interface	PascalCase	<code>PaymentProcessor</code>	Start each word with uppercase; represent a contract/behavior
Method	camelCase	<code>calculateTotal()</code>	Start lowercase; generally use verbs
Variable	camelCase	<code>studentName</code>	Start lowercase; use meaningful names
Constant	UPPER_SNAKE_CASE	<code>MAX_SIZE</code>	Use uppercase letters with underscores; usually <code>final</code>
Package	lowercase	<code>com.company.project</code>	All lowercase; commonly use reverse-domain naming
Temporary Variable	camelCase / short name	<code>i</code> , <code>j</code> , <code>count</code>	Short names are acceptable when the purpose is obvious
Boolean Variable	camelCase	<code>isActive</code>	Prefer descriptive names indicating true/false state

1.4 main() method

Java’s main() method is the entry point of every Java program, where the JVM begins execution. If the main() method is missing or its signature is incorrect, the program will compile but won’t run.

- The JVM looks specifically for a properly defined main() method to start execution.
- The java.exe launcher initializes the JVM using JNI and invokes the main() method.
- The main thread created by the JVM is always a non-daemon thread.



1.4.1 Components of public static void main(String[] args)

A. Public

`public` is an access modifier. It allows the Java launcher to access the `main()` method from outside the class.

- `public` provides the required access for the traditional `main()` method.
- If the main method is not public, it's access is restricted.

B. Static

`static` indicates that the method belongs to the class rather than an object.

- The traditional `main()` method is static so that the Java launcher can invoke it without first creating an object of the class.
- If you try to run Java code where main is not static, you will get an error.

C. Void

void specifies that the main() method does not return a value.

- A method declared with `void` cannot return a value using a `return` statement with an expression.
- If main method is not void, we will get an error.

D. main

It is the name of the Java main method. It is the identifier that the JVM looks for as the starting point of the Java program. It's not a keyword. If we change the name while initiating main method, we will get an error.

E. String[] args

It stores Java command-line arguments and is an array of type java.lang.String class. Here, the name of the String array is args but it is not fixed and the user can use any name in place of it.

Example: Execution Process of String[]

F. Overloading main() Method in Java

Overloading the main() method is possible in Java, meaning we can create any number of main() methods in a program. To overload the main() method in Java, we need to create the main() method with different parameters. Only `public static void main(String[] args)` serves as the traditional program entry point. Other overloaded `main()` methods must be called explicitly

Example:-

```
public class Main {

    public static void main(String[] args) {
        if (args.length == 0) {
            System.out.println("Running main() with no arguments");
        } else if (args.length == 1) {
            try {
                int value = Integer.parseInt(args[0]);
                main(value);
            } catch (NumberFormatException e) {
                main(args[0]);
            }
        } else {
            // Handle more arguments as needed
        }
    }

    public static void main(int value) {
        System.out.println("Running main() with integer argument: " + value);
    }

    public static void main(String message) {
        System.out.println("Running main() with string argument: " + message);
    }
}
```

1.5 Java Packages

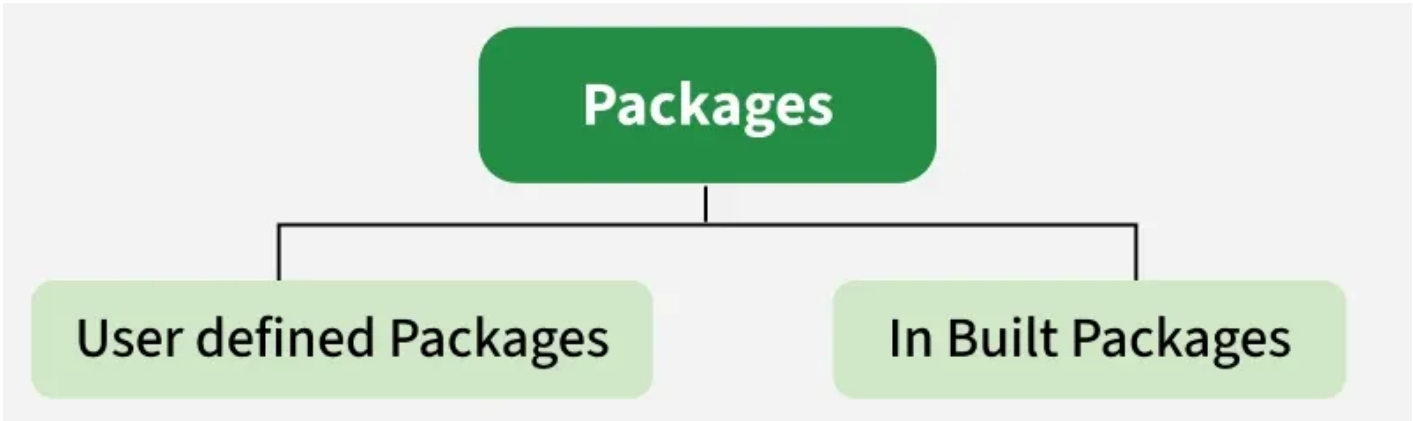
A package in Java is used to group related classes, **interfaces**, enums, and sub-packages. Packages help organize code, avoid naming conflicts, control access, and make large applications easier to maintain.

- A package can contain classes, interfaces, enums, and sub-packages.
- The package statement defines the package of a source file.

- They make code easier to maintain and reuse.

1.5.1 Types of Java Packages

Java packages are mainly divided into two types:



A. Built-in Packages

Built-in Packages are packages provided by the Java API. They contain classes and interfaces for common programming tasks. Some of the commonly used built-in packages are:

- **java.lang:** Contains language support classes(e.g, classes that define primitive data types, math operations). This package is automatically imported.
- **java.io:** Contains classes for supporting input/output operations.
- **java.util:** Contains utility classes that implement data structures such as Linked Lists and Dictionaries, as well as support for date and time operations.
- **java.awt:** Contains classes for implementing the components for graphical user interfaces (like buttons, menus, etc). 6)

B. User-defined Packages

User-defined Packages are packages created by programmers to organize application-specific classes and interfaces.

1.5.2 Package Declaration

The `package` statement is used to specify the package to which a class belongs.

Syntax:-

```
package packageName;
```

1.5.3 Accessing Classes Inside a Package

In Java, we can import classes from a package using either of the following methods:

Syntax

Purpose	General Syntax	Example
Import a Single Class	<code>import packageName.ClassName;</code>	<code>import java.util.Scanner;</code>
Import All Classes from a Package	<code>import packageName.*;</code>	<code>import java.util.*;</code>

Example

```
import java.util.Scanner; // Single class
import java.util.*;      // All classes from java.util
```

Pointer: `*` imports classes directly inside the package, **not classes from its sub-packages.**

1.5.4 Access Modifiers and Packages

Packages directly influence Java access levels:

	Default	Private	Protected	Public
Same Class	Yes	Yes	Yes	Yes
Same Package Subclass	Yes	No	Yes	Yes
Same Package Non-Subclass	Yes	No	Yes	Yes
Different Package Subclass	No	No	Yes	Yes
Different Package Non-Subclass	No	No	No	Yes

Default access is also called package-private because it is accessible only within the same package.

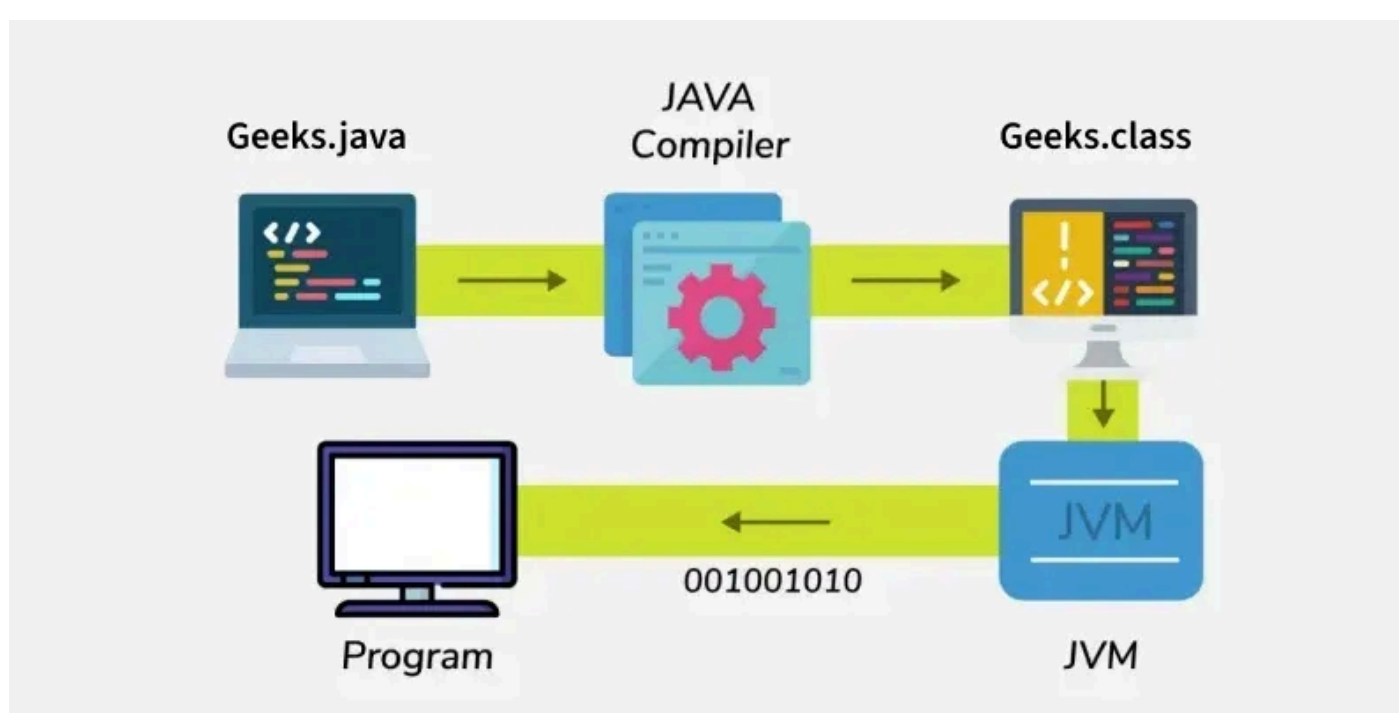
1.5.5 Advantages of Packages

- Organize related classes and interfaces.
- Avoid naming conflicts.
- Provide access control.
- Improve code maintainability.
- Promote code reuse.
- Make large applications easier to manage.
- Support modular application design.

1.6 Compiling and running Java programs

1.6.1 How to run the code?

- Write code in a file like Geeks.java.
- The Java Compiler "javac" compiles it into bytecode "Geeks.class".
- The **JVM** (Java Virtual Machine) reads the .class file and interprets the bytecode.
- The JVM loads and executes Java bytecode. During execution, the JVM may interpret bytecode and/or compile frequently executed code into native machine code using JIT compilation.



1.6.2 Steps to Compile and Execute a Java Program

Running a Java program involves two main steps: compilation and execution. Let's look at each step in detail.

A. Step 1: Writing the Java Program

Before you can run a Java program, you need to write it in a text editor. You can use any text editor like Notepad (Windows), TextEdit (Mac), or an IDE (Integrated Development Environment) like Eclipse, IntelliJ IDEA, or VS Code.

Example:

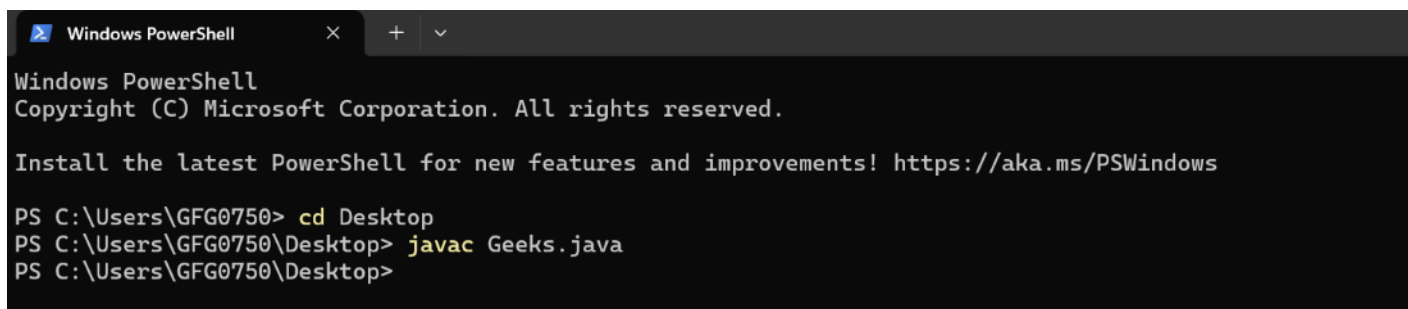
- **Class name:** Geeks
- **File name:** Geeks.java

B. Step 2: Compiling the Java Program

Once the Java program is written, it needs to be compiled. Compilation is the process of converting the human-readable Java code into bytecode, which can be understood by the Java Virtual Machine (JVM).

Here:

- **javac** is the Java compiler.
- **Geeks.java** is the file to compile.



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

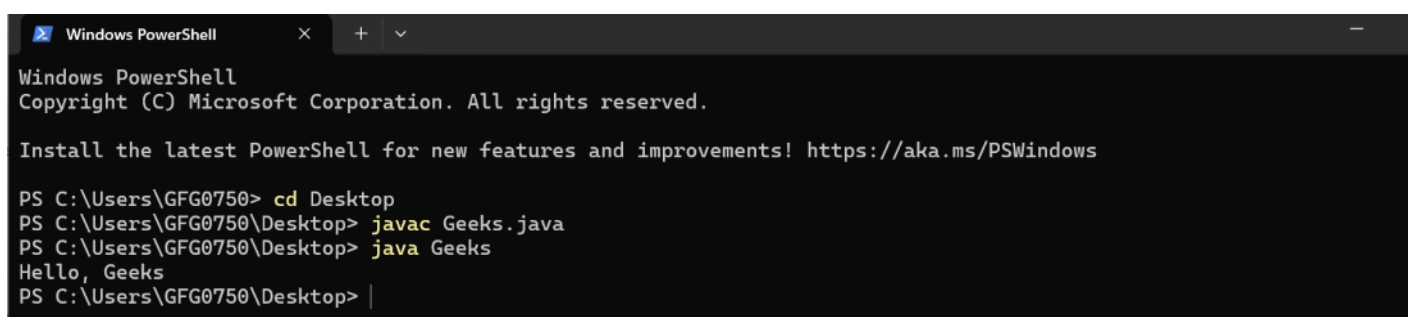
Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\GFG0750> cd Desktop
PS C:\Users\GFG0750\Desktop> javac Geeks.java
PS C:\Users\GFG0750\Desktop>
```

If there are no errors, this command will create a file called Geeks.class. The .class file contains bytecode, which is a platform-independent code that the JVM can execute. If there are errors, the compiler will show them in the terminal. You need to fix the errors before running the program.

C. Step 3: Executing the Java Program

After successful compilation, you can run the program using the Java interpreter.



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\GFG0750> cd Desktop
PS C:\Users\GFG0750\Desktop> javac Geeks.java
PS C:\Users\GFG0750\Desktop> java Geeks
Hello, Geeks
PS C:\Users\GFG0750\Desktop> |
```

Here:

- java is the Java interpreter.
- Geeks is the name of the class containing the main method (do not include .class).

When executed, you will see:

Hello, Geeks

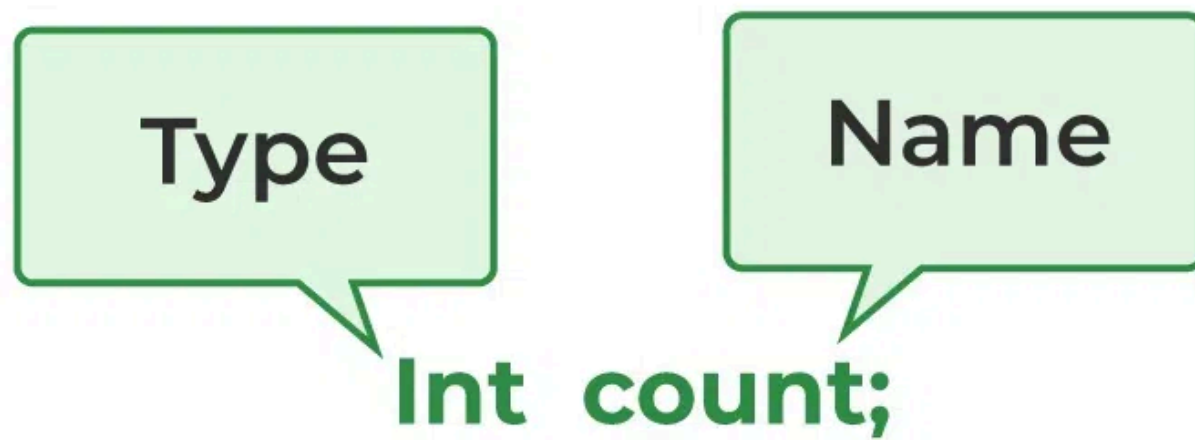
2. Variables , Constants, Modifiers and Data Types

2.1 Variables

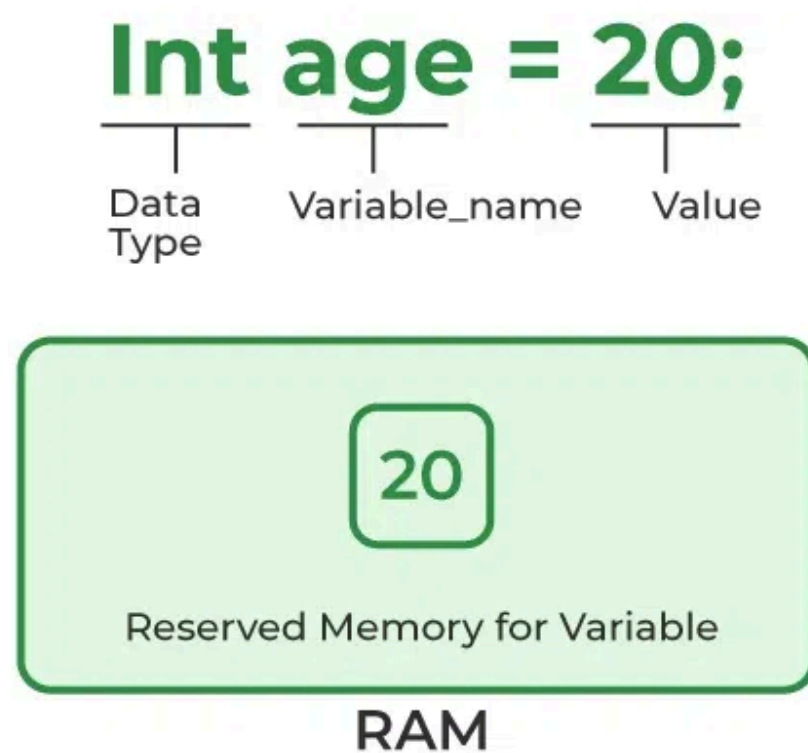
In Java, variables are containers used to store data in memory. Variables define how data is stored, accessed, and manipulated. A variable in Java has three components,

- **Data Type:** Defines the kind of data stored (e.g., int, String, float).
- **Variable Name:** A unique identifier following Java naming rules.
- **Value:** The actual data assigned to the variable.

2.1.1 Declaration



2.1.2 Initialization



2.1.3 Local Variables

- Local variables are declared inside the functions
- Local variables are created when the function is entered and the variable will be destroyed once it exits the function
- The **scope** of a local variable ends when its enclosing block ends. Its memory is an implementation/runtime concern and shouldn't be described simply as "destroyed."
- Access modifiers cannot be used for local variables
- Local variables are visible only within the declared function
- There is no default value for local variables, so local variables should be declared and an initial value should be assigned before the first use

Example Code:-

```
public class Employee {  
  
    public void incrementEmployeeCount() {  
        int count = 0;  
    }  
}
```

```

        count = count + 1;

        System.out.println("Employee Count : " + count);
    }

    public void getEmployeeCount() {
        int count = 0;
        count = count + 7;

        System.out.println("Employee Count : " + count);
    }

    public static void main(String[] args) {

        Employee emp = new Employee();

        emp.incrementEmployeeCount();
        emp.getEmployeeCount();
    }
}

```

Output:-

```

Employee Count : 1
Employee Count : 7

```

2.1.4 Instance/Global Variables

- Instance variables are declared in a class, but outside a method
- Instance variables are created when an object is created with the use of the keyword 'new' and destroyed when the object is destroyed
- Instance variables hold values that must be referenced by more than one method, or essential parts of an object's state that must be present throughout the class
- Access modifiers can be given for instance variables
- The instance variables are visible for all methods in the class. Normally, it is recommended to make these variables private (access level). However, visibility for subclasses can be given for these variables with the use of access modifiers.
- Instance variables have default values. For numbers, the default value is 0, for Booleans it is false, and for object references it is null. Values can be assigned during the declaration as well.

Example Code

```

public class MasterBridge {

    // Instance variable
    public String name;

    // Instance variable
    private String college;

    // Constructor
    public MasterBridge(String studentName) {
        name = studentName;
    }
}

```

```

// Assigns a value to the college instance variable
public void setCollege(String studentCollege) {
    college = studentCollege;
}

// Prints student details
public void printStudent() {
    System.out.println("Name : " + name);
    System.out.println("College : " + college);
}

public static void main(String[] args) {

    MasterBridge obj = new MasterBridge("Manav");

    obj.setCollege("Duke");

    obj.printStudent();
}
}

```

Output

```

Name : Manav
College : Duke

```

2.1.5 Class/Static Variables

- Class variables also known as static variables are declared with the static keyword in a class, but outside a method
- There would only be one copy of each class variable per class, regardless of how many objects are created from it
- Static variables are rarely used other than being declared as constants. Constants are variables that are declared as public/private, final, and static. Constant variables never change from their initial value.
- Static variables are stored in the static memory. It is rare to use static variables other than declared final and used as either public or private constants.
- Static variables are created when the program starts and destroyed when the program stops
- Visibility is similar to instance variables. However, most static variables are declared public since they must be available for users of the class.
- Default values are the same as instance variables. For numbers, the default value is 0; for Booleans, it is false; and for object references, it is null. Values can be assigned during the declaration or within the constructor. Additionally, values can be assigned in special static initializer blocks.
- Static variables can be accessed by calling with the class name
ClassName.VariableName.
- When declaring class variables as public static final, then variable names (constants) are all in uppercase. If the static variables are not public and final, the naming syntax is the same as instance and local variables.

Example Code:-


```

public class Employee {

    // Static / class variable
    public static int count = 0;

    // Static constant
    public static final String TEAM = "Social Ads Team";

    public void incrementEmployeeCount() {
        count = count + 1;
        System.out.println("Employee Count : " + count);
    }

    public void getEmployeeCount() {
        count = count + 7;
        System.out.println("Employee Count : " + count);
    }

    public static void main(String[] args) {

        Employee emp = new Employee();

        emp.incrementEmployeeCount();
        emp.getEmployeeCount();

        System.out.println(TEAM + " (headcount): " + count);
    }
}

```

Output

```

Employee Count : 1
Employee Count : 8
Social Ads Team (headcount): 8

```

2.2 Constants

In Java, symbolic constants are fixed values represented by meaningful names, rather than hard-coded values, which improves code readability and reliability. Java commonly uses **final variables** and **enums** to define symbolic constants. It's of various uses:

- Makes programs easier to maintain
- Prevents accidental modification of values
- Eliminates magic numbers
- Enhances consistency across the application

2.2.1 Constants Using the final Keyword

- **final** is used to make a **variable, method, or class unchangeable/restricted**.
- A **final variable** can be assigned a value **only once**.
- Once a final variable is initialized, its value **cannot be changed**.
- A **final method** cannot be **overridden** by a subclass.
- A **final class** cannot be **inherited/extended** by another class.
- **final** is commonly used to define **constants**.
- Constant names conventionally use **UPPER_SNAKE_CASE**.

Syntax:-

```
final dataType CONSTANT_NAME = value;
```

Example Code:-

```
class GFG {
    public static final double PI = 3.14159;

    public static void main(String[] args) {
        System.out.println("Value of PI: " + PI);
    }
}
```

Output:-

```
class GFG {
    public static final double PI = 3.14159;

    public static void main(String[] args) {
        System.out.println("Value of PI: " + PI);
    }
}
```

2.2.2 Constants Using enum

An enumeration (enum) is a distinct data type used in programming to define a collection of named constants. Here are its primary properties:

- **Type Safety:** Variables defined as an enum type can only accept specific, predefined values from that enum, preventing invalid data assignment.
- **Readability:** Enums replace obscure "magic numbers" (like `0`, `1`, `2`) with descriptive, human-readable names (like `IDLE`, `RUNNING`, `JUMPING`).
- **Fixed Collection:** They represent a finite, unchangeable grouping of related items (e.g., days of the week, user roles, or cardinal directions).
- **Underlying Numeric Value:** Under the hood, enum members are typically backed by integers (usually starting at `0`), though developers can often assign custom numeric or string values depending on the language.
- **Switch/Match Optimization:** Enums are highly optimized for use in `switch` statements, allowing compilers to efficiently route logic and often check if all possible cases are handled.
- **Namespace Grouping:** They group related constants together under a single type name, which prevents naming collisions in your code.

*Java enums are objects/classes, not C/C++-style integer enums. `ordinal()` gives an integer position, starting at 0, but that does **not** mean the enum constant itself is backed by that integer.*

Example Code:-

```
enum Day {
    MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY
}

class GFG {
    public static void main(String[] args) {
        Day today = Day.MONDAY;
        System.out.println(today);
    }
}
```

```
}  
}
```

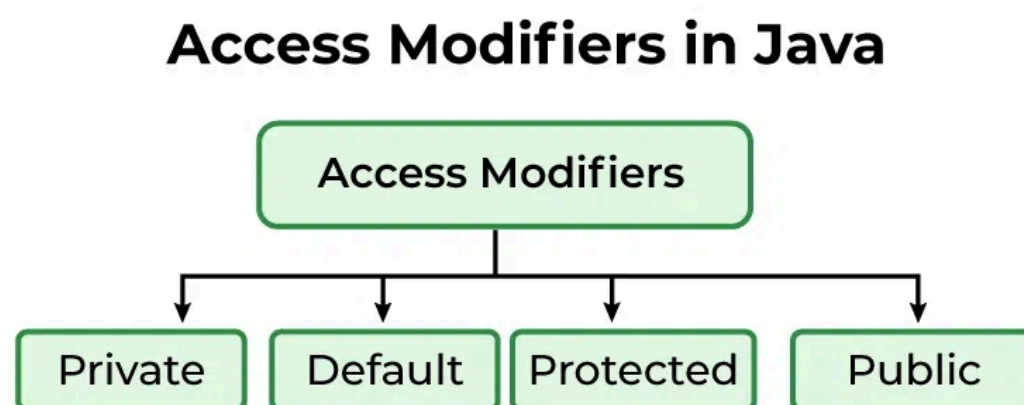
Output

MONDAY

2.3 Modifiers

2.3.1 Access Modifiers

Access modifiers in **Java** are used to control the visibility and accessibility of classes, **methods**, and **variables**. They help enforce encapsulation by restricting access to different parts of a program. Java provides four types of access modifiers to define scope and protection levels.



A. Public modifier

Accessible from anywhere in the program

Example Code:-

```
class MathUtils {  
  
    public static int add(int a, int b) {  
        return a + b;  
    }  
}  
  
public class Main {  
  
    public static void main(String[] args) {  
  
        System.out.println(MathUtils.add(5, 10)); // accessible anywhere  
    }  
}
```

Output:-

15

B. Protected modifier

Accessible within the same package and by subclasses

Example Code:-

```

// File: Vehicle.java in package p1
package p1;
public class Vehicle {
    protected int speed;
}

// File: Bike.java in package p2
package p2;
import p1.Vehicle;
public class Bike extends Vehicle {
    void showSpeed() {
        speed = 100; // allowed: subclass in different package
        System.out.println(speed);
    }
}

// File: Test.java in package p2
package p2;
import p1.Vehicle;
public class Test {
    public static void main(String[] args) {
        Bike b = new Bike();
        b.showSpeed(); // prints 100

        Vehicle v = new Vehicle();
        // System.out.println(v.speed); // ERROR: cannot access protected outside package & n
on-subclass
    }
}

```

Output:-

```

Access via subclass method: 100
0

```

C. Private modifier

Accessible only within the same class

Example Code:-

```

class Person {

    // private variable
    private String name;

    public void setName(String name) {

        this.name = name; // accessible within class
    }

    public String getName() { return name; }
}

public class Geeks {
    public static void main(String[] args)
    {

```

```
    Person p = new Person();
    p.setName("Alice");

    // System.out.println(p.name); // Error: 'name'
    // has private access
    System.out.println(p.getName());
}
}
```

Output:-

Alice

D. Default modifier

Accessible only within the same package

Example Code:-

```
class Car {
    String model; // default access
}

public class Main {

    public static void main(String[] args){

        Car c = new Car();
        c.model = "Tesla"; // accessible within the same package
        System.out.println(c.model);
    }
}
```

Output

Tesla

Comparision Table of Access Modifiers in Java

	Default	Private	Protected	Public
Same Class	Yes	Yes	Yes	Yes
Same Package Subclass	Yes	No	Yes	Yes
Same Package Non-Subclass	Yes	No	Yes	Yes
Different Package Subclass	No	No	Yes	Yes
Different Package Non-Subclass	No	No	No	Yes

2.3.2 Non-Access modifiers

Non-access modifiers provide information about the characteristics of a class, method, or variable to the JVM. Seven types of Non-Access modifiers are present in Java. They are -

A. static

The static keyword means that the entity to which it is applied is available outside any particular instance of the class. That means the static methods or the attributes are a part of the class and not an object. The memory is allocated to such an attribute or method at the time of class loading. The use of a static modifier makes the program more efficient by saving memory. A static field exists across all the class instances, and without creating an object of the class, they can be called.

Example Code:-

```
import java.io.*;

// static variable
class static_gfg {
    static String s = "GeeksforGeeks";
}
class GFG {
    public static void main(String[] args)
    {
        // No object required
        System.out.println(
            static_gfg.s);
    }
}
```

Output:-

```
GeeksforGeeks
```

B. final

The **final** keyword indicates that the specific class cannot be extended or a method cannot be overridden.

C. abstract

abstract keyword is used to declare a class as partially implemented means an object cannot be created directly from that class. Any subclass needs to be either implement all the methods of the abstract class, or it should also need to be an abstract class. The abstract keyword cannot be used with static, final, or private keywords because they prevent overriding, and we need to override methods in the case of an abstract class.

Example Code:-

```
// abstract class
abstract class abstract_gfg{
    abstract void myMethod();
}

//extending abstract class
class MyClass extends abstract_gfg{

    // overriding abstract method otherwise
    // code will produce error
    void myMethod(){
        System.out.println("GeeksforGeeks");
    }
}
class GFG{
    public static void main(String[] args) {
        MyClass obj=new MyClass();
        obj.myMethod();
    }
}
```

```
}  
}
```

Output:-

GeeksforGeeks

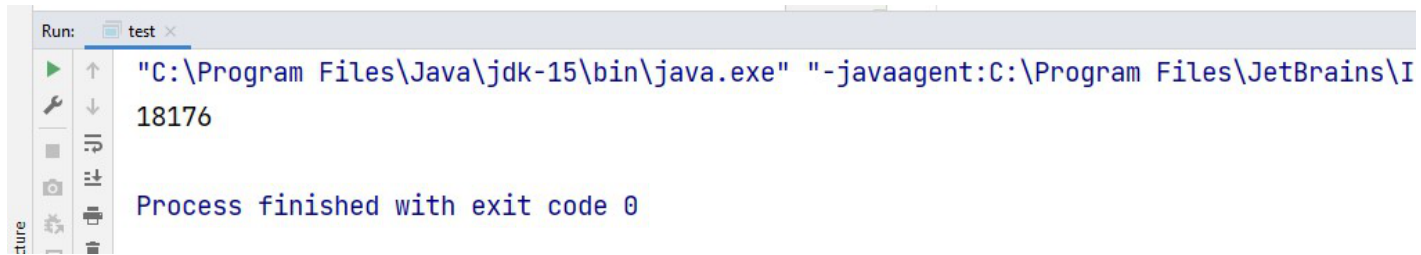
D. synchronized

synchronized keyword prevents a block of code from executing by multiple threads at once. It is very important for some critical operations

Example Code:-

```
import java.io.*;  
  
class Counter{  
    int count;  
    void increment(){  
        count++;  
    }  
}  
  
class GFG{  
    public static void main(String[] args) throws InterruptedException {  
        Counter c=new Counter();  
  
        // Thread 1  
        Thread t1=new Thread(new Runnable() {  
            @Override  
            public void run() {  
                for(int i=1;i<=10000;i++){  
                    c.increment();  
                }  
            }  
        });  
  
        // Thread 2  
        Thread t2=new Thread(new Runnable() {  
            @Override  
            public void run() {  
                for(int i=1;i<=10000;i++){  
                    c.increment();  
                }  
            }  
        });  
  
        t1.start();  
        t2.start();  
        t1.join();  
        t2.join();  
  
        System.out.println(c.count);  
    }  
}
```

Output:-



The above code should be an output value of **20000** as two threads increment it 10000 times each, and the main is waiting for **Thread1**, and **Thread2** to finish. Sometimes it may not be true. Depending upon the system, it may not give 20000 as output. As both threads are accessing the value of **count**, it may happen that Thread1 fetches the value of count, and before it could increment it, Thread2 reads the value and increments that. So thus, the result may be less than 20000. To solve this issue, we use the synchronized keyword. If the synchronized keyword is used while declaring the **increment()** method, then a thread needs to wait for another thread to complete the operation of the method then only another one can work on it. So we can get guaranteed output of 20000.

E. volatile

volatile keyword only and only ensures that changes that are made to variables are reflected across all threads that are accessing the same variable/thread. The usage of volatile keywords does not guarantee thread safety for class but it does not ensure thread safety and concurrency issues. Hence if we want our class to be thread-safe then we need to invoke it via the below technologies:

- Using synchronized keyword
- Using monitors
- using concurrency techniques (language dependent)

Example Code:-

```
// Java Program to Illustrate volatile keyword

import java.io.*;
import java.util.*;

// Class 1
class Geeks extends Thread {
    boolean running = true;
    @Override public void run()
    {
        while (running) {
            System.out.println("GeeksforGeeks");
        }
    }
    public void shutdown() { running = false; }
}

// Class 2
class GFG {

    // Main driver method
    public static void main(String[] args)
    {
        // Creating object of above class
        // which is extending Thread class
        Geeks obj = new Geeks();

        // start() method invoke run() method
        obj.start();

        Scanner input = new Scanner(System.in);
```

```

        input.nextLine();
        obj.shutdown();
    }
}

```

Output:-



In the above code, the program should ideally stop if Return Key/Enter is pressed, but in some machines, it may happen that the variable **running** is cached, and we are not able to change its value using the **shutdown()** method. In such a case, the program will execute infinite times and cannot be exited properly. To avoid caching and make it Thread-safe, we can use volatile keywords while declaring the **running** variable.

F. transient

The transient keyword may be applied to member variables of a class to indicate that the member variable should not be serialized when the containing class instance is serialized. Serialization is the process of converting an object into a byte stream. When we do not want to serialize the value of a variable, then we declare it as transient. To make it more transparent, let's take an example of an application where we need to accept UserID and Password. At that moment, we need to declare some variable to take the input and store the data, but as the data is susceptible, we do not want to keep it stored after the job is done. To achieve this, we can use the transient keyword for variable declaration. That particular variable will not participate in the serialization process, and when we deserialize that, we will receive the default value of the variable.

Example Code:-

```

import java.io.*;

class transient_gfg implements Serializable {
    // normal variable
    int a = 10;

    // Transient variables
    transient String UserID = "admin";
    transient String Password = "tiger123";
}

class GFG {
    public static void main(String[] args)
        throws IOException, ClassNotFoundException
    {
        transient_gfg obj = new transient_gfg();

        // printing the value of transient
        // variable before serialization process
    }
}

```

```

        System.out.println("UserID :" + obj.UserID);
        System.out.println("Password: " + obj.Password);
        System.out.println("a = " + obj.a);

        // serialization
        FileOutputStream fos
            = new FileOutputStream("abc.txt");
        ObjectOutputStream oos
            = new ObjectOutputStream(fos);
        oos.writeObject(obj);

        // de-serialization
        FileInputStream fis
            = new FileInputStream("abc.txt");
        ObjectInputStream ois = new ObjectInputStream(fis);
        transient_gfg output
            = (transient_gfg)ois.readObject();

        // printing the value of transient
        // variable after de-serialization process
        System.out.println("UserID :" + output.UserID);
        System.out.println("Password: " + output.Password);
        System.out.println("a = " + obj.a);
    }
}

```

Output:-

```

        UserID :admin
        Password: tiger123
        a = 10
        UserID :null
        Password: null
        a = 10

```

As you see from the output, after serialization, the values of UserID and Password are no longer present. However, the value of 'a', which is a normal variable, is still present.

G. native

The native keyword may be applied to a method to indicate that the method is implemented in a language other than Java. Using this java application can call code written in C, C++, or assembler language. A shared code library or DLL is required in this case.

Example Code:-

```

import java.io.*;

class GFG {
    // native method
    public native void printMethod();

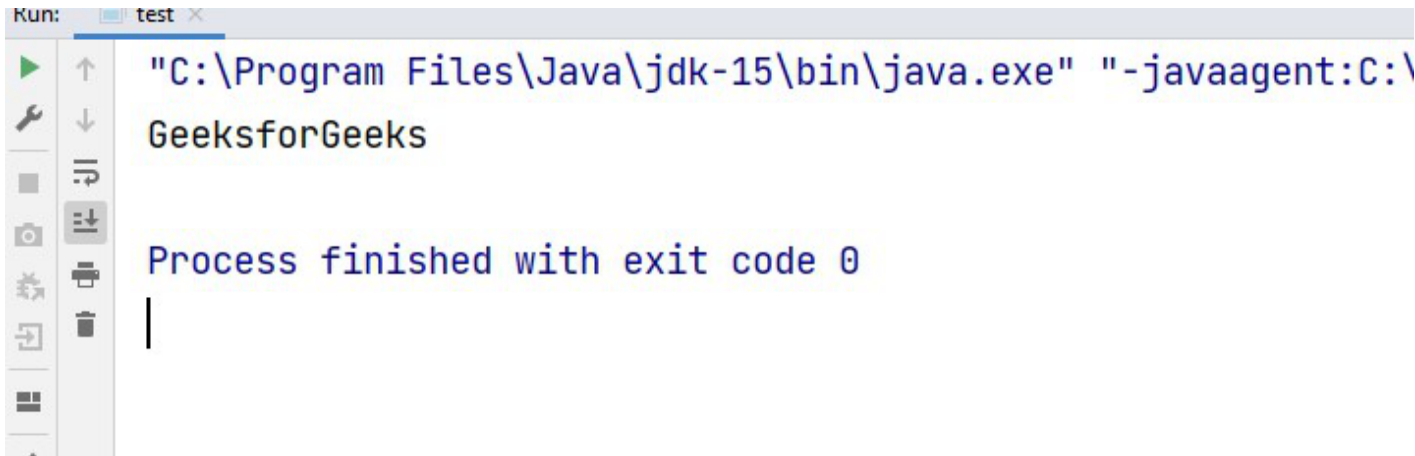
    static
    {
        // The name of DLL file
        System.loadLibrary("LibraryName");
    }
    public static void main(String[] args)
    {

```



```
GFG obj = new GFG();
obj.printMethod();
}
}
```

Output:-



In the above code, we have a native method. The method is defined in any other language and loaded by a java application using the shared DLL file. Implementation of the DLL file is out of the scope of this article, so if you want to know more about it, you can refer to this article.

2.4 Data Types

Java data types define the type of data a variable can store in a program. They help the compiler allocate memory efficiently and ensure type safety. Java provides two main categories: primitive and non-primitive data types.

- Memory allocation determines how much memory is required for each variable
- Operations support defines what operations can be performed on data
- Each data type has a default value when not initialized

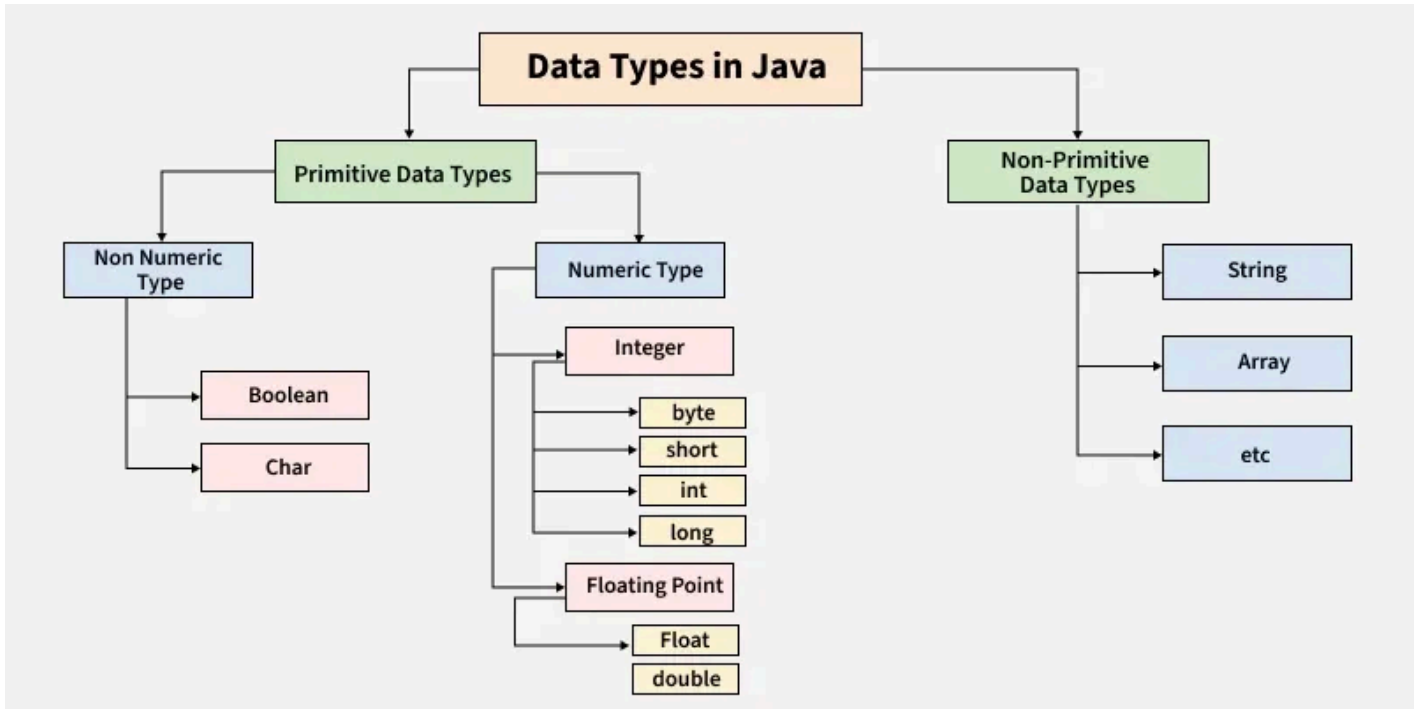
Syntax:-

```
dataType variableName = value;
```

2.4.1 Types of Data Types

Data types in Java define the kind of data a variable can hold and the memory required to store it. They are broadly divided into two categories:

- **Primitive Data Types:** Store simple values directly in memory.
- **Non-Primitive (Reference) Data Types:** Store memory references to objects.



2.4.2 Primitive Data Types

Primitive data types store simple values directly in memory. Java provides eight primitive data types, each with a fixed size and range, which are summarized below

Type	Description	Default	Size	Example	Range
boolean	Logical values	false	Not JVM-defined	true, false	true or false
byte	8-bit signed integer	0	1 byte	10	-128 to 127
char	16-bit Unicode character	\u0000	2 bytes	'A', '\u0041'	0 to 65,535
short	16-bit signed integer	0	2 bytes	2000	-32,768 to 32,767
int	32-bit signed integer	0	4 bytes	1000, -500	-2,147,483,648 to 2,147,483,647
long	64-bit signed integer	0L	8 bytes	123456789L	±9.22e18
float	32-bit floating point	0.0f	4 bytes	3.14f	~6–7 digits precision
double	64-bit floating point	0.0d	8 bytes	3.14159d	~15–16 digits precision

2.4.3 Non-Primitive (Reference) Data Types

Non-primitive data types store references (memory addresses) rather than actual values. They are created by users and include types like String, Class, Object, Interface, and Array.

Type	Description	Default	Size	Example	Range
String	Sequence of characters	null	Variable (depends on length & JVM)	"Hello World"	N/A (Limited by heap memory)
Array	Fixed-size collection of elements of the same type	null	Variable (depends on length & element type)	int[] numbers = {1, 2, 3}	N/A (Max length Integer.MAX_VALUE)
Class (Object)	Instance of a user-defined or built-in blueprint	null	Variable (reference size + object overhead)	new Scanner(System.in)	N/A (Limited by heap memory)
Interface	Abstract type used to specify a behavior or contract	null	N/A (Cannot be instantiated directly)	List<String> , Runnable	N/A
Enum	Special class representing a group of named constants	null	Variable (reference size)	Day.MONDAY	N/A (Defined by the constant set)

2.5 Type Casting

Java provides primitive data types with fixed memory sizes, and when assigning values between them, compatible types are converted automatically, while incompatible types require explicit casting.

- Java has 8 primitive data types, each with a fixed memory size.
- Automatic type conversion happens when assigning a smaller type to a larger type (e.g., int → long).
- Explicit type casting is required when converting a larger type to a smaller type (e.g., double → int).
- Automatic conversion is safe, while explicit casting may cause data loss.

2.5.1 Types of Type Conversion

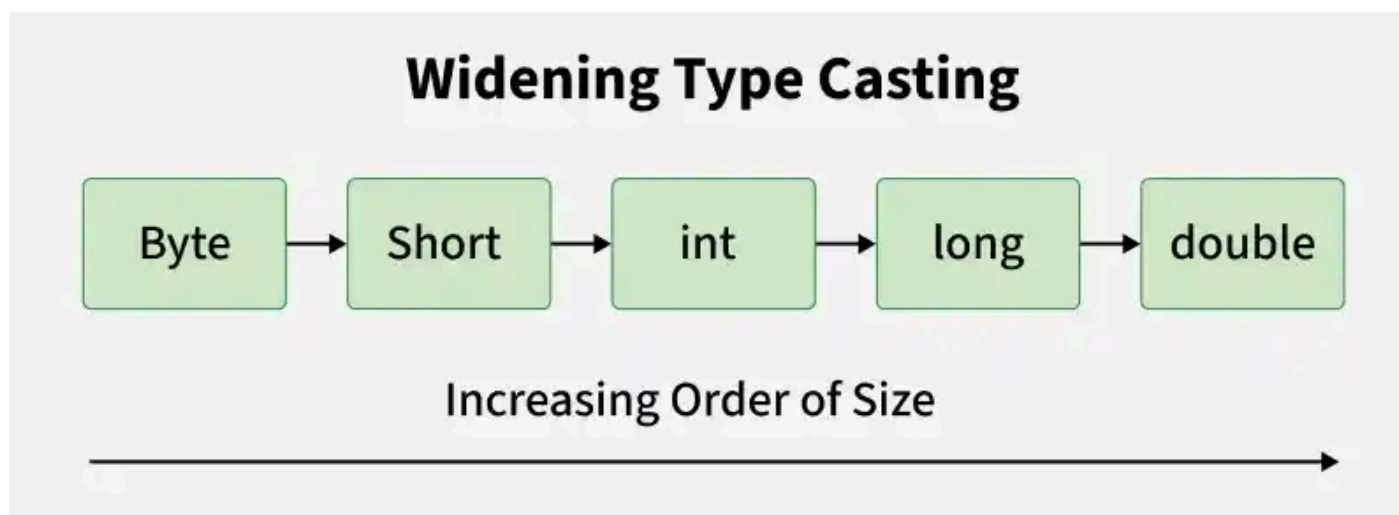
Type conversion in Java is done in two ways:

A. Widening or Automatic Type Conversion

Widening conversion takes place when two data types are automatically converted. This happens when:

- The two data types are compatible.
- When we assign a value of a smaller data type to a bigger data type.

For Example, in java, the numeric data types are compatible with each other but no automatic conversion is supported from numeric type to char or boolean. Also, char and boolean are not compatible with each other.



Example Code:-

```
class GFG {  
  
    // Main driver method  
    public static void main(String[] args)  
    {  
        int i = 100;  
  
        // Automatic type conversion  
        // Integer to long type  
        long l = i;  
  
        // Automatic type conversion  
        // long to float type  
        float f = l;  
  
        // Print and display commands  
        System.out.println("Int value " + i);  
        System.out.println("Long value " + l);  
        System.out.println("Float value " + f);  
    }  
}
```

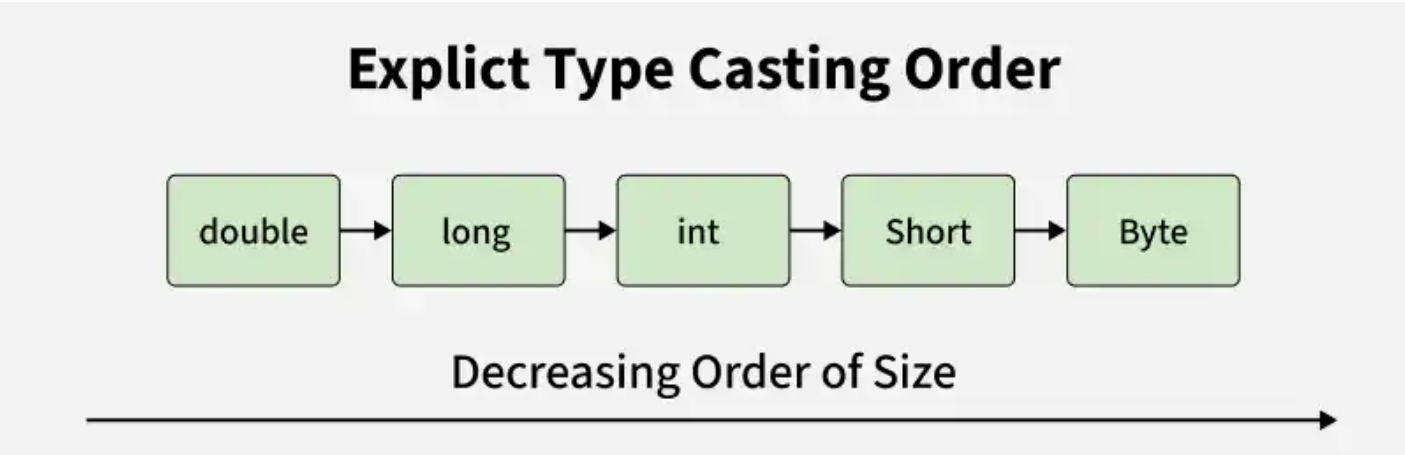
Output:-

```
Int value 100  
Long value 100  
Float value 100.0
```

B. Narrowing or Explicit Conversion

If we want to assign a value of a larger data type to a smaller data type we perform explicit type casting or narrowing.

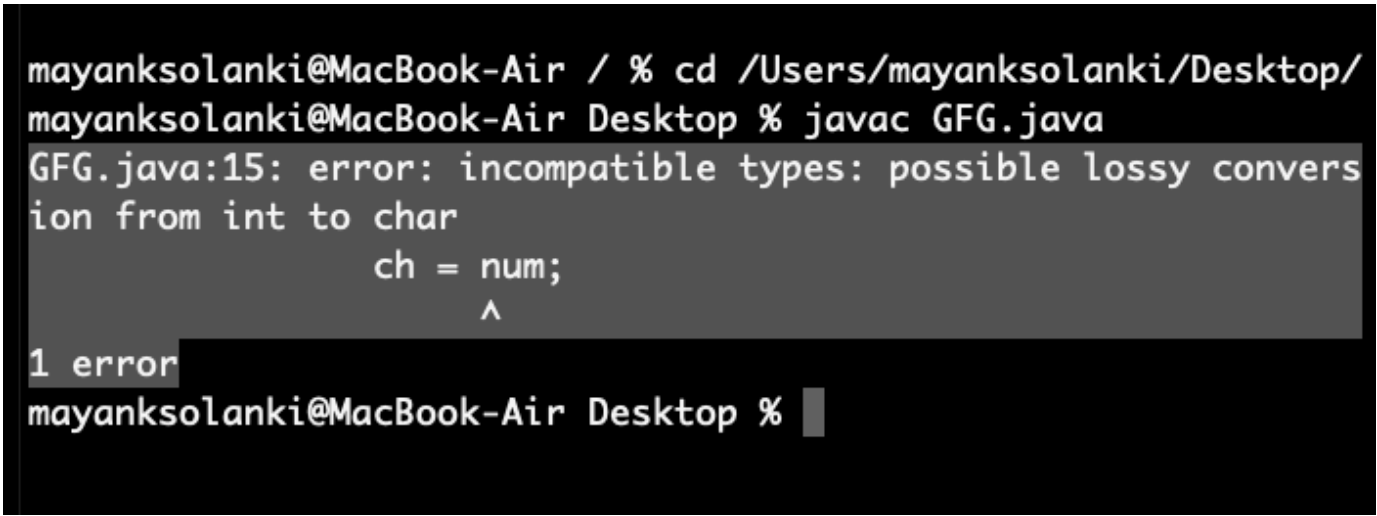
- This is useful for incompatible data types where automatic conversion cannot be done.
- Here, the target type specifies the desired type to convert the specified value to.



Example Code:-

```
public class GFG {  
  
    // Main driver method  
    public static void main(String[] argv)  
    {  
  
        // Declaring character variable  
        char ch = 'c';  
        // Declaring integer variable  
        int num = 88;  
        // Trying to insert integer to character  
        ch = num;  
    }  
}
```

Output:-



How To Solve The Error:

This error occurs because Java does not allow implicit narrowing conversion from `int` to `char`, even if the value can fit. To assign an `int` to a `char`, explicit casting is required:

```
char ch = (char) num; // Now valid
```

2.5.2 Difference Between Explicit and Implicit Conversion

The below table demonstrates the key difference between Explicit and Implicit Conversion

Aspect	Implicit Conversion	Explicit Conversion
Syntax	No additional syntax required.	Requires explicit type casting (e.g., (int)).
Data Compatibility	Only works with compatible types (e.g., numeric).	Works with both compatible and incompatible types.
Risk of Data Loss	No risk of data loss.	May result in data loss (e.g., truncation).
Direction	Smaller type to larger type.	Larger type to smaller type.

Note:-



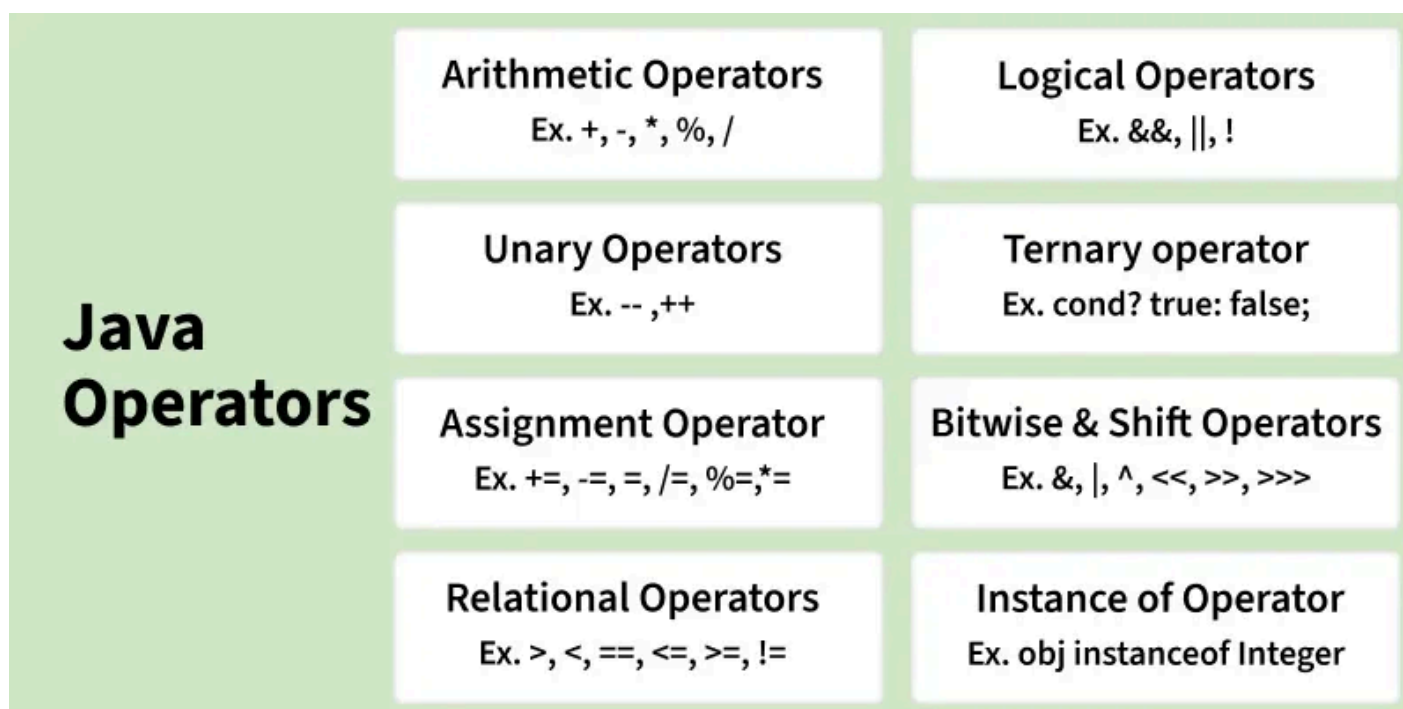
Use implicit conversion when assigning a smaller data type to a larger one (e.g., `int` → `long`).

Use explicit conversion when assigning a larger data type to a smaller one (e.g., `double` → `int`) or for incompatible types.

3. Operators

Java operators are symbols used to perform operations on variables and values. They play a key role in expressions, calculations, and decision-making in programs. Operators help simplify complex logic into concise statements.

- They follow a defined precedence and associativity to determine execution order.
- Some operators work on a single operand (unary), while others require two or more operands.



3.1 Arithmetic Operators

Arithmetic Operators are used to perform arithmetic operations on primitive numeric data types such as `int`, `float`, and `double`.

Example Code:-

```
public class GFG{
    public static void main(String[] args) {
        int a = 10, b = 3;

        // Addition
        int sum = a + b;

        // Subtraction
        int diff = a - b;

        // Multiplication
        int mul = a * b;

        // Division
        int div = a / b;

        // Modulus
        int mod = a % b;        // Modulus

        System.out.println("Sum: " + sum);
        System.out.println("Difference: " + diff);
    }
}
```

```

        System.out.println("Multiplication: " + mul);
        System.out.println("Division: " + div);
        System.out.println("Modulus: " + mod);
    }
}

```

Output:-

```

Sum: 13
Difference: 7
Multiplication: 30
Division: 3
Modulus: 1

```

3.2 Unary Operators

Unary Operators need only one operand. They are used to increment, decrement, or negate a value.

Example Code:-

```

import java.io.*;

// Driver Class
class Geeks{

    public static void main(String[] args){

        // Integer declared
        int a = 10;
        int b = 10;

        // Using unary operators
        System.out.println("Postincrement : " + (a++));
        System.out.println("Preincrement : " + (++a));

        System.out.println("Postdecrement : " + (b--));
        System.out.println("Predecrement : " + (--b));
    }
}

```

Output:-

```

Postincrement : 10
Preincrement : 12
Postdecrement : 10
Predecrement : 8

```

3.3 Assignment Operator

The **assignment operator** assigns a value from the right-hand side to a variable on the left. Since it has right-to-left associativity, the right-hand value must be declared or constant.

Example Code:-

```

public class GFG {
    public static void main(String[] args) {

        // initial value
    }
}

```



```

int num = 10;
System.out.println("Initial: " + num);

// add 5 → num = num + 5
num += 5;
System.out.println("After +5: " + num);

// multiply by 2 → num = num * 2
num *= 2;
System.out.println("After *2: " + num);

// subtract 5 → num = num - 5
num -= 5;
System.out.println("After -5: " + num);

// divide by 2 → num = num / 2
num /= 2;
System.out.println("After /2: " + num);

// remainder after dividing by 3 → num = num % 3
num %= 3;
System.out.println("After %3: " + num);
}
}

```

Output:-

```

Initial: 10
After +5: 15
After *2: 30
After -5: 25
After /2: 12
After %3: 0

```

3.4 Relational Operators

Relational Operators are used to check for relations like equality, greater than, and less than. They return boolean results after the comparison and are extensively used in looping statements as well as conditional if-else statements.

Example Code:-

```

import java.io.*;

class Geeks{

    public static void main(String[] args){

        // Comparison operators
        int a = 10;
        int b = 3;
        int c = 5;

        System.out.println("a > b: " + (a > b));
        System.out.println("a < b: " + (a < b));
        System.out.println("a >= b: " + (a >= b));
        System.out.println("a <= b: " + (a <= b));
        System.out.println("a == c: " + (a == c));
        System.out.println("a != c: " + (a != c));
    }
}

```

```
}  
}
```

Output:-

```
a > b: true  
a < b: false  
a >= b: true  
a <= b: false  
a == c: false  
a != c: true
```

3.5 Logical Operators

Logical Operators are used to perform "logical AND" and "logical OR" operations, similar to AND gate and OR gate in digital electronics. They have a short-circuiting effect, meaning the second condition is not evaluated if the first is false.

Example Code:-

```
import java.io.*;  
  
class Geeks {  
  
    // Main Function  
    public static void main (String[] args) {  
  
        // Logical operators  
        boolean x = true;  
        boolean y = false;  
  
        System.out.println("x && y: " + (x && y));  
        System.out.println("x || y: " + (x || y));  
        System.out.println("!x: " + (!x));  
    }  
}
```

Output:-

```
x && y: false  
x || y: true  
!x: false
```

3.6 Ternary operator

The **Ternary Operator** is a shorthand version of the if-else statement. It has three operands and hence the name Ternary. The general format is,

Example Code:-

```
public class Geeks{  
  
    public static void main(String[] args){  
  
        int a = 20, b = 10, c = 30, result;  
  
        // result holds max of three  
        // numbers  
        result = ((a > b) ? (a > c) ? a : c : (b > c) ? b : c);  
        System.out.println("Max of three numbers = "+ result);  
    }  
}
```

```
}  
}
```

Output:-

```
Max of three numbers = 30
```

3.7 Bitwise Operators

These operators perform operations at the bit level.

- **Bitwise Operators** manipulate individual bits using AND, OR, XOR, and NOT.
- **Shift Operators** move bits to the left or right, effectively multiplying or dividing by powers of two.

Example Code:-

```
import java.io.*;  
  
class Geeks  
{  
    public static void main(String[] args)  
    {  
        // Bitwise operators  
        int d = 0b1010;  
        int e = 0b1100;  
  
        System.out.println("d & e : " + (d & e));  
        System.out.println("d | e : " + (d | e));  
        System.out.println("d ^ e : " + (d ^ e));  
        System.out.println("~d : " + (~d));  
        System.out.println("d << 2 : " + (d << 2));  
        System.out.println("e >> 1 : " + (e >> 1));  
        System.out.println("e >>> 1 : " + (e >>> 1));  
    }  
}
```

Output:-

```
d & e : 8  
d | e : 14  
d ^ e : 6  
~d : -11  
d << 2 : 40  
e >> 1 : 6  
e >>> 1 : 6
```

3.8 instanceof Operator

The **instanceof operator** is used for type checking. It can be used to test if an object is an instance of a class, a subclass, or an interface.

Example Code:-

```
public class GFG {  
    public static void main(String[] args) {  
  
        String str = "Hello";  
        System.out.println(str instanceof String);  
    }  
}
```

```

Object obj = 10;
System.out.println(obj instanceof Integer);
System.out.println(obj instanceof String);
}
}

```

Output:-

```

true
true
false

```

3.9 Common Mistakes to Avoid

The common mistakes that can occur when working with **Java** Operators are listed below:

- **Confusing == with =:** Using == for assignment instead of = for equality check leads to logical errors.
- **Incorrect Use of Floating Point Comparison:** Comparing floating point numbers using == can lead to unexpected results due to precision issues.
- **Integer Division Confusion:** Dividing two integers will result in integer division (truncating the result).
- **Overusing + for String Concatenation in Loops:** Using + for concatenating strings in loops leads to performance issues because it creates new string objects on each iteration.

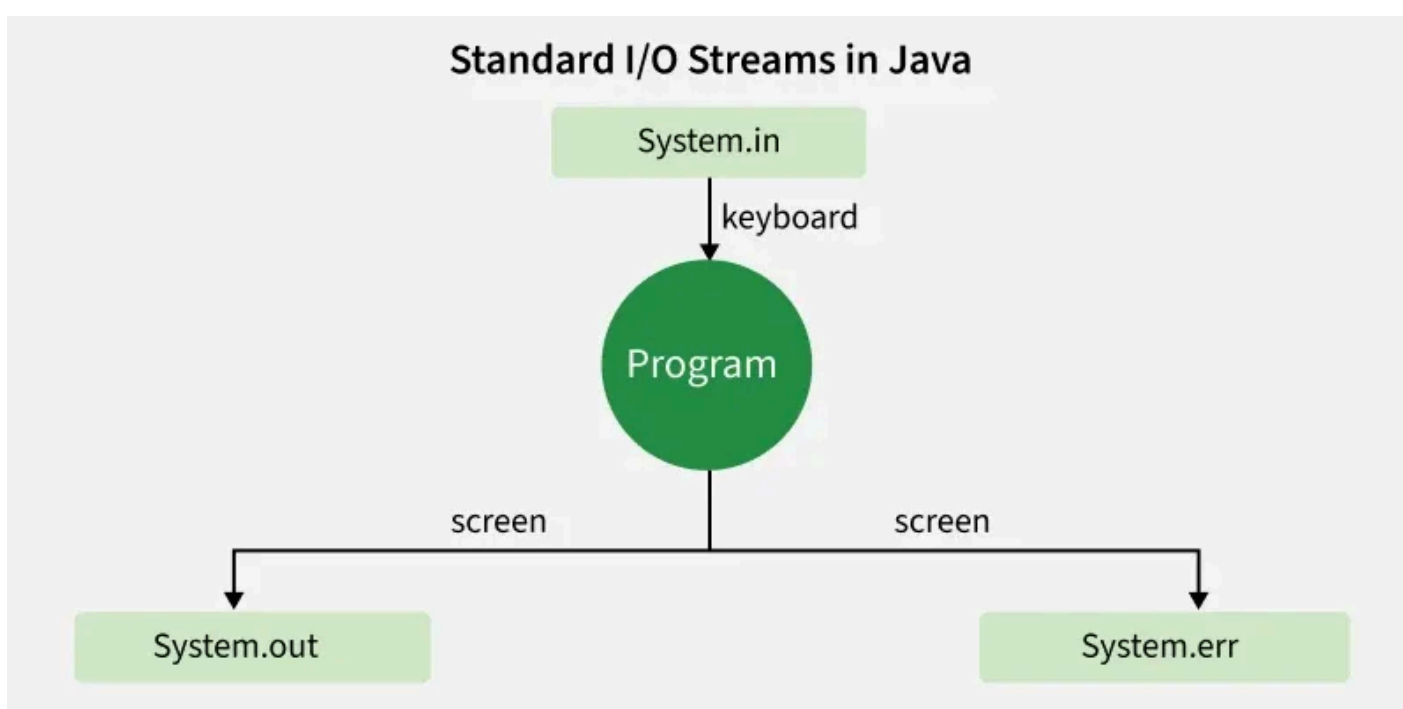
4. Input and Output

Java I/O (Input/Output) is a collection of classes and streams in the [java.io](#) package that handle reading data from sources (like files, keyboard, or network) and writing data to destinations (like files, console or sockets). It provides both byte and character streams to support all types of data.



4.1 Default/Standard Streams in Java

Before exploring various input and output streams, let's look at 3 most commonly used default streams that Java has provided:



- **System.in:** This is the standard input stream that is used to read characters from the keyboard or any other standard input device.

- **System.out:** This is the standard output stream that is used to produce the result of a program on an output device like the computer screen.
- **System.err:** This is the standard error stream that is used to display error messages separately from normal output.

4.2 Functions used with Default Streams

4.2.1 System.in

System.in is an InputStream, so it provides methods from the InputStream class. Commonly used ones are:

- **int read():** reads one byte of data.
- **int read(byte[] b):** reads bytes into an array.
- **int read(byte[] b, int off, int len):** reads bytes into part of an array.
- **void close():** closes the input stream.
- **int available():** returns the number of bytes that can be read without blocking.

Example Code:-

```
import java.io.IOException;

public class SystemInExample {
    public static void main(String[] args) throws IOException {
        System.out.println("Enter a character:");

        // Reads a single byte from System.in
        int data = System.in.read();

        // Print the character and its ASCII value
        System.out.println("You entered: " + (char) data);
        System.out.println("ASCII Value: " + data);
    }
}
```

Output:-

```
Enter a character:
a
You entered: a
ASCII Value: 97
```

4.2.2 System.out

print(): This Java method displays text on the console, keeping the cursor at the end of the printed text so the next output continues from the same line.

Example Code:-

```
import java.io.*;

class Geeks {
    public static void main(String[] args)
    {
        // using print() all are printed in the same line
        System.out.print("GfG! ");
        System.out.print("GfG! ");
        System.out.print("GfG! ");
    }
}
```

Output:-

```
GfG! GfG! GfG!
```

println(): This method in Java is also used to display a text on the console. It prints the text on the console and the cursor moves to the start of the next line at the console. The next printing takes place from the next line.

Example Code:-

```
import java.io.*;

class Geeks {
    public static void main(String[] args)
    {
        // using println() all are printed in the different line
        System.out.println("GfG! ");
        System.out.println("GfG! ");
        System.out.println("GfG! ");
    }
}
```

Output:-

```
GfG!
GfG!
GfG!
```

printf(): The printf() method in Java is used for formatted output and can take multiple arguments, making it more flexible than print() or println().

Example Code:-

```
class Geeks {

    public static void main(String[] args) {
        int x = 100;

        // Printing a simple integer
        System.out.printf("Printing simple integer: x = %d\n", x);

        // Printing a floating-point number with precision
        System.out.printf("Formatted with precision: PI = %.2f\n", Math.PI);

        float n = 5.2f;

        // Formatting a float to 4 decimal places
        System.out.printf("Formatted to specific width: n = %.4f\n", n);

        n = 2324435.3f;

        // Right-aligning and formatting a float to 20-character width
        System.out.printf("Formatted to right margin: n = %20.4f\n", n);
    }
}
```

Output:-

```
Printing simple integer: x = 100
Formatted with precision: PI = 3.14
```



```
Formatted to specific width: n = 5.2000
Formatted to right margin: n = 2324435.2500
```

4.2.3 System.err

It is used to display the error messages. It works similarly to System.out with print(), println(), and printf() methods.

Example Code:-

```
public class Geeks {

    public static void main(String[] args) {

        // Using print()
        System.err.print("This is an error message using print().\n");

        // Using println()
        System.err.println("This is another error message using println().");

        //Using printf()
        System.err.printf("Error code: %d, Message: %s\n", 404, "Not Found");

    }
}
```

Output:-

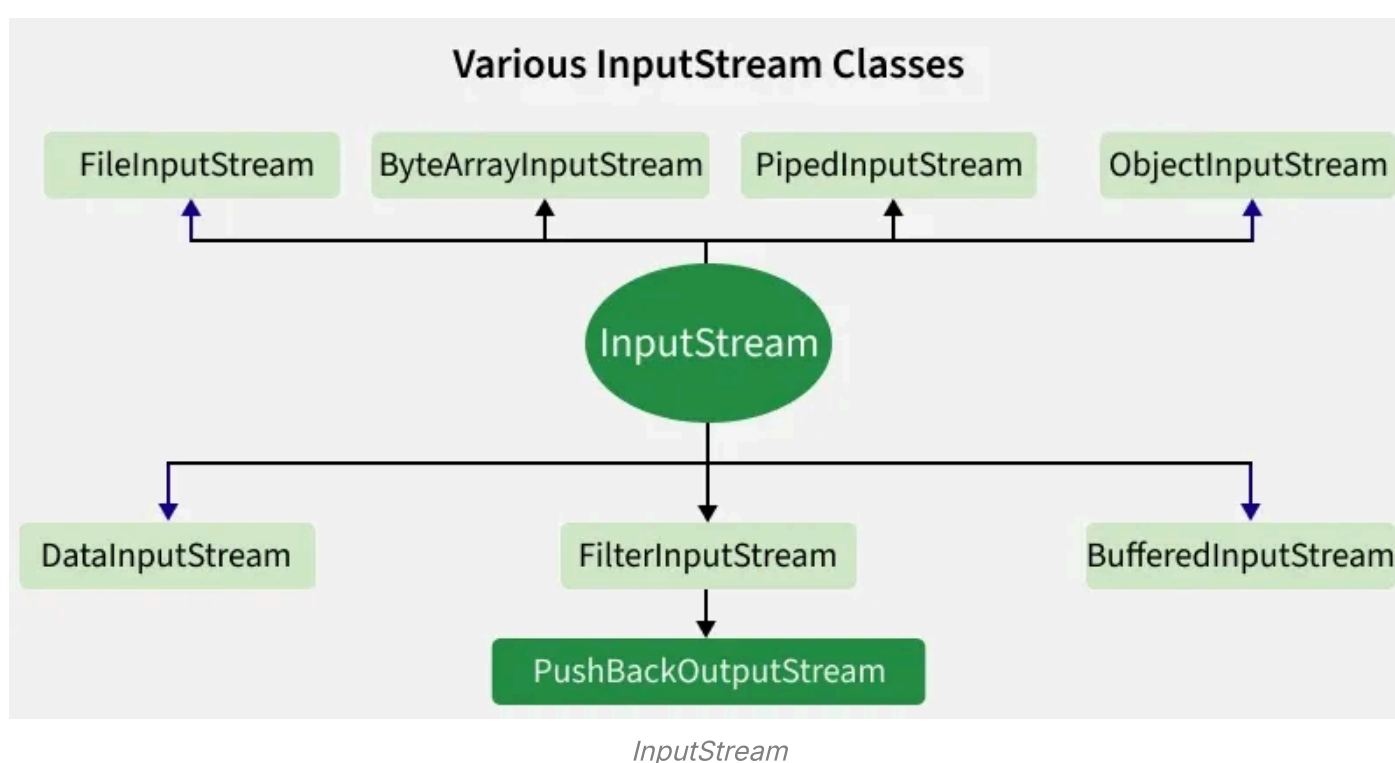
```
This is an error message using print().
This is another error message using println().
Error code: 404, Message: Not Found
```

4.2 Types of Streams

Depending on the type of operations, streams can be divided into two primary classes:

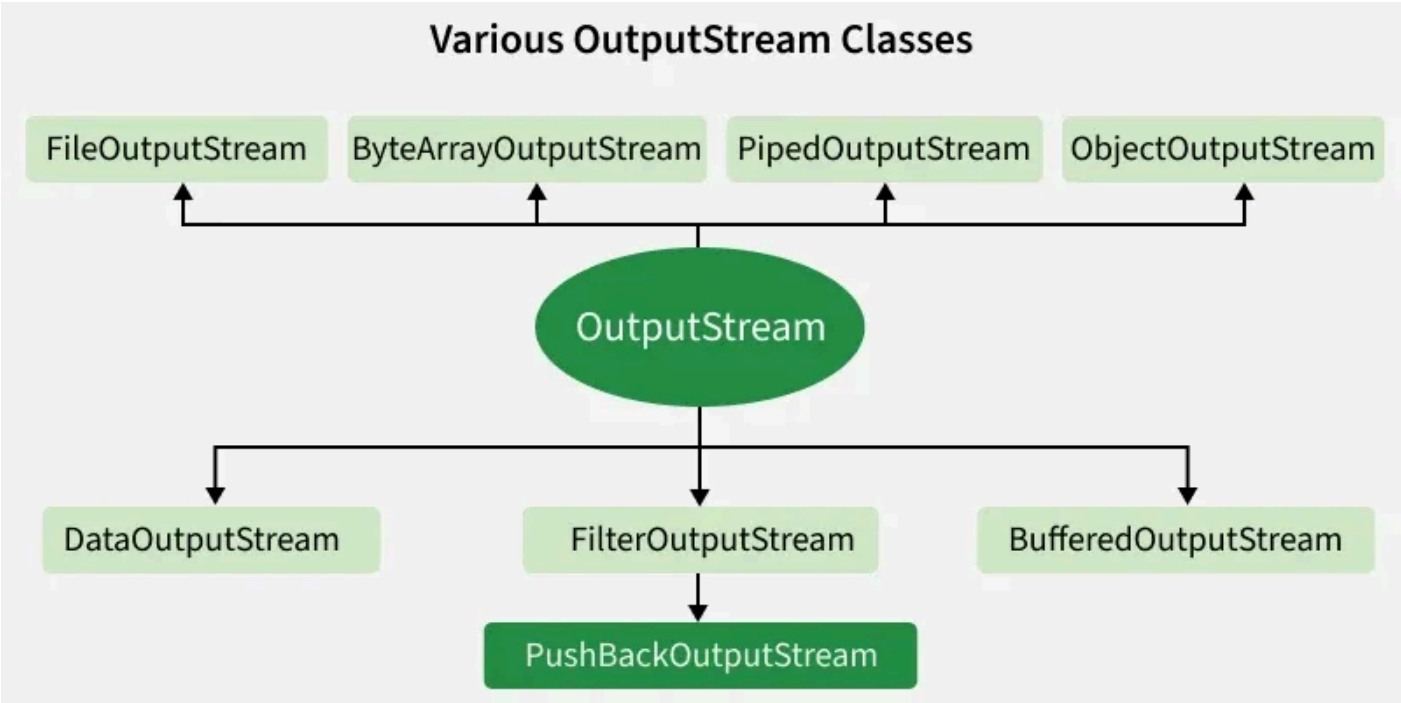
4.2.1 Input Stream

These streams are used to read data that must be taken as an input from a source array or file or any peripheral device. For eg., FileInputStream, BufferedInputStream, ByteArrayInputStream etc.



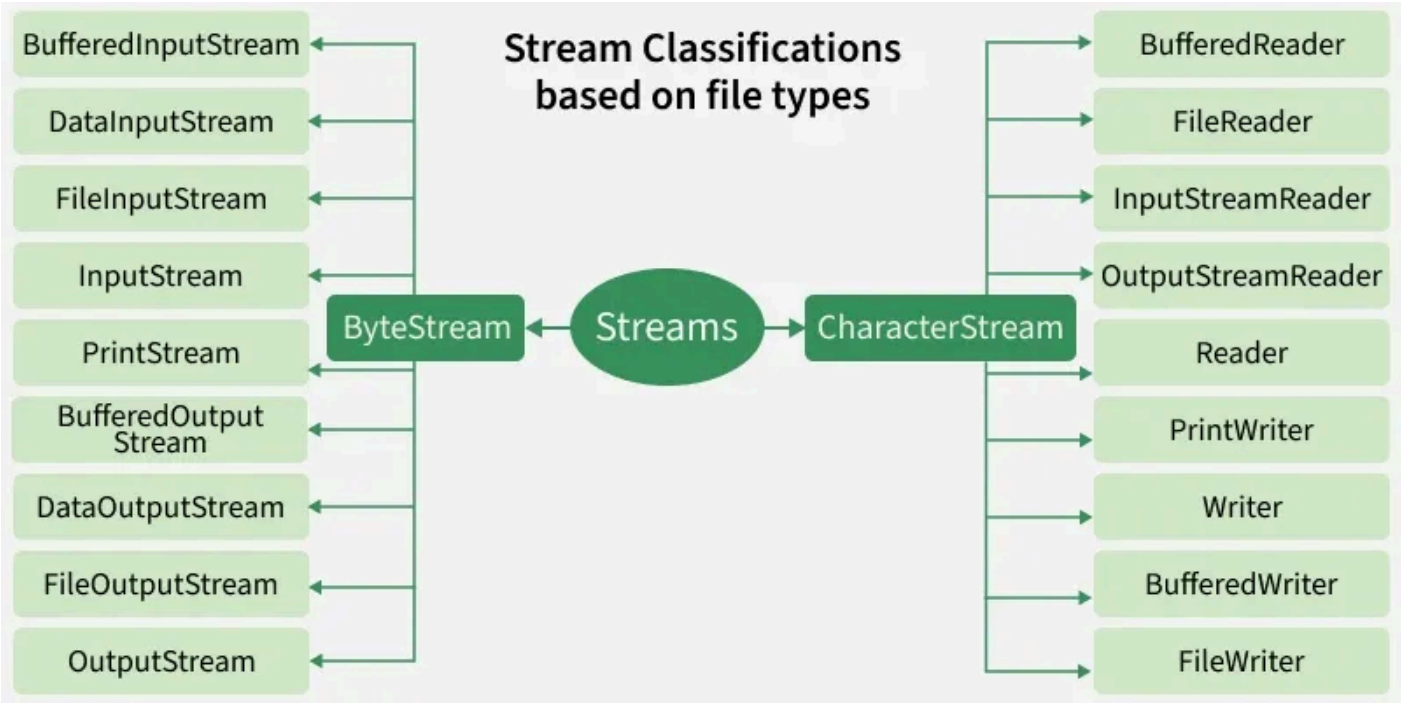
4.2.2 Output Stream

These streams are used to write data as outputs into an array or file or any output peripheral device. For eg., FileOutputStream, BufferedOutputStream, ByteArrayOutputStream etc.



4.3 Types of Streams Depending on type of data (byte vs character

Depending on the types of file, Streams can be divided into two primary classes which can be further divided into other classes as can be seen through the diagram below followed by the explanations.



4.3.1 ByteStream

Byte streams in Java are used to perform input and output of 8-bit bytes. They are suitable for handling raw binary data such as images, audio, and video, using classes like InputStream and OutputStream.

Stream class	Description
<u>BufferedInputStream</u>	Used to read data more efficiently with buffering.
<u>DataInputStream</u>	Provides methods to read Java primitive data types.
<u>FileInputStream</u>	This is used to read from a file.
<u>InputStream</u>	This is an abstract class that describes stream input.
<u>PrintStream</u>	This contains the most used print() and println() method
<u>BufferedOutputStream</u>	This is used for Buffered Output Stream.
<u>DataOutputStream</u>	This contains method for writing java standard data types.
<u>FileOutputStream</u>	This is used to write to a file.
<u>OutputStream</u>	This is an abstract class that describes stream output.

Example Code:-

```
import java.io.*;
public class Geeks {
    public static void main(
        String[] args) throws IOException
```

```
{

    FileInputStream sourceStream = null;
    FileOutputStream targetStream = null;

    try {
        sourceStream
            = new FileInputStream("sourcefile.txt");
        targetStream
            = new FileOutputStream("targetfile.txt");

        // Reading source file and writing content to target file byte by byte
        int temp;
        while ((
            temp = sourceStream.read())
            != -1)
            targetStream.write((byte)temp);
    }
    finally {
        if (sourceStream != null)
            sourceStream.close();
        if (targetStream != null)
            targetStream.close();
    }
}
}
```

4.3.2 CharacterStream

Character streams in Java are used to perform input and output of 16-bit Unicode characters. They are best suited for handling text data, using classes like Reader and Writer which automatically handle character encoding and decoding.

Stream class	Description
<u>BufferedReader</u>	It is used to handle buffered input stream.
<u>FileReader</u>	This is an input stream that reads from file.
<u>InputStreamReader</u>	This input stream is used to translate byte to character.
<u>OutputStreamWriter</u>	Converts character stream to byte stream.
<u>Reader</u>	This is an abstract class that define character stream input.
<u>PrintWriter</u>	This contains the most used print() and println() method
<u>Writer</u>	This is an abstract class that define character stream output.
<u>BufferedWriter</u>	This is used to handle buffered output stream.
<u>FileWriter</u>	This is used to output stream that writes to file.

Example Code:-

```
import java.io.*;

public class Geeks
{
    public static void main(String[] args) throws IOException
    {
        FileReader sourceStream = null;

        try {
            sourceStream = new FileReader("test.txt");

            // Reading sourcefile character by character.
```

```

    int temp;

    while (( temp = sourceStream.read()) != -1 )
        System.out.println((char)temp);
}
finally {

    // Closing stream as no longer in use
    if (sourceStream != null)
        sourceStream.close();

}
}
}

```

5. Control Structures

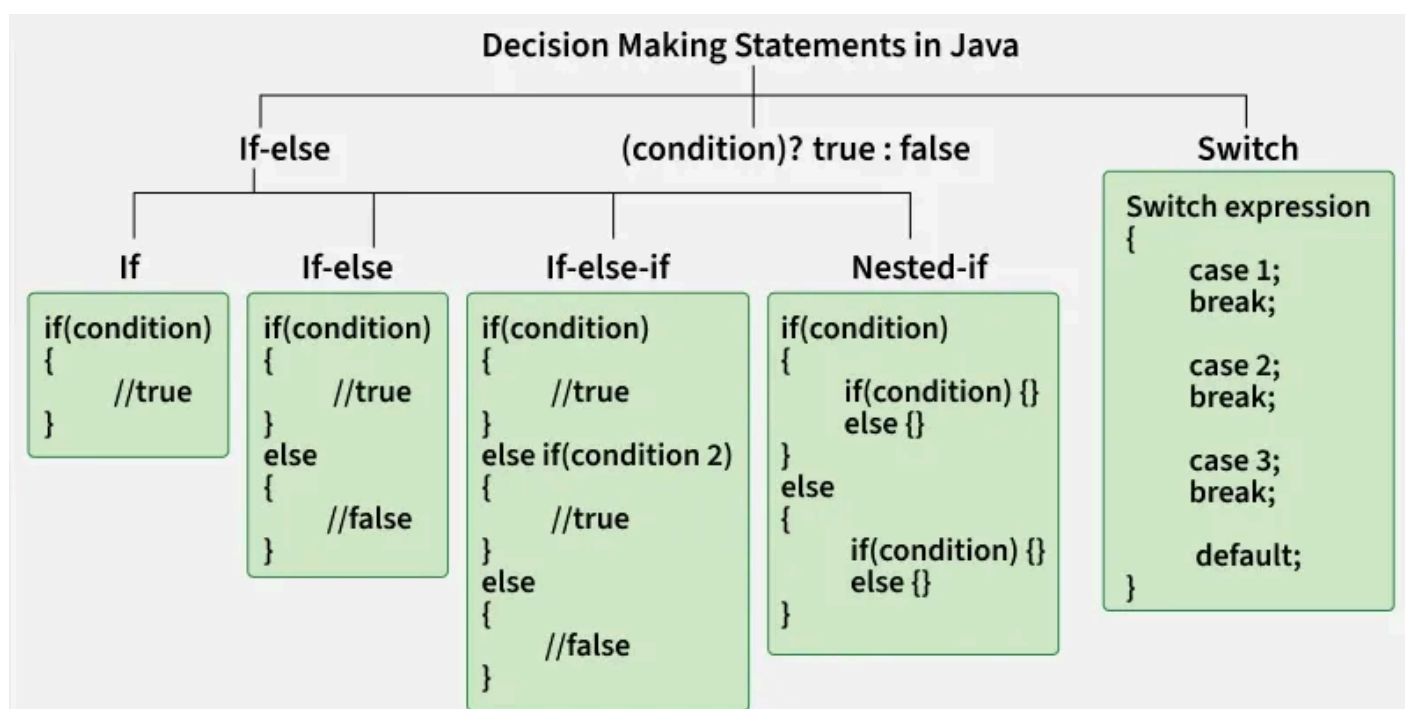
5.1 Decision Making Statements

Decision-making statements in Java allow a program to choose different execution paths based on specified conditions. They help control the flow of a program by executing certain blocks of code only when particular conditions are true. These statements are essential for implementing logic, validations, and user-driven actions in applications.

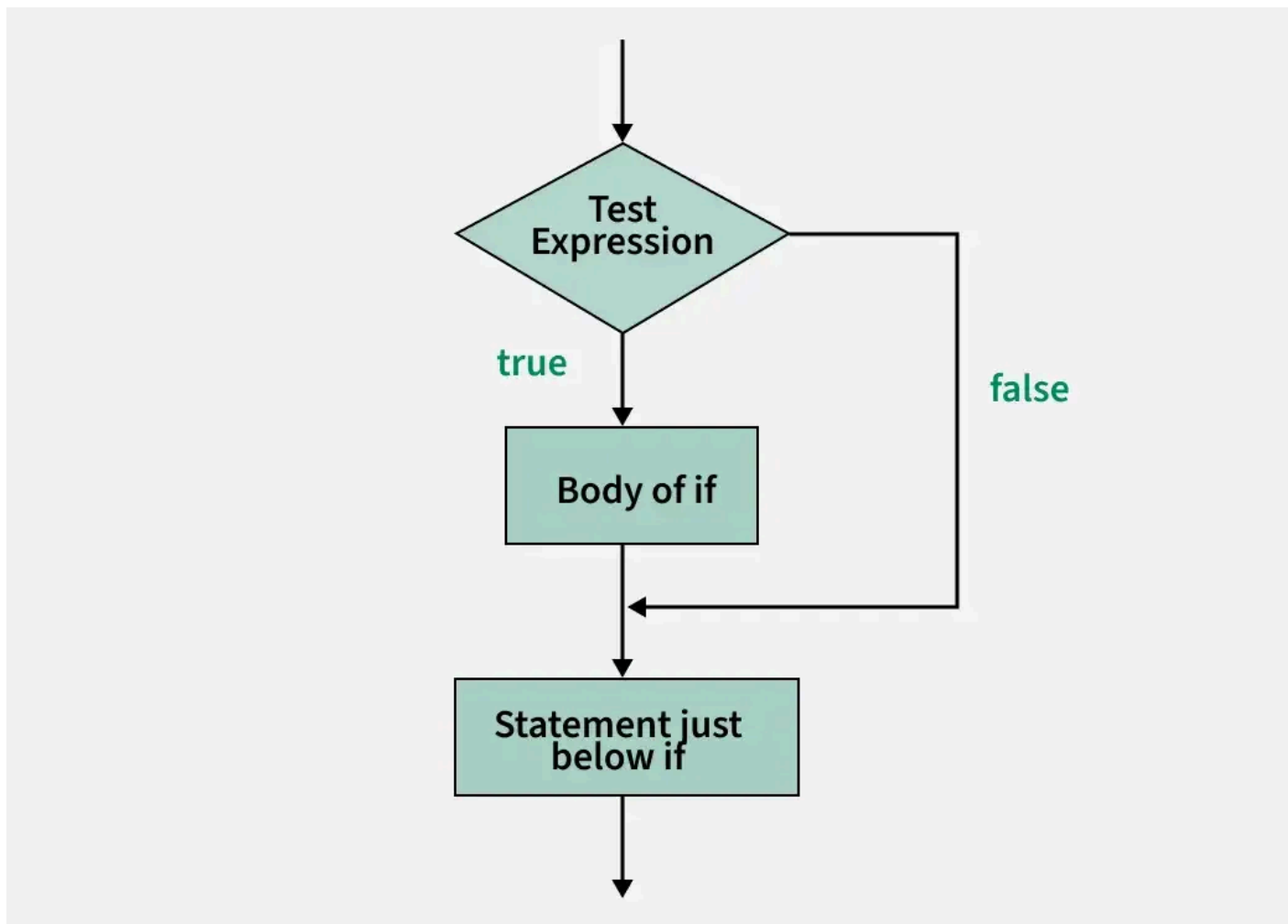
- Improve flexibility and intelligence of programs.
- Support simple, multiple, and nested decision-making scenarios.

5.1.1 Types of Decision-Making Statements

Java provides the following decision-making statements:



5.1.2 Java if Statement



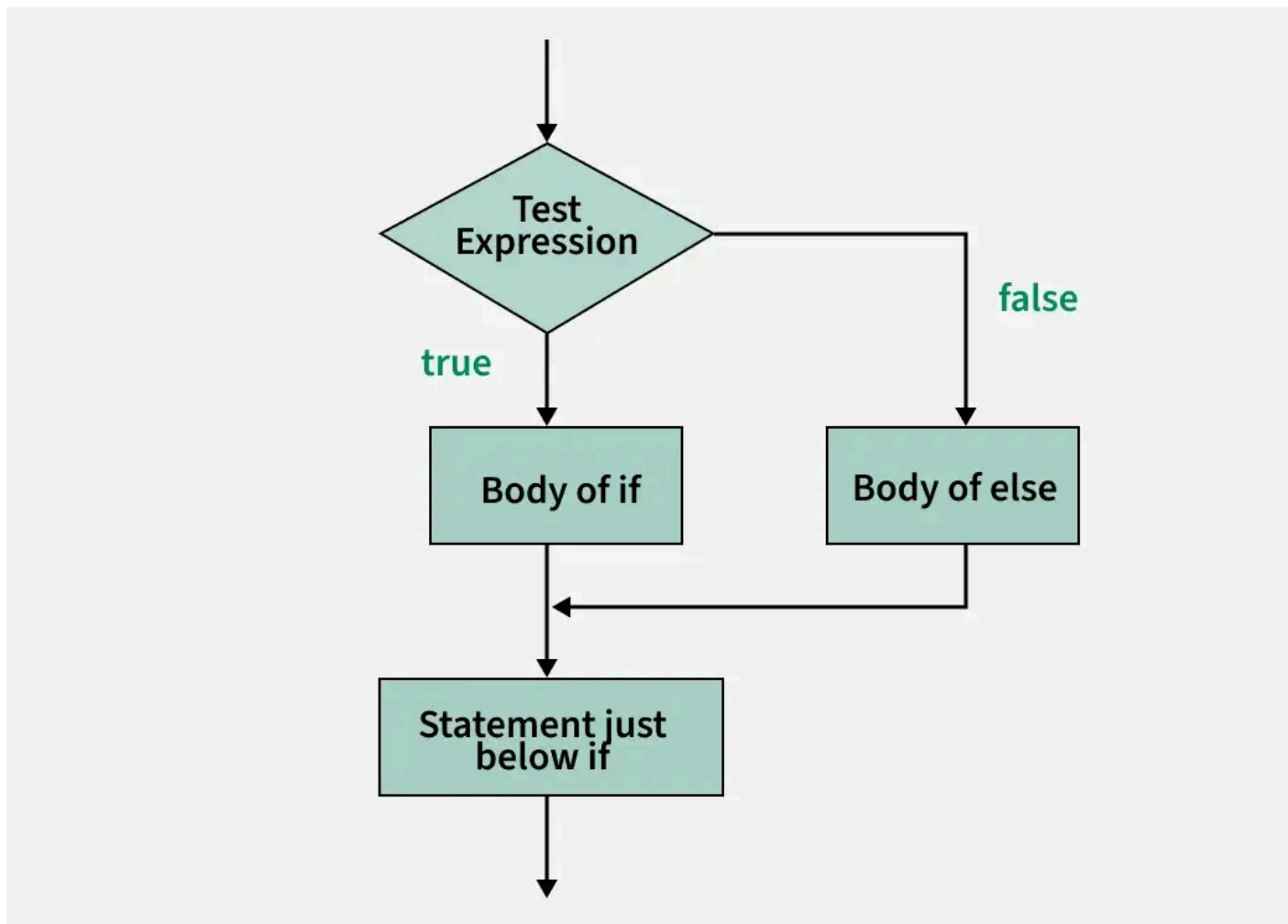
Example Code:-

```
class Geeks {  
    public static void main(String args[])  
    {  
        int i = 10;  
  
        if (i < 15) {  
            System.out.println("Condition is True");  
        }  
    }  
}
```

Output:-

```
Condition is True
```

5.1.3 Java if-else Statement



Example Code:-

```
import java.util.*;

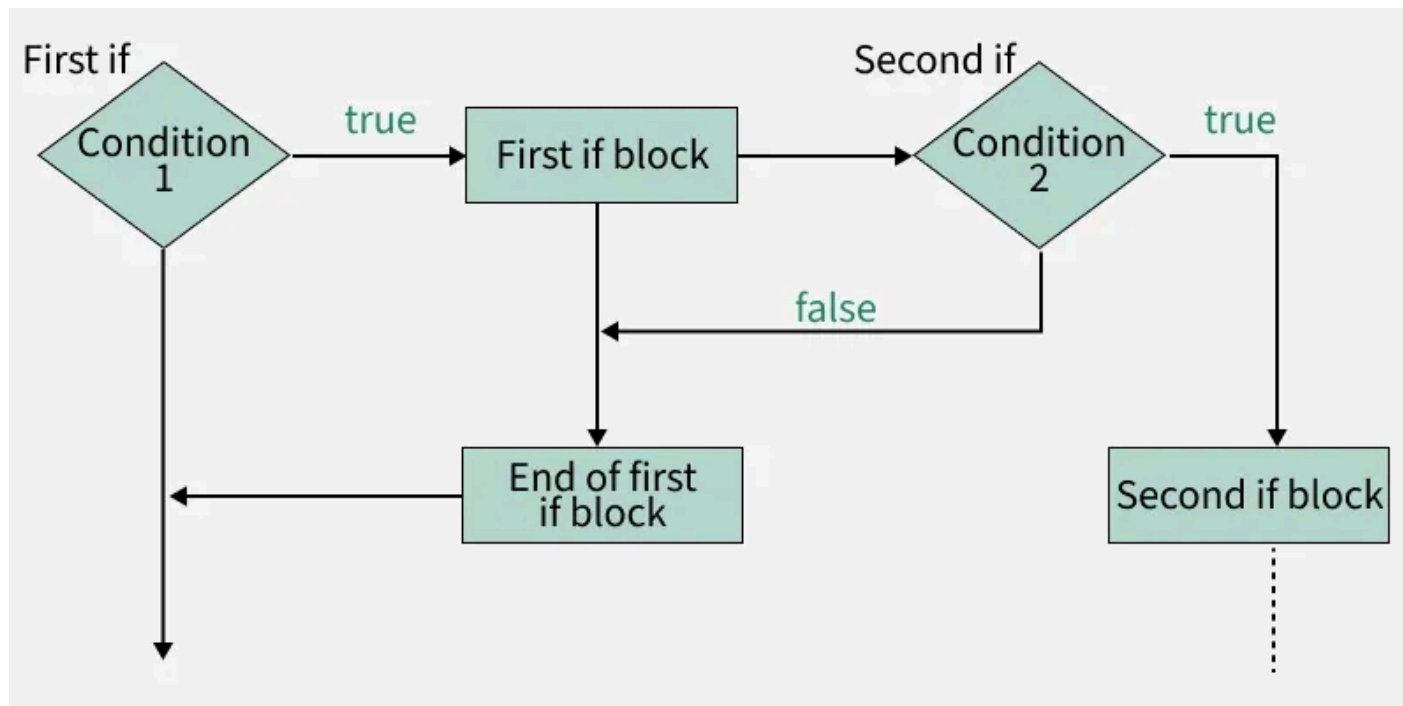
class Geeks {
    public static void main(String args[])
    {
        int i = 10;

        if (i < 15)
            System.out.println("i is smaller than 15");
        else
            System.out.println("i is greater than 15");
    }
}
```

Output:-

```
i is smaller than 15
```

5.1.4 Java nested-if Statement



Example Code:-

```

class Geeks {
    public static void main(String args[])
    {
        int i = 10;

        // Outer if statement
        if (i < 15) {
            System.out.println("i is smaller than 15");

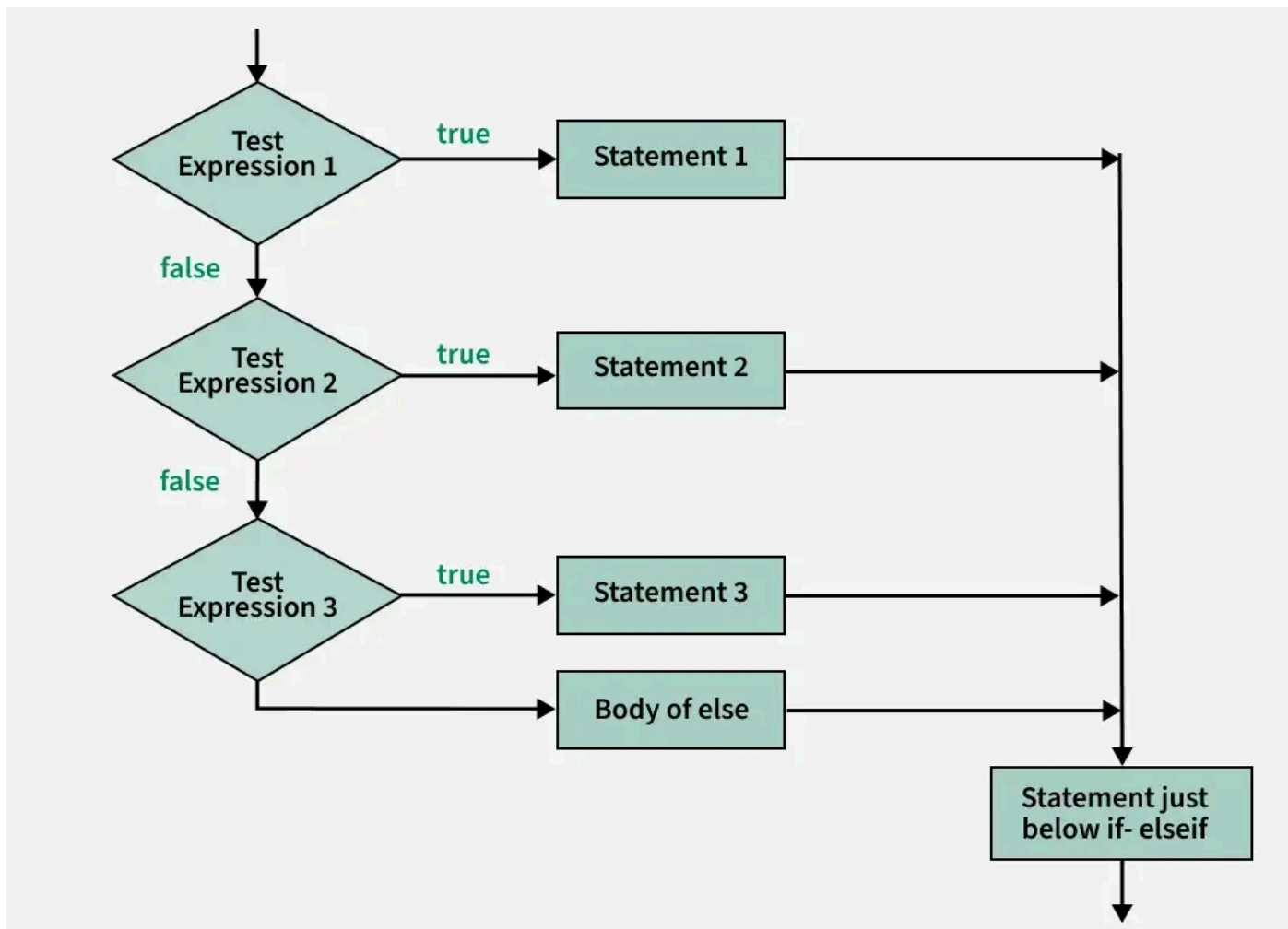
            // Nested if statement
            if (i == 10) {
                System.out.println("i is exactly 10");
            }
        }
    }
}
  
```

Output:-

```

i is smaller than 15
i is exactly 10
  
```

5.1.5 Java if-else-if ladder



Example Code:-

```
import java.util.*;

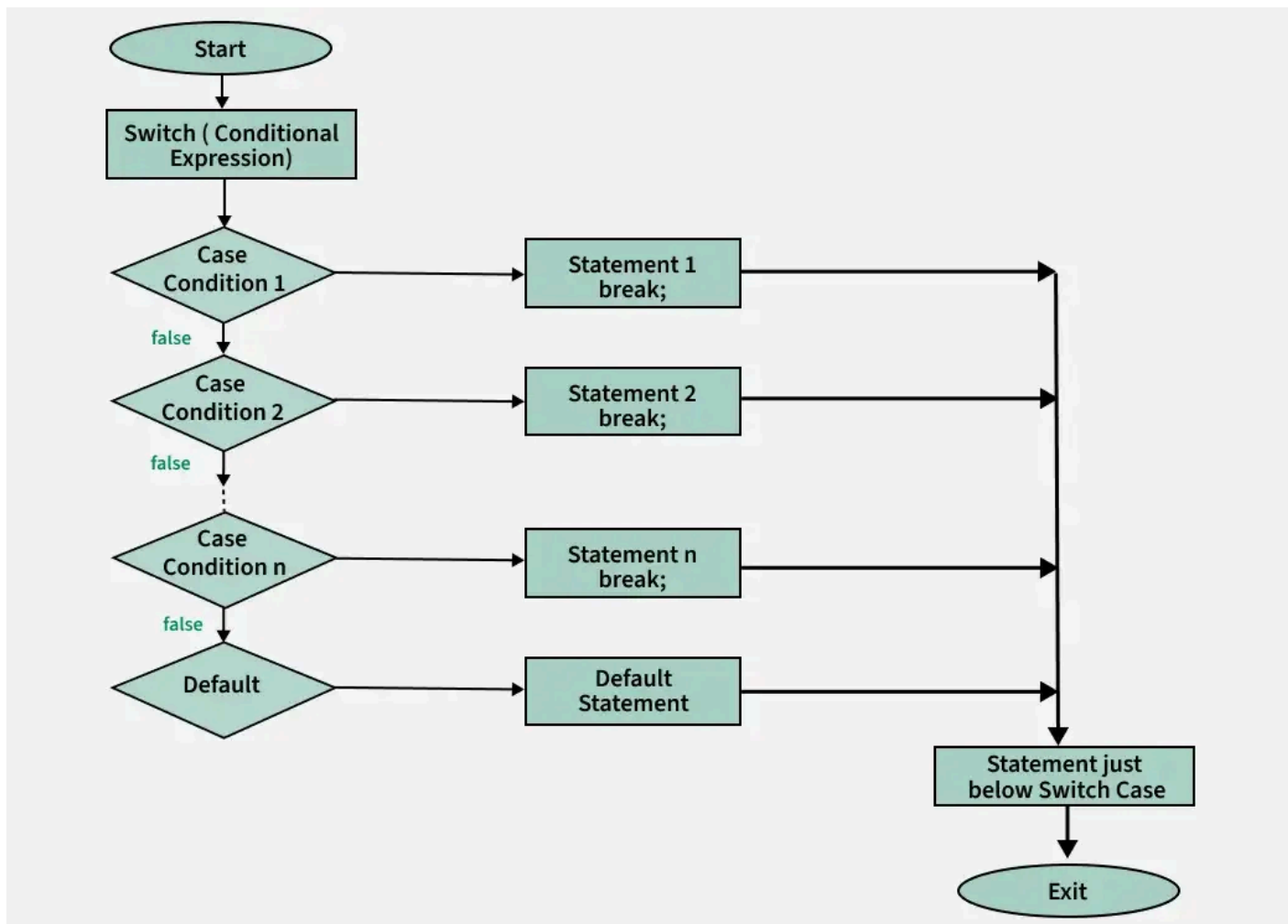
class Geeks {
    public static void main(String args[])
    {
        int i = 20;

        if (i == 10)
            System.out.println("i is 10");
        else if (i == 15)
            System.out.println("i is 15");
        else if (i == 20)
            System.out.println("i is 20");
        else
            System.out.println("i is not present");
    }
}
```

Output:-

```
i is 20
```

5.1.6 Java Switch Case



Example Code:-

```

import java.io.*;

class Geeks {
    public static void main(String[] args)
    {
        int num = 20;
        switch (num) {
            case 5:
                System.out.println("It is 5");
                break;
            case 10:
                System.out.println("It is 10");
                break;
            case 15:
                System.out.println("It is 15");
                break;
            case 20:
                System.out.println("It is 20");
                break;
            default:
                System.out.println("Not present");
        }
    }
}

```

Output:-

```
It is 20
```

5.1.7 Ternary Operator (? :) in Java

The ternary operator in Java is a conditional operator that provides a shorthand way to write simple if-else statements

Syntax

```
condition ? expression_if_true : expression_if_false;
```

Example Code:-

```
class Geeks {
    public static void main(String args[]) {
        int a = 10, b = 20;
        int max = (a > b) ? a : b;

        System.out.println("Maximum is " + max);
    }
}
```

Output:-

```
Maximum is 20
```

5.1.8 if-else vs switch-case

The table below demonstrates the difference between if-else and switch-case.

Features	if-else	switch-case
Use Case	Suitable for condition-based checks	Best for exact value matching
Readability	More readable for a few conditions	More readable and efficient for many cases
Performance	Slower for many checks due to multiple conditions	Faster and optimized for handling many cases
Flexibility	Supports ranges and complex conditions	Only supports exact matches of values

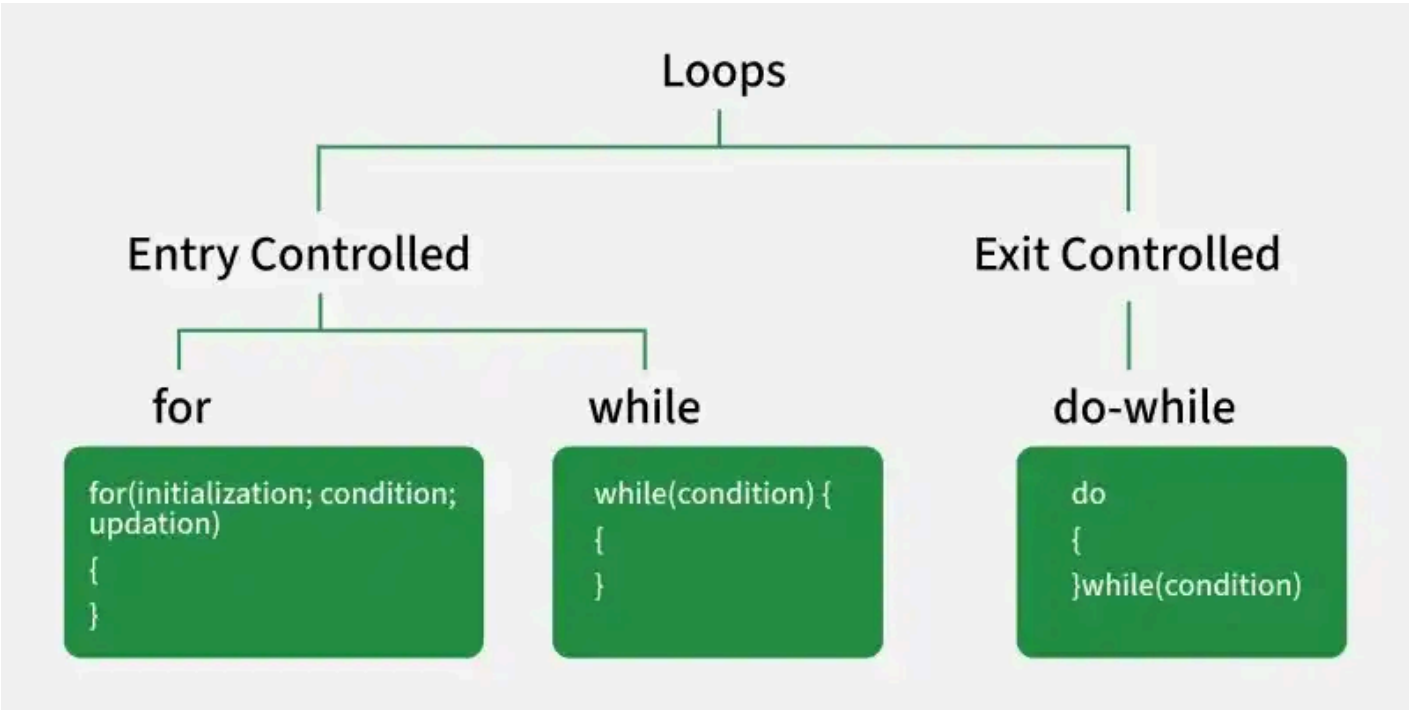
5.2 Loops

Loops in programming are used to execute a block of code repeatedly until a specified condition is met. They help reduce code duplication and make programs more efficient and readable. In **Java**, loops are essential for handling repetitive tasks.

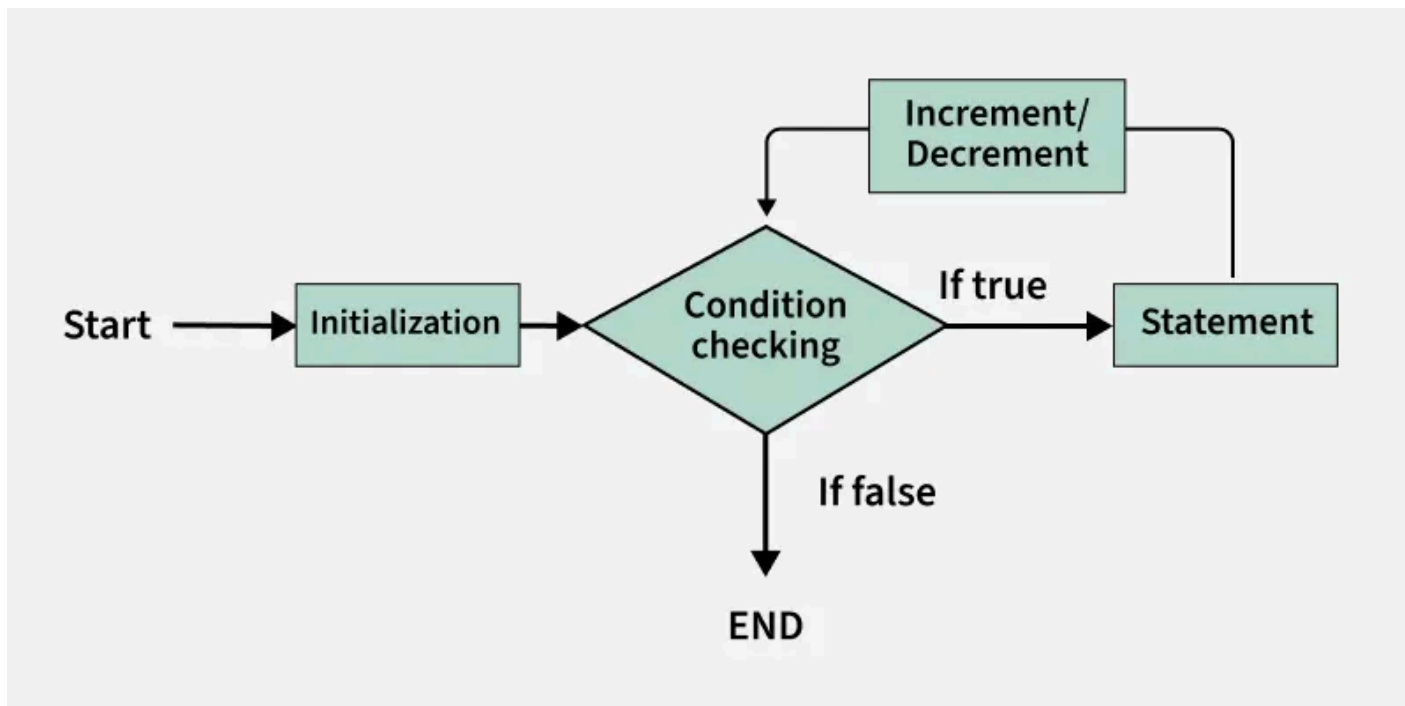
- Allow execution of code multiple times based on a condition
- Improve efficiency by avoiding repetitive code

5.2.1 Types of Loops in java

Java provides different types of loops that allow a block of code to be executed repeatedly based on a specified condition.



5.2.2 for loop



Example Code:-

```

import java.io.*;

class Geeks {
    public static void main(String[] args)
    {
        for (int i = 0; i <= 10; i++) {
            System.out.print(i + " ");
        }
    }
}

```

Output:-

```

0 1 2 3 4 5 6 7 8 9 10

```

5.2.3 Enhanced for loop (for each)

Example Code:-

```

import java.io.*;

class Geeks {
    public static void main(String[] args)
    {
        String[] names = { "Sweta", "Gudly", "Amiya" };

        for (String name : names) {
            System.out.println("Name: " + name);
        }
    }
}

```

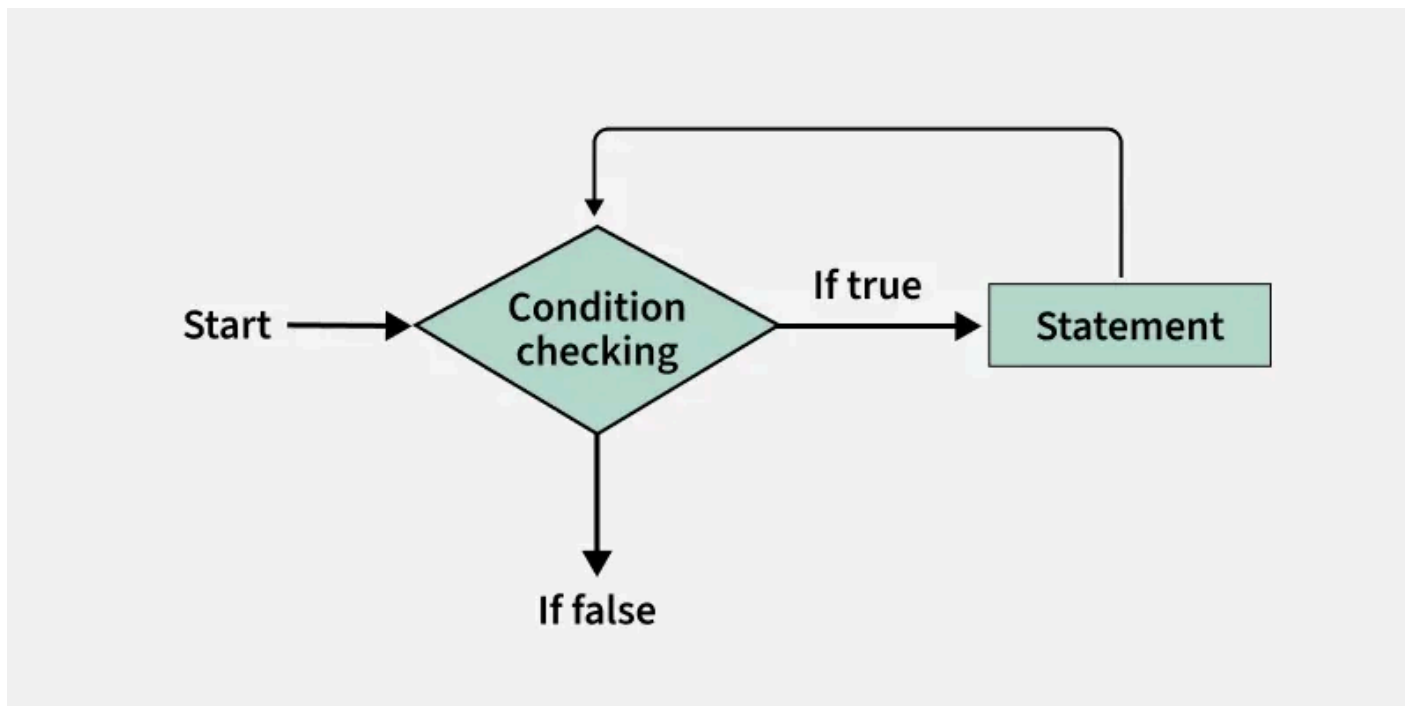
Output:-

```

Name: Sweta
Name: Gudly
Name: Amiya

```

5.2.4 while Loop



Example Code:-

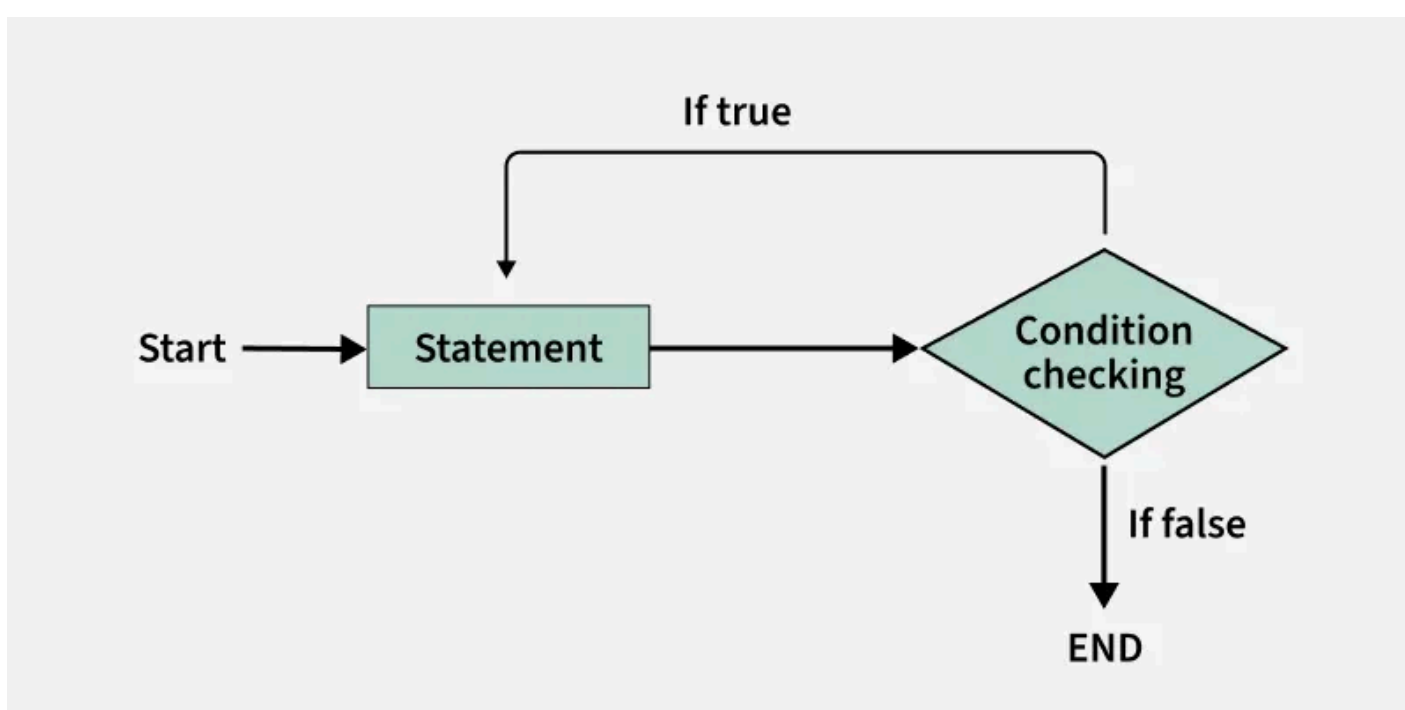
```
import java.io.*;

class Geeks {
    public static void main(String[] args)
    {
        int i = 0;
        while (i <= 10) {
            System.out.print(i + " ");
            i++;
        }
    }
}
```

Output:-

```
0 1 2 3 4 5 6 7 8 9 10
```

5.2.5 do-while Loop



Example Code:-

```
import java.io.*;

class Geeks {
```



```
public static void main(String[] args)
{
    int i = 0;
    do {
        System.out.print(i + " ");
        i++;
    } while (i <= 10);
}
```

Output:-

```
0 1 2 3 4 5 6 7 8 9 10
```

5.2.6 Common Loop Mistakes and How to Avoid them

If loops are not used correctly, they can introduce pitfalls and bugs that affect code performance, readability, and functionality. Below are some common pitfalls of loops:

A. Infinite Loops

This is one of the most common mistakes while implementing any sort of looping is that it may not ever exit, that is the loop runs for infinite time. This happens when the condition fails for some reason.

Types of Infinite Loops:

- infinite for Loop
- infinite while Loop

B. Off-by-One Errors

Off-by-One Errors are caused when the loop runs one more or one fewer time than you wanted. It basically happens when the loop condition is not set correctly.

C. Modifying Loop Variables Inside the Loop

When we change the loop condition (like i) inside the loop, it can cause the loop to skip certain iterations or behave in ways that we did not expected. This might leads to errors or unexpected behavior.

D. Empty Loop Body

An empty loop body occurs when a loop is written to iterate but does not perform any operations inside the loop. Running a loop without any useful operations inside it can be confusing.

Summary Table

Loop Type	When to Use	Condition Checking	Executes At Least Once?
for loop	When you want exact iterations	Before loop body, It is called Entry-controlled.	no
while loop	When you need condition check first.	Before loop body, It is called Entry-controlled.	no
do-while loop	When you need to run at least once	After loop body, It is called Exit-controlled.	yes
for-each loop	When you process all collection items	Internally handled	no

5.2.7 A Nested loop

A Nested loop means a **loop statement** inside another loop statement. That is why nested loops are also called "**loop inside loop**".

A. Nested For loop

```

for ( initialization; condition; increment ) {
    for ( initialization; condition; increment ) {
        // statement of inside loop
    }
    // statement of outer loop
}

```

B. Nested While loop

```

while(condition) {
    while(condition) {
        // statement of inside loop
    }
    // statement of outer loop
}

```

C. Nested Do-While loop

```

do{
    do{
        // statement of inside loop
    }
    while(condition);
    // statement of outer loop
}
while(condition);

```

Note: There is no rule that a loop must be nested inside its own type. In fact, there can be any type of loop nested inside any type and to any level.

D. Hybrid Java Nested Loops

```

do{
    while(condition) {
        for ( initialization; condition; increment ) {
            // statement of inside for loop
        }
        // statement of inside while loop
    }
    // statement of outer do-while loop
}while(condition);

```

Eg1 :- Below program uses a nested for loop to print a 2D matrix.

```

// Java program to print the elements of
// a 2 D array or matrix
import java.io.*;

// Driver Class
class GFG {
    public static void print2D(int mat[][])
    {
        // Loop through all rows
        for (int i = 0; i < mat.length; i++) {

```

```

        // Loop through all elements of current row
        for (int j = 0; j < mat[i].length; j++)
            System.out.print(mat[i][j] + " ");

        System.out.println();
    }
}

// main function
public static void main(String args[]) throws IOException
{
    int mat[][] = { { 1, 2, 3, 4 },
                    { 5, 6, 7, 8 },
                    { 9, 10, 11, 12 } };

    print2D(mat);
}
}

```

Output:-

```

1 2 3 4
5 6 7 8
9 10 11 12

```

Eg2:- Below program uses a nested for loop to print all prime factors of a number.

```

// Java program to print all prime factors
import java.io.*;
import java.lang.Math;

// Driver Class
class GFG {
    // A function to print all prime factors
    // of a given number n
    public static void primeFactors(int n)
    {
        // Print the number of 2s that divide n
        while (n % 2 == 0) {
            System.out.print(2 + " ");
            n /= 2;
        }

        // n must be odd at this point. So we can
        // skip one element (Note i = i +2)
        for (int i = 3; i <= Math.sqrt(n); i += 2) {
            // While i divides n, print i and divide n
            while (n % i == 0) {
                System.out.print(i + " ");
                n /= i;
            }
        }

        // This condition is to handle the case when
        // n is a prime number greater than 2
        if (n > 2)
            System.out.print(n);
    }
}

```

```

// main function
public static void main(String[] args)
{
    int n = 315;
    primeFactors(n);
}
}

```

Output:-

```

3 3 5 7

```

6. Functions (Methods)

Java Methods are blocks of code that perform a specific task. A method allows us to reuse code, improving both efficiency and organization. All methods in Java must belong to a class. Methods are similar to functions and expose the behavior of objects.

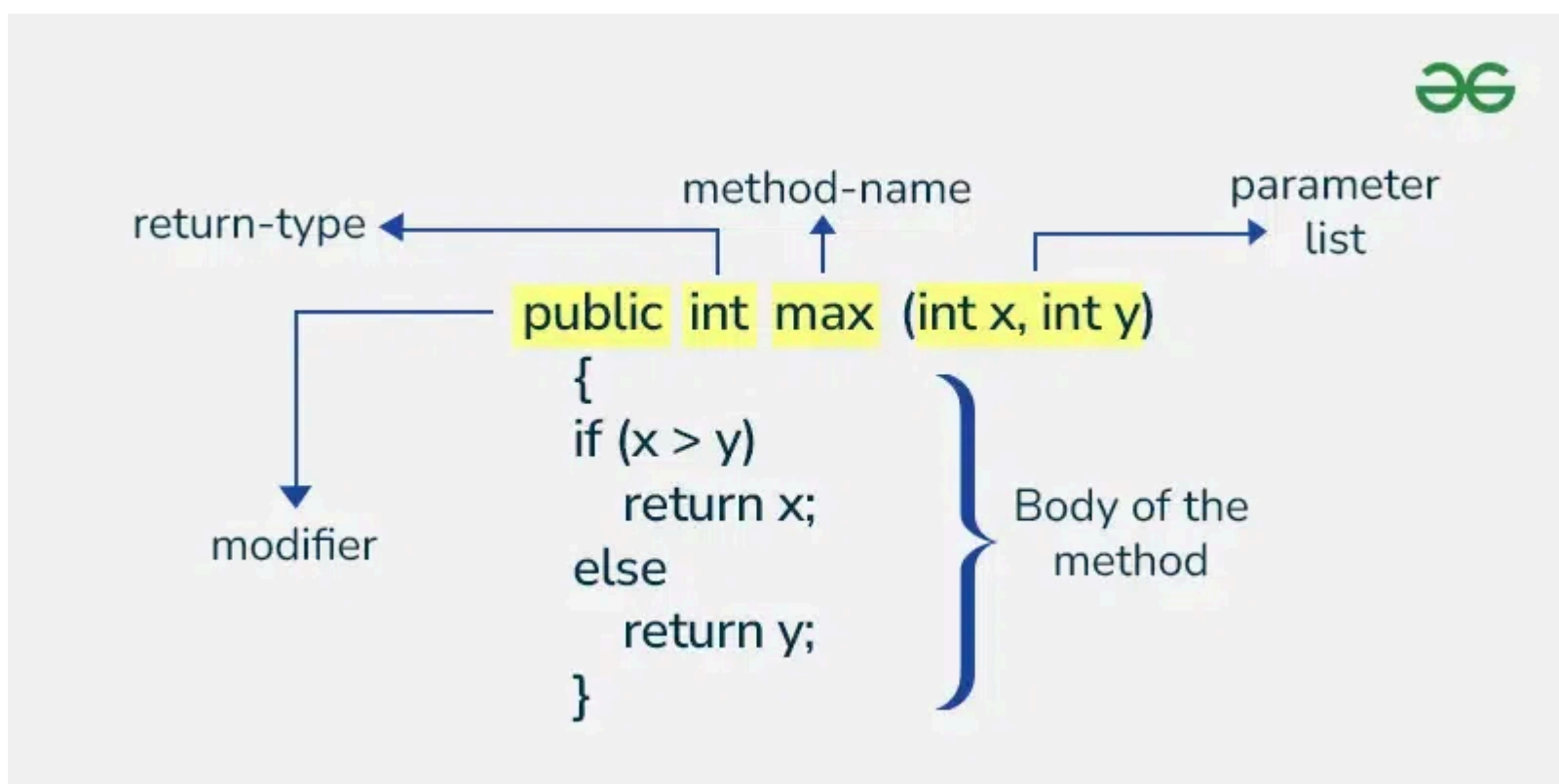
- A method allows to write a piece of logic once and reuse it wherever needed in the program.
- This helps keep your code clean, organized, easier to understand and manage.

Syntax of Java Methods

```

returnType methodName(parameters) {
    // method body
    return value; // optional (only if returnType is not void)
}

```

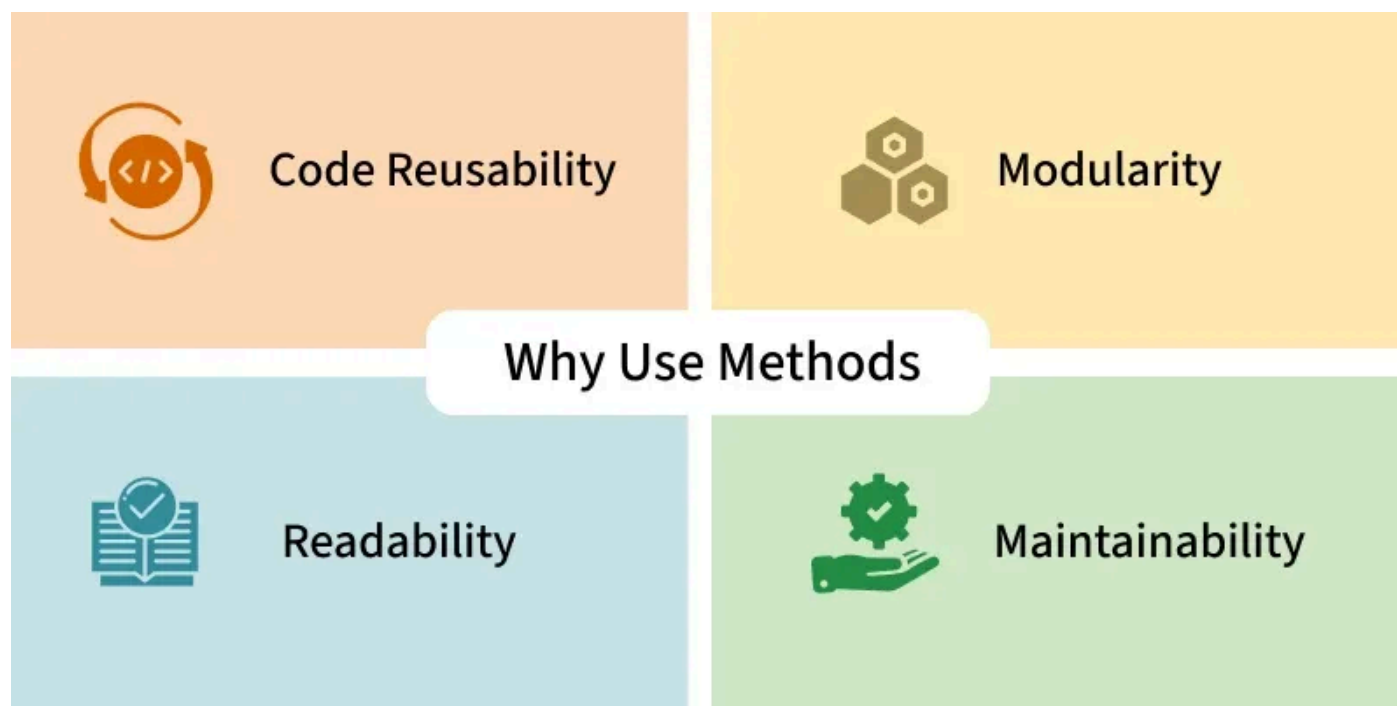


Key Components of a Method Declaration:

- **Modifier:** Defines access level (public, private, protected, default)
- **Return Type:** Specifies what value the method returns or `void` if no return
- **Method Name:** Name of the method, follows camelCase convention
- **Parameters:** Optional inputs passed to the method
- **Method Body:** Contains the logic or statements to be executed

6.1 Why Use Methods?

Breaking code into separate methods helps improve readability, reusability, and maintainability



- **Code Reusability:** Write once, use multiple times without repeating code so that code reusability increase.
- **Modularity:** Dividing a program into separate methods allows each method to handle a specific task, making the code more organized and easier to manage.
- **Readability:** Smaller, named methods make the code easier to read and understand.
- **Maintainability:** It's easier to fix bugs or update code when it's organized into methods.

6.2 Method Call Stack in Java

Java is an object-oriented and stack-based programming language where methods play a key role in controlling the program's execution flow. When a method is called, Java uses an internal structure known as the call stack to manage execution, variables, and return addresses.

6.2.1 Call Stack

The call stack is a data structure used by the program during runtime to manage method calls and local variables. It operates in a Last-In-First-Out (LIFO) manner, meaning the last method called is the first one to complete and exit.

6.2.2 Methods Execution

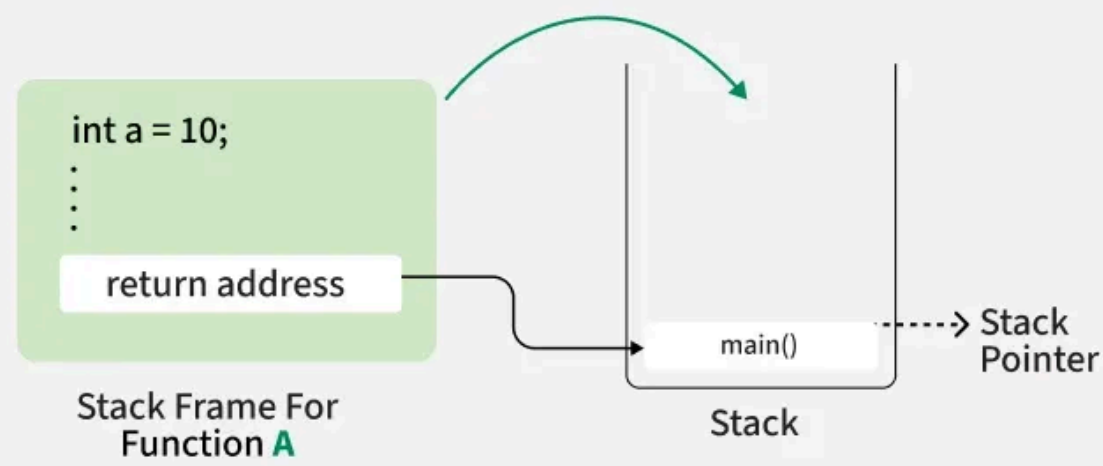
When a method is called:

- A new stack frame is added to the call stack to store method details.
- The method runs its code.
- After execution, the stack frame is removed, and control goes back to the calling method.

Java automatically manages the call stack using the Java Virtual Machine (JVM).

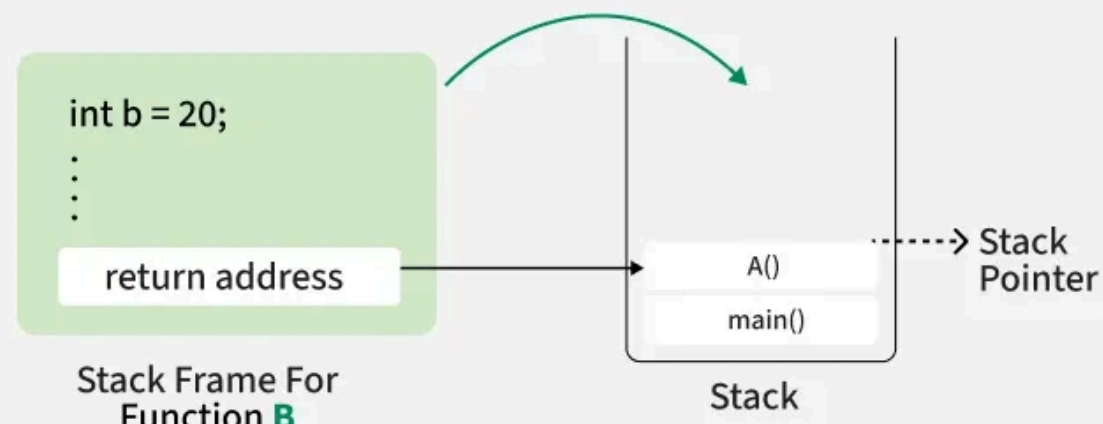
Let's use an image to understand how the method call stack works

Stack frame of A() will be created and pushed into the Call Stack. It will save the address of where to return after executing A() as return address.



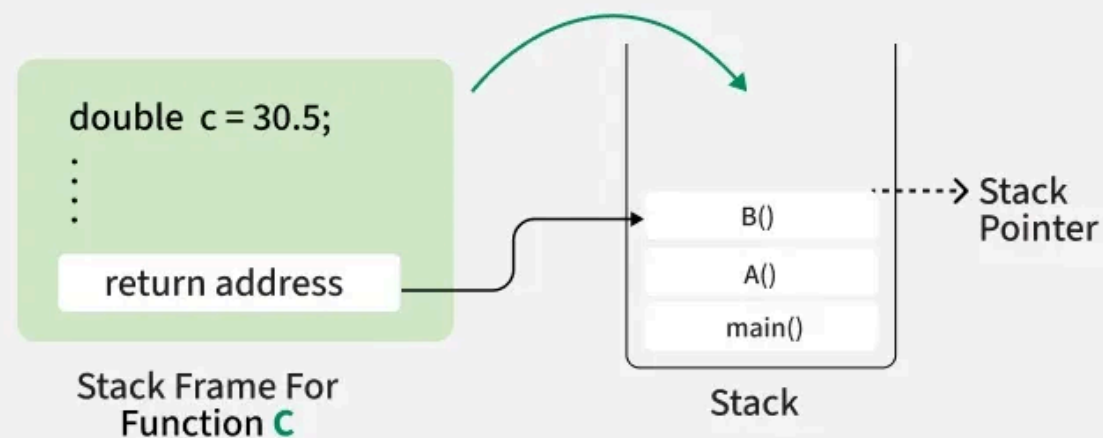
How Methods Are Stored in Call Stack

As B() is called by A(), stack frame of B() will be created and pushed to Call Stack. The return address will be the point after the call of b() in A()



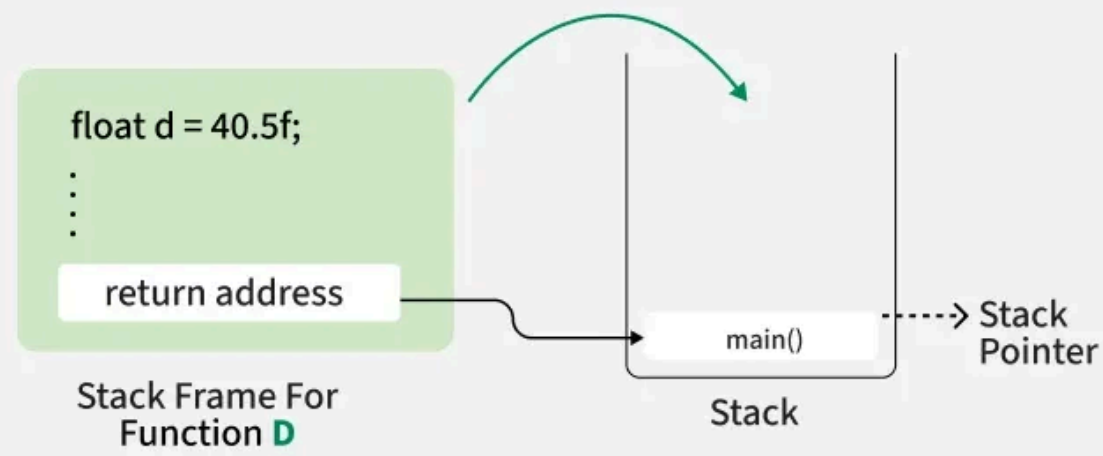
How Methods Are Stored in Call Stack

As C() is called by B(), stack frame of C() will be created and pushed to Call Stack. The return address will be the point after the call of c() in B()



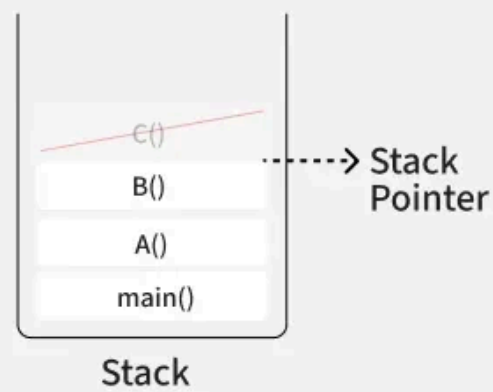
How Methods Are Stored in Call Stack

Since D() is called, its stack frame is created and pushed to the call stack just over the main()



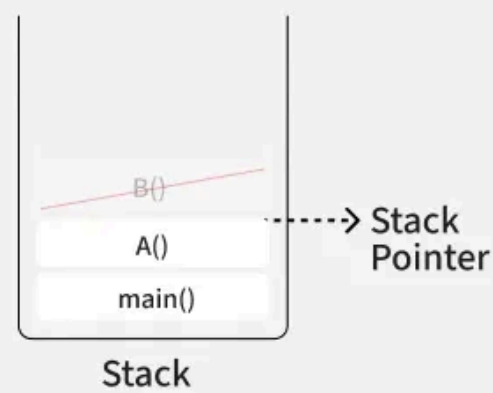
How Methods Are Stored in Call Stack

Now, C() does not call any other function. So, after its complete execution, stack frame of C() is removed from the stack and program control comes back to B() using return address.



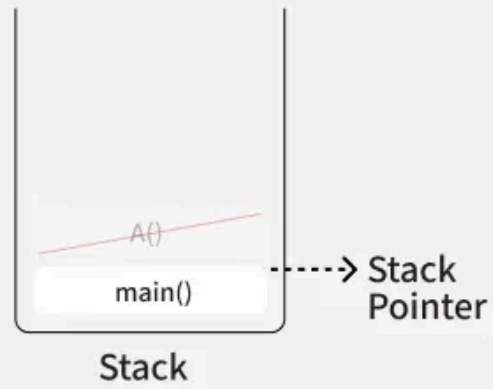
How Methods Are Stored in Call Stack

B() executes completely. So the stack frame of B() is removed from the call stack and control comes to A() using return address.



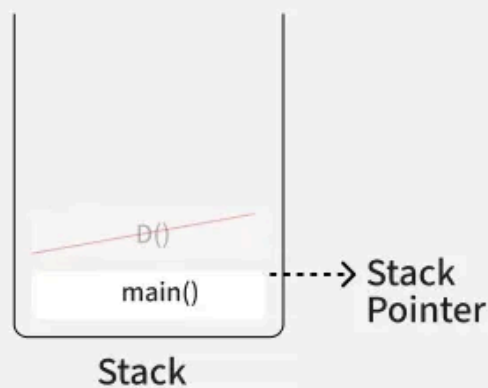
How Methods Are Stored in Call Stack

A() executes completely. So the stack frame of A() is removed from the call stack and control comes to main() using return address.



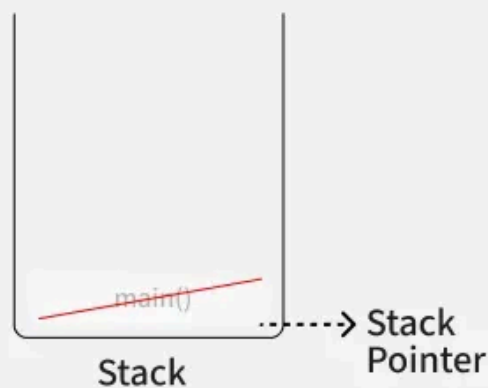
How Methods Are Stored in Call Stack

After D() executes completely, the stack frame of D() is removed and control comes back to main().



How Methods Are Stored in Call Stack

After the main() returns 0, the stack frame of main() is also deleted leading to the termination of the program.



How Methods Are Stored in Call Stack

Example Code:-

```
public class CallStackExample {  
  
    public static void D() {  
        float d = 40.5f;  
        System.out.println("In Method D");  
    }  
  
    public static void C() {  
        double c = 30.5;  
    }  
}
```

```

        System.out.println("In Method C");
    }

    public static void B() {
        int b = 20;
        C(); // Calling C
        System.out.println("In Method B");
    }

    public static void A() {
        int a = 10;
        B(); // Calling B
        System.out.println("In Method A");
    }

    public static void main(String[] args) {
        A(); // Start with function A
        D(); // Then call D
    }
}

```

Output:-

```

In Method C
In Method B
In Method A
In Method D

```

6.3 Types of Methods in Java

6.3.1 Predefined Method

Predefined methods are the method that is already defined in the Java class libraries. It is also known as the standard library method or built-in method.

For example, `random()` method which is present in the `Math` class and we can call it using the `ClassName.methodName()` as shown in the below example.

```

Math.random()    // returns random value
Math.PI          //returns the value of π as a constant. it is a Constant not a Method

```

6.3.2 User-defined Method

The method written by the user or programmer is known as a user-defined method. These methods are modified according to the requirement.

Example:

```

sayHello          // user define method created above in the article
Greet()
setName()

```

6.4 Different Ways to Create Java Method

6.4.1 Instance Method

Access the instance data using the object name. Declared inside a class.

```
// Instance Method
void method_name() {
    // instance method body
}
```

6.4.2 Static Method

Access the static data using class name. Declared inside class with **static** keyword.

```
// Static Method
static void method_name() {
    // static method body
}
```

6.5 Method Signature

It consists of the method name and a parameter list.

- Number of parameters
- Type of the parameters
- Order of the parameters

Note: The `return` type and exceptions are not considered as part of it.

Method Signature of the above function:

`max(int x, int y)` Number of parameters is 2, Type of parameter is `int`.

6.6 Naming a Method

In Java language method name is typically a single word that should be a verb in lowercase or a multi-word, that begins with a verb in lowercase followed by an adjective, noun. After the first word, the first letter of each word should be capitalized.

Rules to Name a Method:

- Method names must start with a verb in lowercase.
- Multi-word names should follow camelCase format.
- Method names should be unique within a class unless **method overloading is allowed in Java**.

6.7 Calling Different Types of Methods in Java

Method calling in Java means invoking a method to execute the code it contains. It transfers control to the process, runs its logic, and then returns to the calling point after execution.

6.7.1 Calling a User-Defined Method

User-defined methods are blocks of code written by the programmer. To execute a user-defined method, we first create an object of the class (if the method is non-static) and then call the method using that object.

Example Code:-

```
class Geeks {
    void hello() {
        System.out.println("This is a user-defined method.");
    }

    public static void main(String[] args) {
        Geeks obj = new Geeks(); // Create object
    }
}
```

```

        obj.hello();           // Call method
    }
}

```

Output:-

```

This is a user-defined method.

```

6.7.2 Calling an Abstract Method

Abstract methods have no body and must be overridden in a subclass. They are called using an object of the subclass.

Example Code:-

```

abstract class GeeksHelp {
    abstract void check(String name); // Abstract method
}

public class Geeks extends GeeksHelp {
    @Override
    void check(String name) {
        System.out.println(name);
    }

    public static void main(String[] args) {
        Geeks obj = new Geeks(); // Subclass object
        obj.check("GeeksforGeeks");
    }
}

```

Output:-

```

GeeksforGeeks

```

6.7.3 Calling the Predefined Methods

Java provides many built-in methods via the Java Standard Library, like hashCode().

Example Code:-

```

public class Geeks {
    public static void main(String[] args) {
        Geeks obj = new Geeks();
        System.out.println(obj.hashCode()); // Predefined method
    }
}

```

Output:-

```

1510467688

```

6.7.4 Calling a Static Method

Static methods belong to the class, not the object. They can be called without creating an object.

Example Code:-

```

class Test {
    static void hello() {
        System.out.println("Hello");
    }
}

```

```

    }
}
public class Geeks {
    public static void main(String[] args) {
        Test.hello(); // Call static method directly
    }
}

```

Output:-

```

Hello

```

6.8 Method Overloading

Method overloading in **Java** allows a class to have multiple methods with the same name but different parameter lists. It is a form of compile-time **polymorphism**, because the compiler determines which overloaded method to call based on the arguments passed.

- Their parameter list must be different.
- Parameters can differ in number, type, or order.
- Changing only the return type does not create method overloading.

Example Code:-

```

class Calculator {

    int add(int a, int b) {
        return a + b;
    }

    int add(int a, int b, int c) {
        return a + b + c;
    }

    double add(double a, double b) {
        return a + b;
    }
}

public class Geeks {
    public static void main(String[] args) {

        Calculator calc = new Calculator();

        System.out.println(calc.add(10, 20));
        System.out.println(calc.add(10, 20, 30));
        System.out.println(calc.add(10.5, 20.5));
    }
}

```

Output:-

```

30
60
31.0

```

6.8.1 Ways of method overloading

A. Changing the Number of Parameters

Method overloading can be achieved by changing the number of parameters when passing to different methods.

Example Code:-

```
import java.io.*;

class Product{

    // Multiplying two integer values
    public int multiply(int a, int b){

        int prod = a * b;
        return prod;
    }

    // Multiplying three integer values
    public int multiply(int a, int b, int c){

        int prod = a * b * c;
        return prod;
    }
}

class Geeks{

    public static void main(String[] args)
    {

        Product ob = new Product();

        // Calling method to Multiply 2 numbers
        int prod1 = ob.multiply(1, 2);

        // Printing Product of 2 numbers
        System.out.println(
            "Product of the two integer value: " + prod1);

        // Calling method to multiply 3 numbers
        int prod2 = ob.multiply(1, 2, 3);

        // Printing product of 3 numbers
        System.out.println(
            "Product of the three integer value: " + prod2);
    }
}
```

Output:-

```
Product of the two integer value: 2
Product of the three integer value: 6
```

B. Changing Data Types of Parameters

In many cases, methods can be considered overloaded if they have the same name but have different parameter types, methods are considered to be overloaded.

Example Code:-


```

class Product{

    public int prod(int a, int b, int c){
        return a * b * c;
    }
    public double prod(double a, double b, double c){
        return a * b * c;
    }
}

public class Geeks {
    public static void main(String[] args){

        Product p = new Product();
        System.out.println(p.prod(1, 2, 3));
        System.out.println(p.prod(1.0, 2.0, 3.0));
    }
}

```

Output:-

```

6
6.0

```

C. Changing the Order of Parameters

Method overloading can also be implemented by rearranging the parameters of two or more overloaded methods.

Example Code:-

```

class Student {

    public void studentId(String name, int rollNo){

        System.out.println("Name: " + name
                            + ", Roll-No: " + rollNo);
    }
    public void studentId(int rollNo, String name){

        System.out.println("Roll-No: " + rollNo
                            + ", Name: " + name);
    }
}

public class Geeks{

    public static void main(String[] args){

        Student s = new Student();
        s.studentId("Sweta", 1);
        s.studentId(2, "Gudly");
    }
}

```

Output:-

Name: Sweta, Roll-No: 1
Roll-No: 2, Name: Gudly

6.8.2 What if the Exact Prototype Does Not Match?

When an exact overloaded method is not available, Java may perform method invocation conversions, including certain primitive widening conversions, to find a compatible method. Java tries **type promotion in overloading**

- Convert to a higher type in the same hierarchy (e.g., byte → int).
- Convert to the next higher hierarchy if needed (e.g., int → float).

Example Code:-

```
class Demo{

    public void show(int x){

        System.out.println("In int: " + x);

    }
    public void show(String s){

        System.out.println("In String: " + s);

    }
    public void show(byte b){

        System.out.println("In byte: " + b);

    }

}

public class UseDemo {
    public static void main(String[] args) {
        Demo obj = new Demo();
        obj.show((byte) 25);
        obj.show("hello");
        obj.show(250);
        obj.show('A');
        // obj.show(7.5);      // Error: no suitable method for double
    }
}
```

Output:-

In byte: 25
In String: hello
In int: 250
In int: 65

6.9 Overloading vs Overriding

Method Overloading	Method Overriding
Same method name, different parameters	Same method signature in parent and child
Usually occurs within the same class	Requires inheritance
Compile-time polymorphism	Runtime polymorphism
Compiler selects the method	JVM selects the overridden instance method at runtime

Method Overloading	Method Overriding
Parameters must differ	Parameters must match

6.10 Function Scope

In Java, "**function scope**" is officially referred to as **method scope** or **local scope**. It means that **any variable declared inside a method (Java's term for a function) is only accessible and visible within that specific method**.

6.11 Key Rules of Method Scope

1. **Declaration to End:** A local variable's scope begins at the exact line it is declared and extends to the closing curly brace `}` of that method.
2. **Method Parameters:** Parameters passed into a method also fall under method scope; they are accessible anywhere inside the method body.
3. **No Outside Access:** You cannot access a method's local variables from another method or from class-level blocks

Example Code:-

```
public class ScopeExample {

    // Class level scope (accessible by all methods)
    int classVariable = 10;

    public void myMethod(int parameterVariable) {
        // 'parameterVariable' has method scope, accessible throughout this method

        int localVariable = 5; // 'localVariable' begins its method scope here

        System.out.println(localVariable);    // Works fine!
        System.out.println(parameterVariable); // Works fine!
        System.out.println(classVariable);    // Works fine!
    }

    public void anotherMethod() {
        // System.out.println(localVariable); // ERROR! Can't see 'localVariable' here
        // System.out.println(parameterVariable); // ERROR! Can't see 'parameterVariable' here

        System.out.println(classVariable);    // Works fine!
    }
}
```

6.12 Recursion

In Java, Recursion is a process in which a function calls itself directly or indirectly is called recursion and the corresponding function is called a recursive function. Using a recursive algorithm, certain problems can be solved quite easily. A few Java recursion examples are **Towers of Hanoi (TOH)**, **Inorder/Preorder/Postorder Tree Traversals**, **DFS of Graph**, etc.

6.12.1 Base Condition in Recursion

In the recursive program, the solution to the base case is provided and the solution to the bigger problem is expressed in terms of smaller problems.

Example:-

```
int fact(int n)
{
    if (n <= 1) // base case
        return 1;
}
```

```

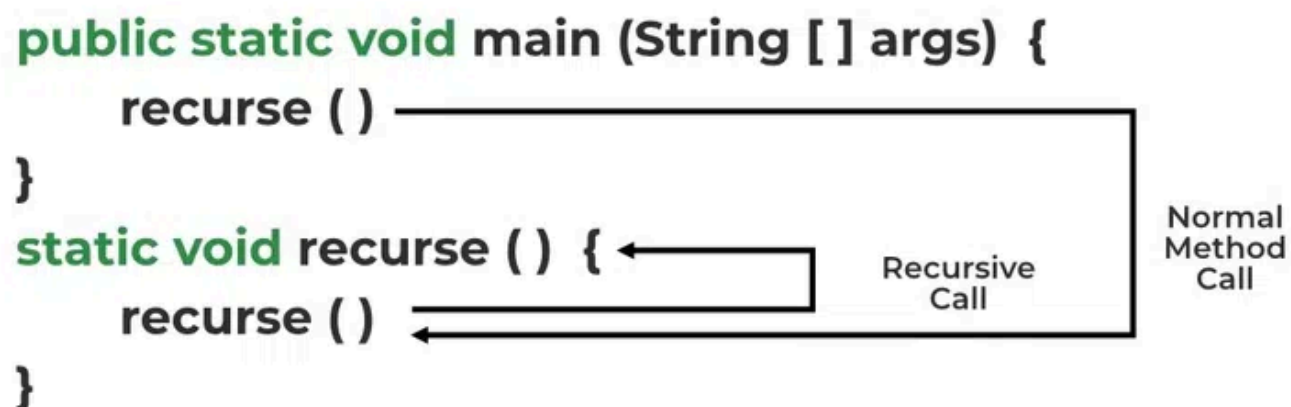
else
    return n*fact(n-1);
}

```

In the above example, the base case for $n \leq 1$ is defined and the larger value of a number can be solved by converting it to a smaller one till the base case is reached.

6.12.2 Working of Recursion

The idea is to represent a problem in terms of one or more smaller sub-problems and add base conditions that stop the recursion. For example, we compute factorial n if we know the factorial of $(n-1)$. The base case for factorial would be $n = 0$. We return 1 when $n = 0$.



6.12.3 Java Recursion Programs

A. Factorial Using Recursion

The **classic example of recursion** is the computation of the factorial of a number. The factorial of a number N is the product of all the numbers between 1 and N . The below-given code computes the factorial of the numbers: 3, 4, and 5.

3= $3 * 2 * 1$ (6) 4= $4 * 3 * 2 * 1$ (24) 5= $5 * 4 * 3 * 2 * 1$ (120)

- 3= $3 * 2 * 1$ (6)
- 4= $4 * 3 * 2 * 1$ (24)
- 5= $5 * 4 * 3 * 2 * 1$ (120)

Example Code:-

```

// Java Program to implement
// Factorial using recursion
class GFG {

    // recursive method
    int fact(int n)
    {
        int result;

        if (n == 1)
            return 1;
        result = fact(n - 1) * n;
        return result;
    }
}

```

```
// Driver Class
class Recursion {

    // Main function
    public static void main(String[] args)
    {
        GFG f = new GFG();

        System.out.println("Factorial of 3 is "
            + f.fact(3));
        System.out.println("Factorial of 4 is "
            + f.fact(4));
        System.out.println("Factorial of 5 is "
            + f.fact(5));
    }
}
```

Output:-

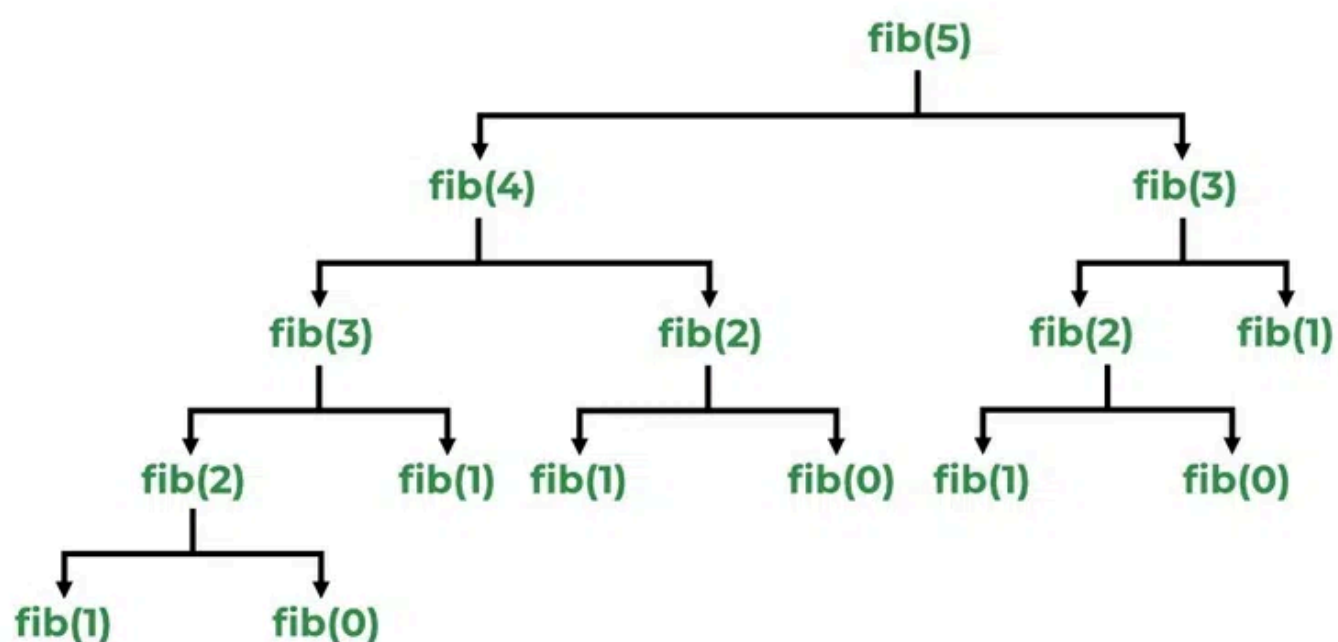
```
Factorial of 3 is 6
Factorial of 4 is 24
Factorial of 5 is 120
```

B. Fibonacci Series

Fibonacci Numbers are the numbers in the integer sequence where $Fib(N) = Fib(N-2) + Fib(N-1)$. Below is the example to find 3,4,5.

$Fib(3) = Fib(2) + Fib(1) = Fib(1) + 0 + 1 = 1+1 = 2$
 $Fib(4) = Fib(3) + Fib(2) = 2+1 = 3$
 $Fib(5) = Fib(4) + Fib(3) = 3 + 2 = 5$

- $Fib(3) = Fib(2) + Fib(1) = Fib(1) + 0 + 1 = 1+1 = 2$
- $Fib(4) = Fib(3) + Fib(2) = 2+1 = 3$
- $Fib(5) = Fib(4) + Fib(3) = 3 + 2 = 5$



Example Code:-

```
// Java Program to implement
// Fibonacci Series
import java.io.*;

// Driver Function
class GFG {
```

```

// Function to return Fibonacci value
static int Fib(int N)
{
    if (N == 0 || N == 1)
        return N;

    return Fib(N - 1) + Fib(N - 2);
}

// Main function
public static void main(String[] args)
{
    // Fibonacci of 3
    System.out.println("Fibonacci of " + 3 + " "
        + Fib(3));

    // Fibonacci of 4
    System.out.println("Fibonacci of " + 4 + " "
        + Fib(4));

    // Fibonacci of 5
    System.out.println("Fibonacci of " + 5 + " "
        + Fib(5));
}
}

```

Output:-

```

Fibonacci of 3 2
Fibonacci of 4 3
Fibonacci of 5 5

```

6.12.4 Stack Overflow error

If the base case is not reached or not defined, then the stack overflow problem may arise. Let us take an example to understand this.

```

int fact(int n)
{
    // wrong base case (it may cause
    // stack overflow).
    if (n == 100)
        return 1;
    else
        return n*fact(n-1);
}

```

If fact(10) is called, it will call fact(9), fact(8), fact(7) and so on but the number will never reach 100. So, the base case is not reached. **If the memory is exhausted by these functions on the stack, it will cause a stack overflow error.**

6.12.5 How is memory allocated to different function calls in recursion?

When any function is called from main(), the memory is allocated to it on the stack. A recursive function calls itself, the memory for the called function is allocated on top of memory allocated to the calling function and a different copy of local variables is created for each function call. When the base case is reached, the function returns its value to the function by whom it is called and memory is de-allocated and the process continues.

Example Code:-

```
// A Java program to demonstrate
// working of recursion

class GFG {
    static void printFun(int test)
    {
        if (test < 1)
            return;

        else {
            System.out.printf("%d ", test);

            // Statement 2
            printFun(test - 1);

            System.out.printf("%d ", test);
            return;
        }
    }

    public static void main(String[] args)
    {
        int test = 3;
        printFun(test);
    }
}
```

Output:-

```
3 2 1 1 2 3
```

6.21.6 Advantages of Recursive Programming

- Recursion provides a clean and simple way to write code.
- Some problems are inherently recursive like tree traversals, **Tower of Hanoi**, etc. For such problems, it is preferred to write recursive code.

6.12.7 Disadvantages of Recursive Programming

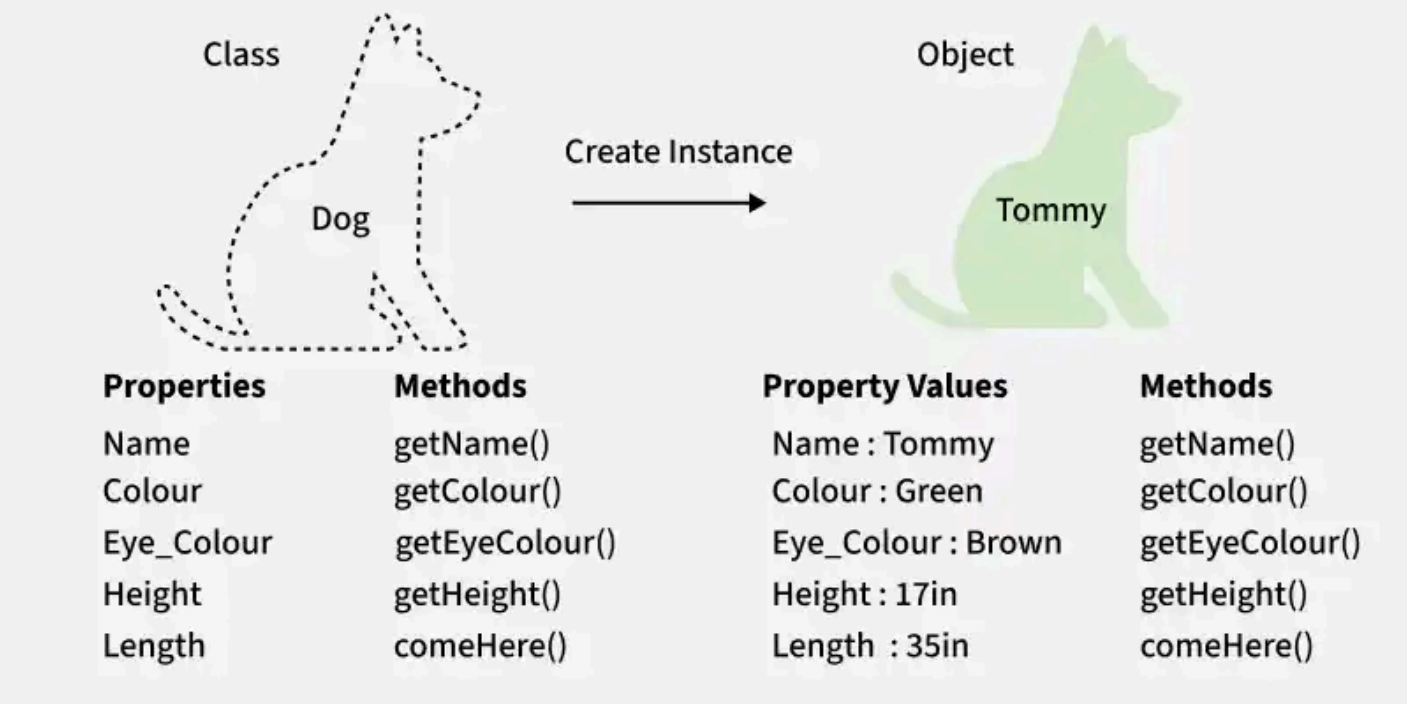
- The recursive program has greater space requirements than the iterative program as all functions will remain in the stack until the base case is reached.
- It also has greater time requirements because of function calls and returns overhead.

7. Object Oriented Programming

7.1 Classes and Objects

In Java, classes and objects form the foundation of Object-Oriented Programming (OOP). They help model real-world entities and organize code in a structured way.

- A class is a blueprint used to create objects that share common properties and behavior.
- An object is an instance of a class. It represents a specific entity created from the class template.



7.1.1 Java Class

A class is a user-defined data type that groups related data and behavior together. It can contain fields, methods, constructors, nested classes, and interfaces.

- Acts as a template to create objects with shared structure.
- Does not occupy memory for fields until instantiation.

Example Code:-

```
class Student {
    int id;
    String n;

    public Student(int id, String n) {
        this.id = id;
        this.n = n;
    }
}

public class Main {
    public static void main(String[] args) {
        Student s1 = new Student(10, "Alice");
        System.out.println(s1.id);
        System.out.println(s1.n);
    }
}
```

Output:-

```
10
Alice
```

7.1.2 Java Objects

An object is an instance of a class created to access its data and operations. Each object holds its own state.

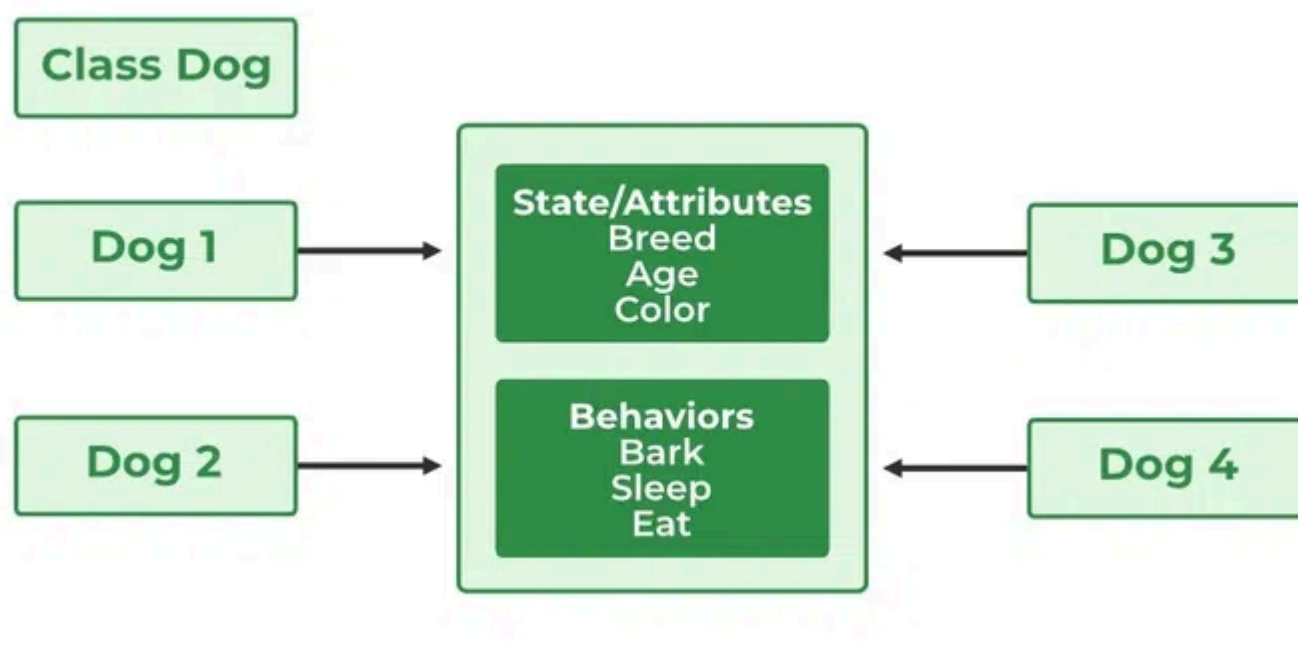
- **State:** Values stored in fields.
- **Behavior:** Actions defined through methods.
- **Identity:** Distinguishes one object from another.

Objects mirror real-world items such as customer, product or circle. Non-primitive objects are stored on the heap while their references remain on the stack.



Object Instantiation

Creating an object is known as instantiation. All instances of a class share structure and behavior while storing different state values.



Java Object Declaration

Declaration:

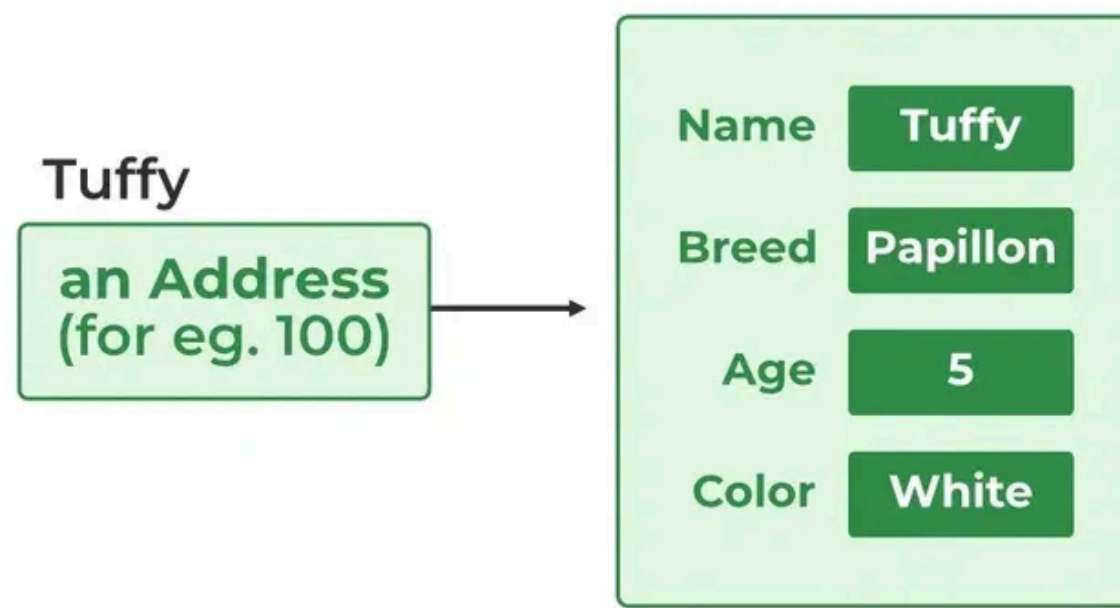
```
Dog tuffy;
```

This only declares a reference. The object is not created and the reference holds null.

Initialization:

```
tuffy = new Dog("Tuffy", "Papillon", 5, "White");
```

Initialization



Initialization

The new operator allocates memory and invokes the constructor.

Example Code:-

```
public class Dog {  
  
    String name;  
    String breed;  
    int age;  
    String color;  
  
    public Dog(String name, String breed, int age, String color) {  
        this.name = name;  
        this.breed = breed;  
        this.age = age;  
        this.color = color;  
    }  
  
    public String getName() { return name; }  
    public String getBreed() { return breed; }  
    public int getAge() { return age; }  
    public String getColor() { return color; }  
  
    @Override  
    public String toString() {  
        return "Name is: " + name  
            + "\nBreed age and color are: "  
            + breed + " " + age + " " + color;  
    }  
  
    public static void main(String[] args) {  
        Dog tuffy = new Dog("tuffy", "papillon", 5, "white");  
        System.out.println(tuffy);  
    }  
}
```

Output:-

```
Name is: tuffy  
Breed age and color are: papillon 5 white
```

7.1.3 Ways to Create Object in Java

Java supports four standard approaches.

A. Using new Keyword

new Keyword is most direct way to create an object.

```
// creating object of class Test
Test t = new Test();
```

B. Using Reflection

Used for dynamic class loading as seen in frameworks like Spring.

```
class Student {
    public Student() {}
}

public class Main {
    public static void main(String[] args) {
        try {
            Class<?> c = Class.forName("Student");
            Student s = (Student) c.getDeclaredConstructor().newInstance();
            System.out.println(s);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

C. Using clone() method

clone() creates a copy of an existing object. The class must implement Cloneable.

```
class Geeks implements Cloneable {
    String name = "GeeksForGeeks";

    @Override
    protected Object clone() throws CloneNotSupportedException {
        return super.clone();
    }

    public static void main(String[] args) {
        try {
            Geeks g1 = new Geeks();
            Geeks g2 = (Geeks) g1.clone();
            System.out.println(g2.name);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

D. Using Deserialization

De-serialization is a technique of reading an object from the saved state in a file. Object is recreated from a stored byte stream.

```
import java.io.*;

class Student implements Serializable {
    private String name;
    public Student(String name) { this.name = name; }
    public String toString() { return "Student: " + name; }
}

public class Main {
    public static void main(String[] args) {
        try (ObjectOutputStream out =
            new ObjectOutputStream(new FileOutputStream("student.ser"))) {
            out.writeObject(new Student("Alice"));
        } catch (IOException e) { e.printStackTrace(); }

        try (ObjectInputStream in =
            new ObjectInputStream(new FileInputStream("student.ser"))) {
            Student s = (Student) in.readObject();
            System.out.println(s);
        } catch (Exception e) { e.printStackTrace(); }
    }
}
```

7.1.4 Anonymous Objects

Anonymous objects are objects created without assigning them to a reference variable. They are generally used when an object is needed only once.

- No reference variable is stored, so the object cannot be directly reused.
- The object is created and used in the same statement.
- They are useful for simple, one-time operations.

Example Code:-

```
class Dog {

    String name;

    Dog(String name) {
        this.name = name;
    }

    void bark() {
        System.out.println(name + " is barking");
    }
}

public class Main {

    public static void main(String[] args) {

        Dog dog = new Dog("Tommy");
        dog.bark();
    }
}
```

Output:-

Tommy is barking

7.2 Constructors

A constructor in Java is a special member that is called when an object is created. It initializes the new object's state. It is used to set default or user-defined values for the object's attributes.

- A constructor does not have a return type, not even `void`.
- A constructor is called automatically when an object is created using `new`.
- Constructors can have parameters and can be overloaded.

Example Code:-

```
class Student {
    String name;

    // Constructor
    Student(String name) {
        this.name = name;
    }

    void display() {
        System.out.println("Name: " + name);
    }

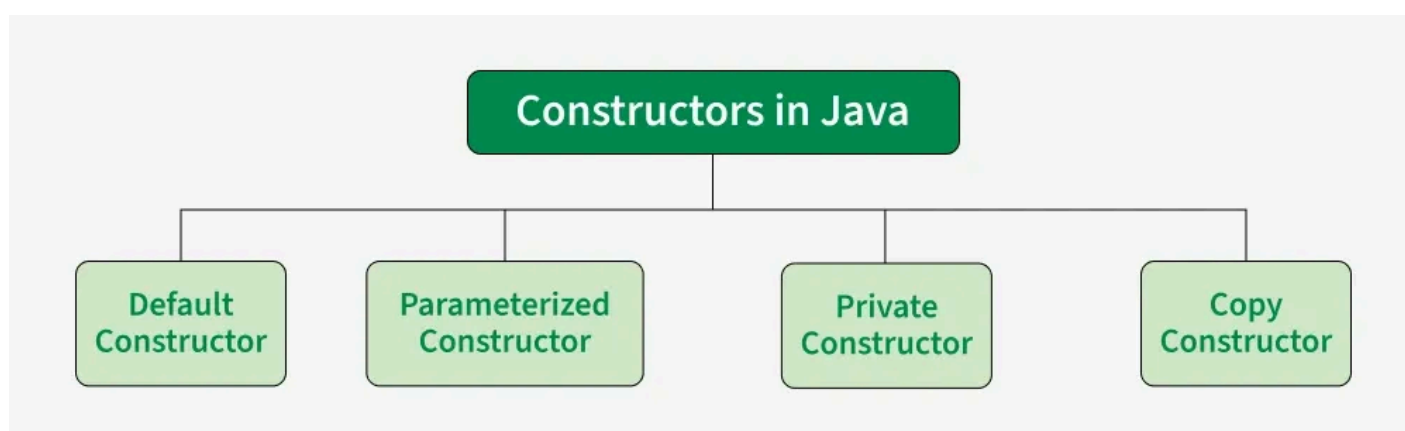
    public static void main(String[] args) {
        Student s1 = new Student("Vishnu");
        s1.display();
    }
}
```

Output:-

Name: Vishnu

7.2.1 Types of Constructors in Java

There are four types of constructors in Java



A. Default Constructor

A default constructor has no parameters. It's used to assign default values to an object. If no constructor is explicitly defined, Java provides a default constructor.

Example Code:-

```

class Student {

    String name;
    int age;
}

public class Main {

    public static void main(String[] args) {

        Student s = new Student();

        System.out.println(s.name);
        System.out.println(s.age);
    }
}

```

Output:-

```

null
0

```

B. Parameterized Constructor

A constructor that has parameters is known as parameterized constructor. If we want to initialize fields of the class with our own values, then use a parameterized constructor.

Example Code:-

```

class Geeks{

    // data members of the class
    String name;
    int id;

    // Parameterized Constructor
    Geeks(String name, int id)
    {
        this.name = name;
        this.id = id;
    }

    // Method to display object data
    void display(){

        System.out.println("GeekName: " + name
                           + " and GeekId: " + id);
    }

    // main() method – placed inside the same class for
    // universal compatibility
    public static void main(String[] args){

        // This will invoke the parameterized constructor
        Geeks geek1 = new Geeks("Sweta", 68);
        geek1.display();
    }
}

```



```
}  
}
```

Output:-

```
GeekName: Sweta and GeekId: 68
```

C. Copy Constructor in Java

Unlike other constructors **copy constructor** is passed with another object which copies the data available from the passed object to the newly created object.

Example Code:-

```
import java.io.*;  
  
class Geeks{  
  
    // data members of the class  
    String name;  
    int id;  
  
    // Parameterized Constructor  
    Geeks(String name, int id)  
    {  
        this.name = name;  
        this.id = id;  
    }  
  
    // Copy Constructor  
    Geeks(Geeks obj2)  
    {  
        this.name = obj2.name;  
        this.id = obj2.id;  
    }  
}  
  
class GFG {  
    public static void main(String[] args)  
    {  
        // This would invoke the parameterized constructor  
        System.out.println("First Object");  
        Geeks geek1 = new Geeks("Sweta", 68);  
        System.out.println("GeekName: " + geek1.name  
                           + " and GeekId: " + geek1.id);  
  
        System.out.println();  
  
        // This would invoke the copy constructor  
        Geeks geek2 = new Geeks(geek1);  
        System.out.println(  
            "Copy Constructor used Second Object");  
        System.out.println("GeekName: " + geek2.name  
                           + " and GeekId: " + geek2.id);  
    }  
}
```

Output:-


```
First Object
GeekName: Sweta and GeekId: 68
```

```
Copy Constructor used Second Object
GeekName: Sweta and GeekId: 68
```

D. Private Constructor

A private constructor cannot be accessed from outside the class. It is commonly used in:

- **Singleton Pattern**: To ensure only one instance of a class is created.
- **Utility/Helper Classes**: To prevent instantiation of a class containing only static methods.

Example Code:-

```
class GFG {

    // Private constructor
    private GFG(){

        System.out.println("Private constructor called");
    }

    // Static method
    public static void displayMessage(){

        System.out.println("Hello from GFG class!");
    }
}

class Main{

    public static void main(String[] args){

        // GFG u = new GFG(); // Error: constructor is
        // private
        GFG.displayMessage();
    }
}
```

Output:-

```
Hello from GFG class!
```

7.2.2 Constructor Overloading

This is a key concept in **OOP** related to constructors. Constructor overloading allows a class to have multiple constructors with different parameter lists. It provides different ways to initialize objects based on the values available at the time of object creation.

- Constructor overloading is resolved at compile time.
- It allows objects of the same class to be initialized in different ways.
- It helps avoid writing separate initialization methods for different sets of values.

Example Code:-

```
import java.io.*;
```

```

class Geeks{

    // constructor with one argument
    Geeks(String name){

        System.out.println("Constructor with one "
                           + "argument - String: " + name);
    }

    // constructor with two arguments
    Geeks(String name, int age){

        System.out.println(
            "Constructor with two arguments: "
            + " String and Integer: " + name + " " + age);
    }

    // Constructor with one argument but with different
    // type than previous
    Geeks(long id)
    {
        System.out.println(
            "Constructor with one argument: "
            + "Long: " + id);
    }
}

class GFG {
    public static void main(String[] args){

        // Creating the objects of the class named 'Geek'
        // by passing different arguments
        Geeks geek2 = new Geeks("Sweta");

        // Invoke the constructor with two arguments
        Geeks geek3 = new Geeks("Amiya", 28);

        // Invoke the constructor with one argument of
        // type 'Long'.
        Geeks geek4 = new Geeks(325614567);
    }
}

```

Output:-

```

Constructor with one argument - String: Sweta
Constructor with two arguments: String and Integer: Amiya 28
Constructor with one argument: Long: 325614567

```

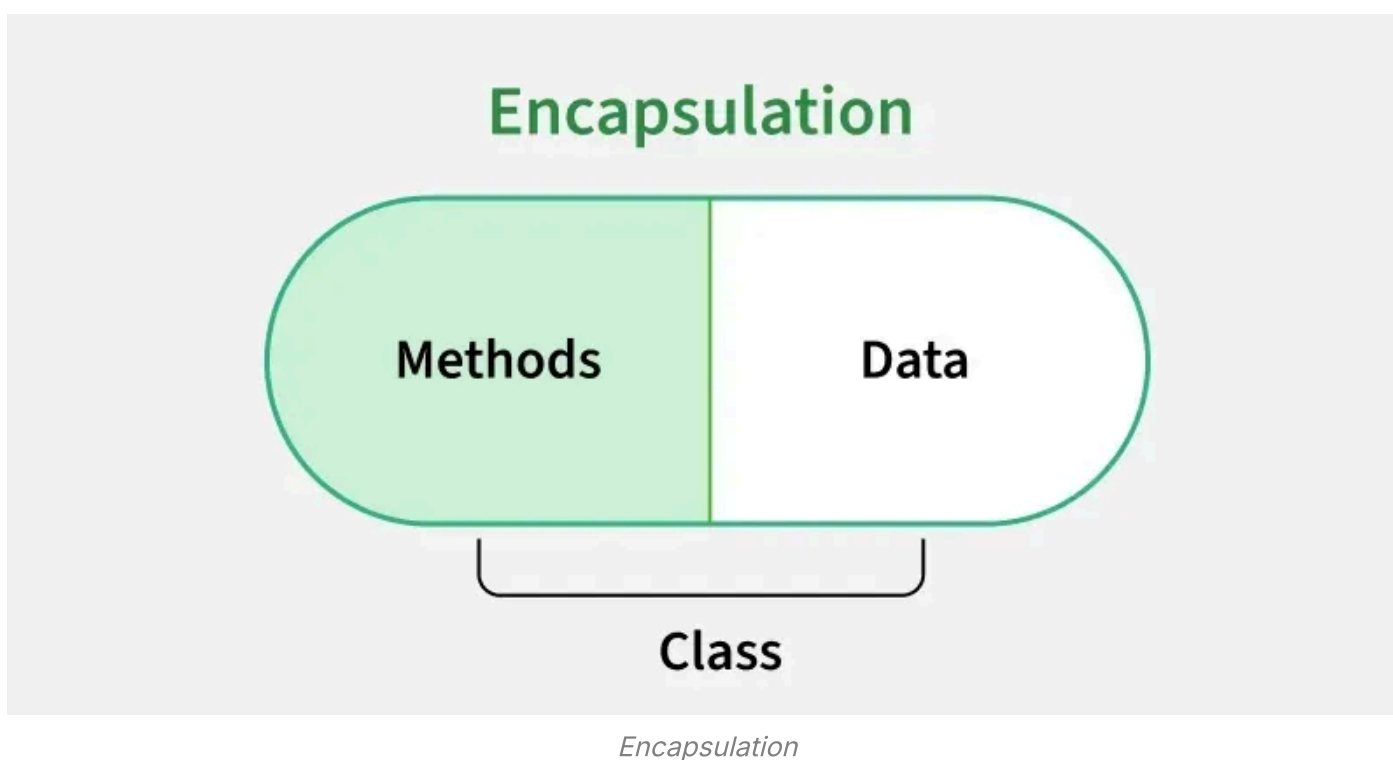
7.3 Encapsulation

Encapsulation in Java is an **object-oriented programming** concept that bundles data and the methods that operate on that data into a single unit, such as a class. It also helps control access to the object's internal state by restricting direct access and providing controlled ways to read or modify it.

- Usually uses private fields to restrict direct access.
- Provides controlled access through methods such as getters and setters when required.

- Allows validation and other rules to be applied before changing data.

Encapsulation



7.3.1 How Encapsulation is Achieved in Java

Encapsulation is commonly implemented by:

- Declaring fields as `private`.
- Providing public or appropriately accessible methods to read or modify the fields.
- Adding validation inside methods when necessary.
- Keeping the internal implementation hidden from code outside the class.

7.3.2 Rules

- **Declare data as private:** Hide the class data so it cannot be accessed directly from outside the class.
- **Use getters and setters:** Keep variables private and provide public getter and setter methods for controlled access and safe modification, often with validation.
- **Apply proper access modifiers:** Use private for data hiding and public for methods that provide access.

Example Code:-

```
class Programmer {

    private String name;

    // Getter method used to get the data
    public String getName() { return name; }

    // Setter method is used to set or modify the data
    public void setName(String name) {

        this.name = name;
    }
}

public class Geeks {

    public static void main(String[] args){

        Programmer p = new Programmer();
        p.setName("Geek");
    }
}
```

```
        System.out.println("Name=> " + p.getName());
    }
}
```

Output:-

```
Name=> Geek
```

7.3.3 Advantages of Encapsulation

- Protects an object's internal data from direct access.
- Allows validation before data is modified.
- Makes code easier to maintain and modify.
- Keeps related data and methods together in one class.
- Reduces dependency on the internal implementation of a class.
- Provides better control over how an object's state can be accessed or changed.

7.3.4 Limitations of Encapsulation

- Can increase the amount of code when many accessor methods are required.
- May add unnecessary complexity for very simple classes.
- Requires careful design to decide which data and operations should be exposed.
- Excessive use of getters and setters can expose more of a class's internal state than necessary.

7.3.5 Data Hiding vs Encapsulation

Aspect	Data Hiding	Encapsulation
Meaning	Restricts direct access to internal data	Bundles data and related methods into a class
Main focus	Controlling access to data	Organizing data and behavior together
Purpose	Protects internal state from unwanted direct access	Provides a controlled and well-defined interface
Commonly achieved using	Access modifiers such as <code>private</code>	Classes, access control, and methods
Scope	Mainly concerned with restricting access	Broader concept that includes bundling and access control
Example	<code>private int balance;</code>	<code>class Account { private int balance; void deposit() {...} }</code>

7.4 Inheritance

Inheritance in Java is a core **OOP** concept that allows a class to acquire properties and behaviors from another class. It helps in creating a new class from an existing class, promoting code reusability and better organization.

- A subclass can reuse the fields and methods of the parent class without rewriting the code
- A subclass can add its own fields and methods or modify existing ones to extend functionality.

Example Code:-

```
// Parent class
class Animal {
    void sound() {
        System.out.println("Animal makes a sound");
    }
}

// Child class
class Dog extends Animal {
```

```

        void sound() {
            System.out.println("Dog barks");
        }
    }

    // Child class
    class Cat extends Animal {
        void sound() {
            System.out.println("Cat meows");
        }
    }

    // Child class
    class Cow extends Animal {
        void sound() {
            System.out.println("Cow moos");
        }
    }

    // Main class
    public class Geeks {
        public static void main(String[] args) {
            Animal a;
            a = new Dog();
            a.sound();

            a = new Cat();
            a.sound();

            a = new Cow();
            a.sound();
        }
    }

```

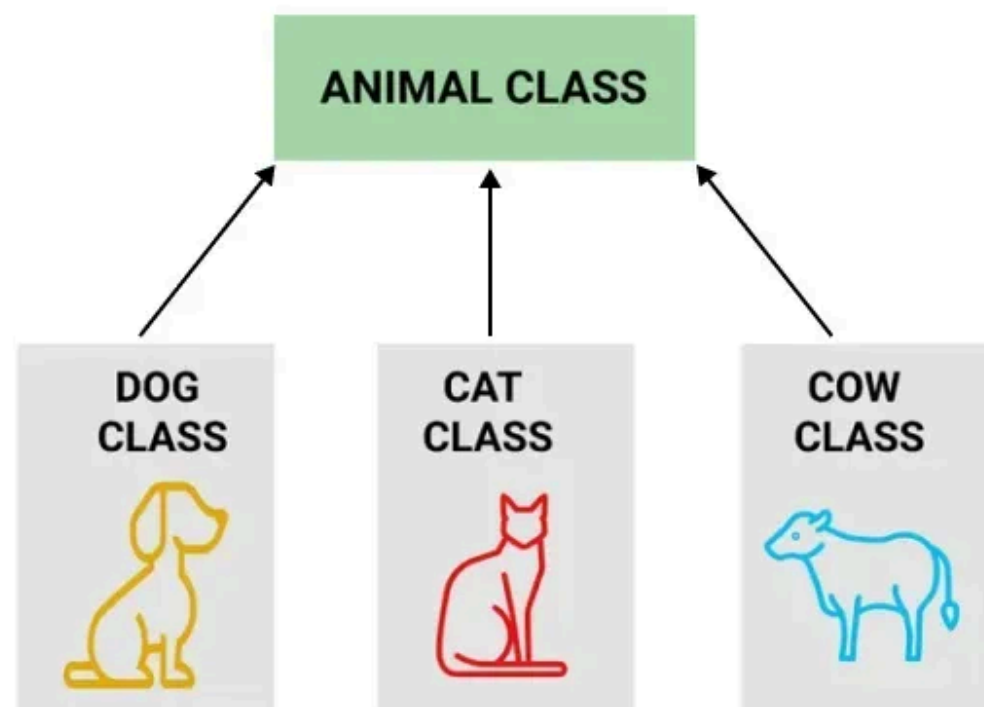
Output:-

```

Dog barks
Cat meows
Cow moos

```

Note: The `sound()` method is an instance method, so it is called using an object reference. Here, `new Dog()` creates a `Dog` object, and its reference is assigned to the `Animal` reference `a`. When `a.sound()` is called, Java executes the overridden `Dog` version of `sound()` at runtime.



Syntax:

```

class Parent {
    // fields and methods
}
class Child extends Parent {
    // additional fields and methods
}
  
```

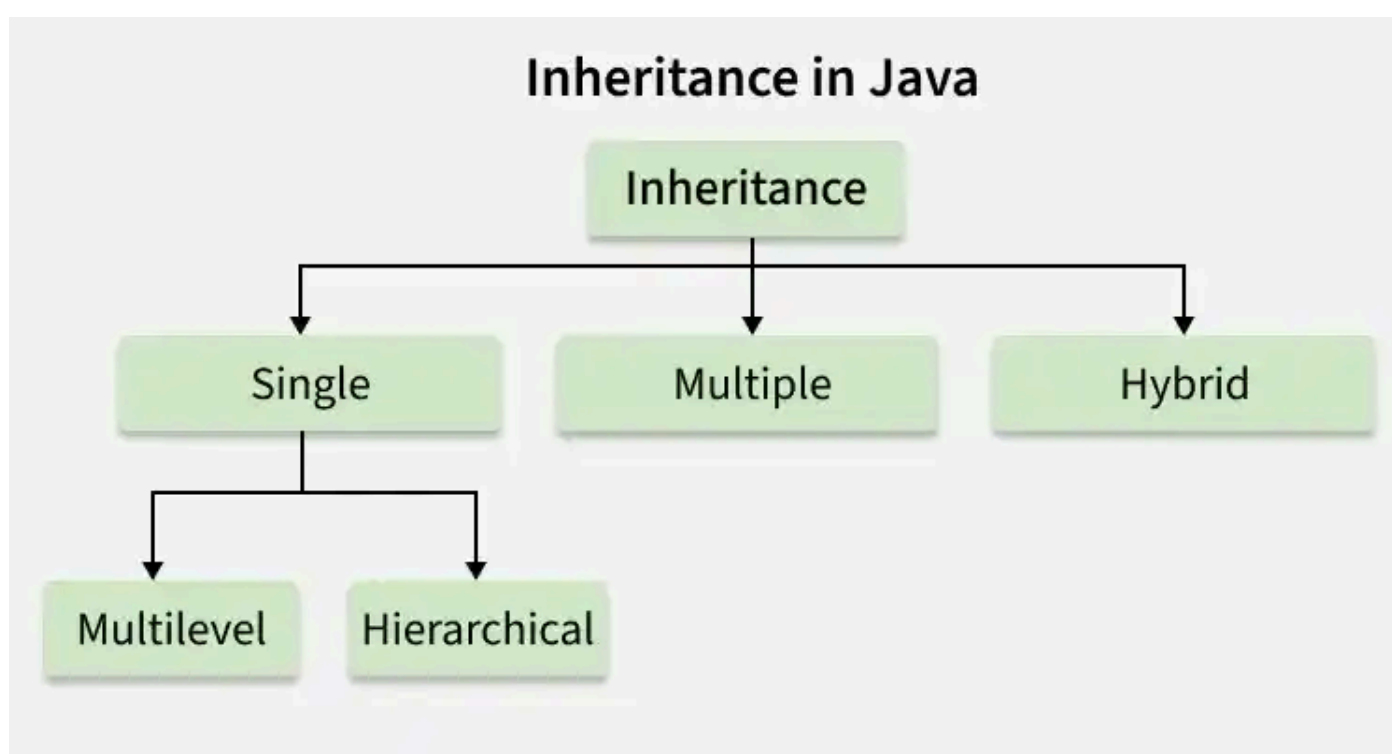
Note: In Java, inheritance is implemented using the `extends` keyword.

Key Terminologies in Java Inheritance

- **Class** : Blueprint from which objects are created
- **Superclass** : Class whose properties are inherited
- **Subclass** : Class that inherits another class
- **extends** : Keyword used to inherit a class

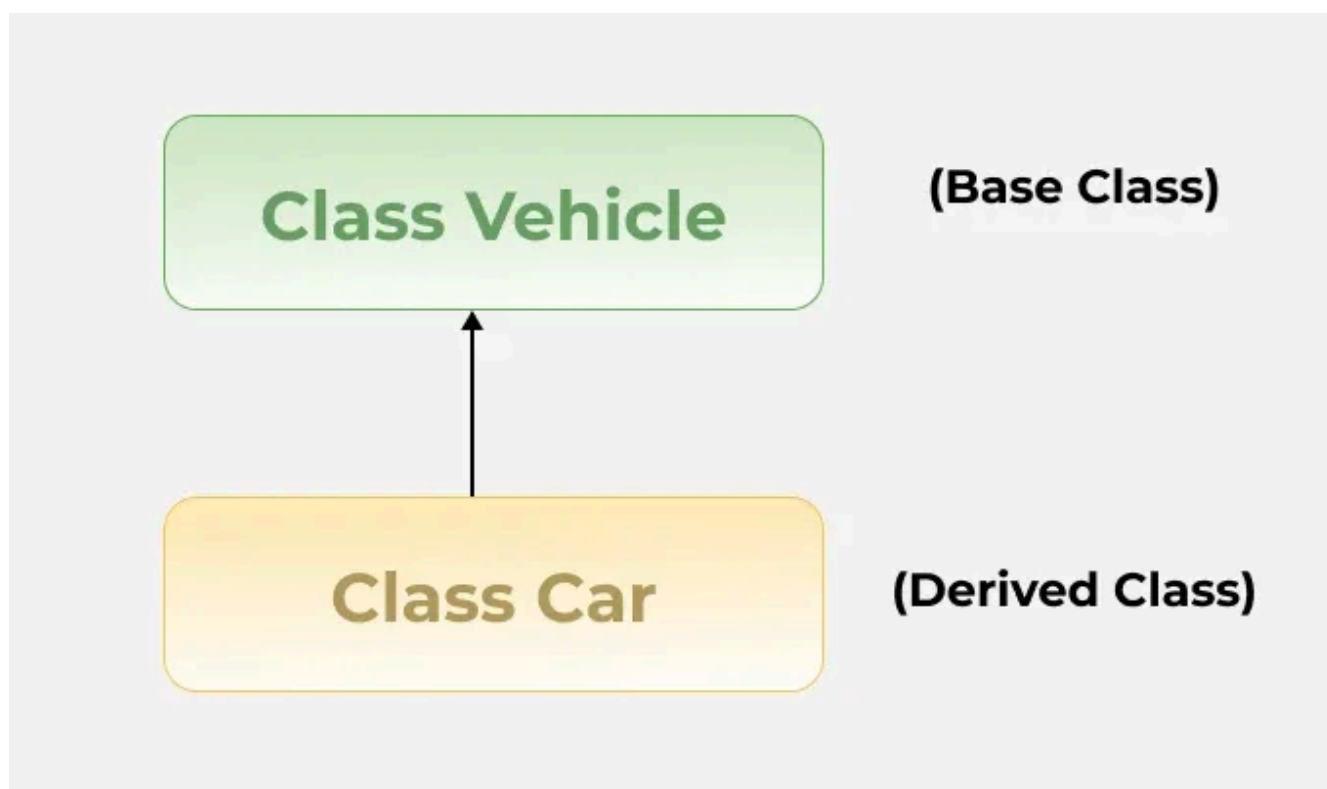
7.4.1 Types of Inheritance in Java

It defines the different ways a class can inherit properties and behavior from one or more classes.



7.4.2 Single Inheritance

In single inheritance, a sub-class is derived from only one super class. It inherits the properties and behavior of a single-parent class. Sometimes, it is also known as simple inheritance.



Example Code:-

```
//Super class
class Vehicle {
    Vehicle() {
        System.out.println("This is a Vehicle");
    }
}

// Subclass
class Car extends Vehicle {
    Car() {
        System.out.println("This Vehicle is Car");
    }
}

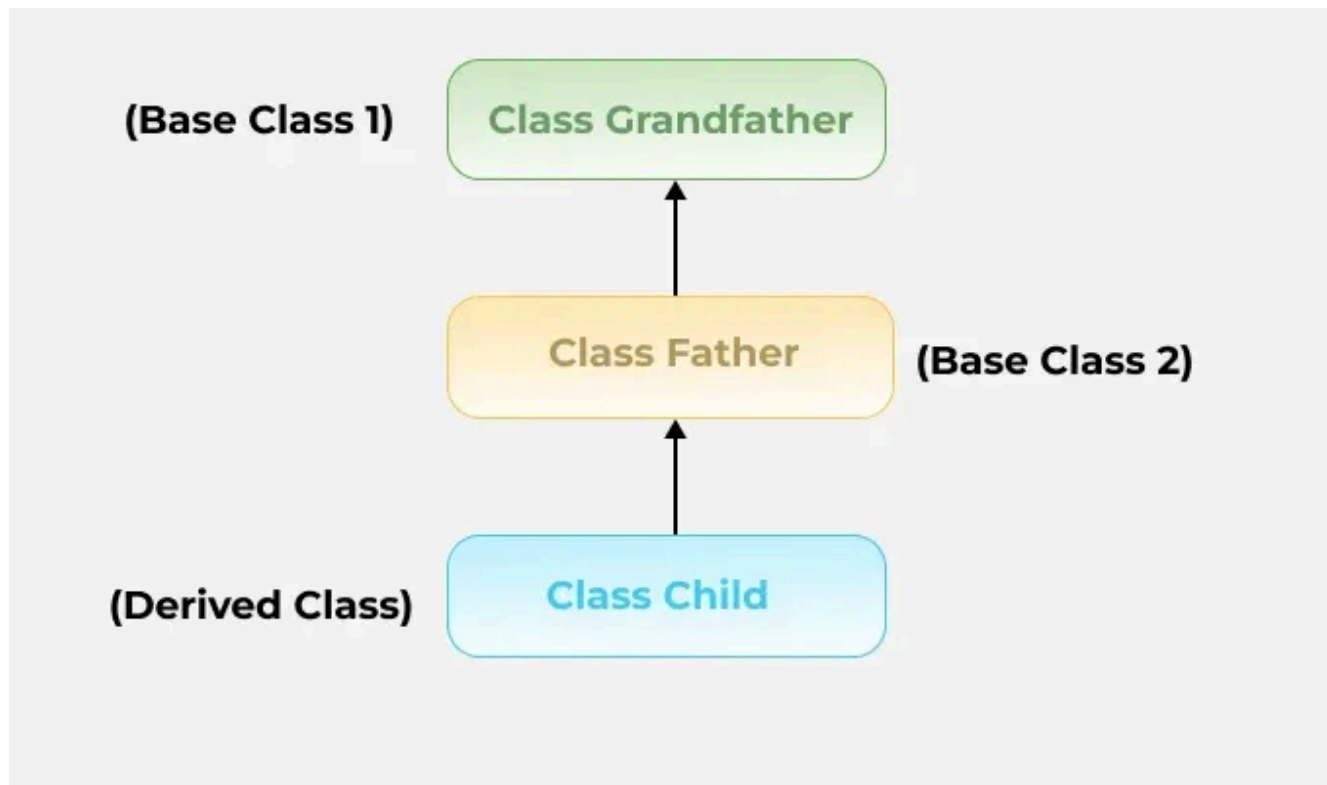
public class Test {
    public static void main(String[] args) {
        // Creating object of subclass invokes base class constructor
        Car obj = new Car();
    }
}
```

Output:-

```
This is a Vehicle
This Vehicle is Car
```

7.4.3 Multilevel Inheritance

In Multilevel Inheritance, a derived class will be inheriting a base class and as well as the derived class also acts as the base class for other classes.



Example Code:-

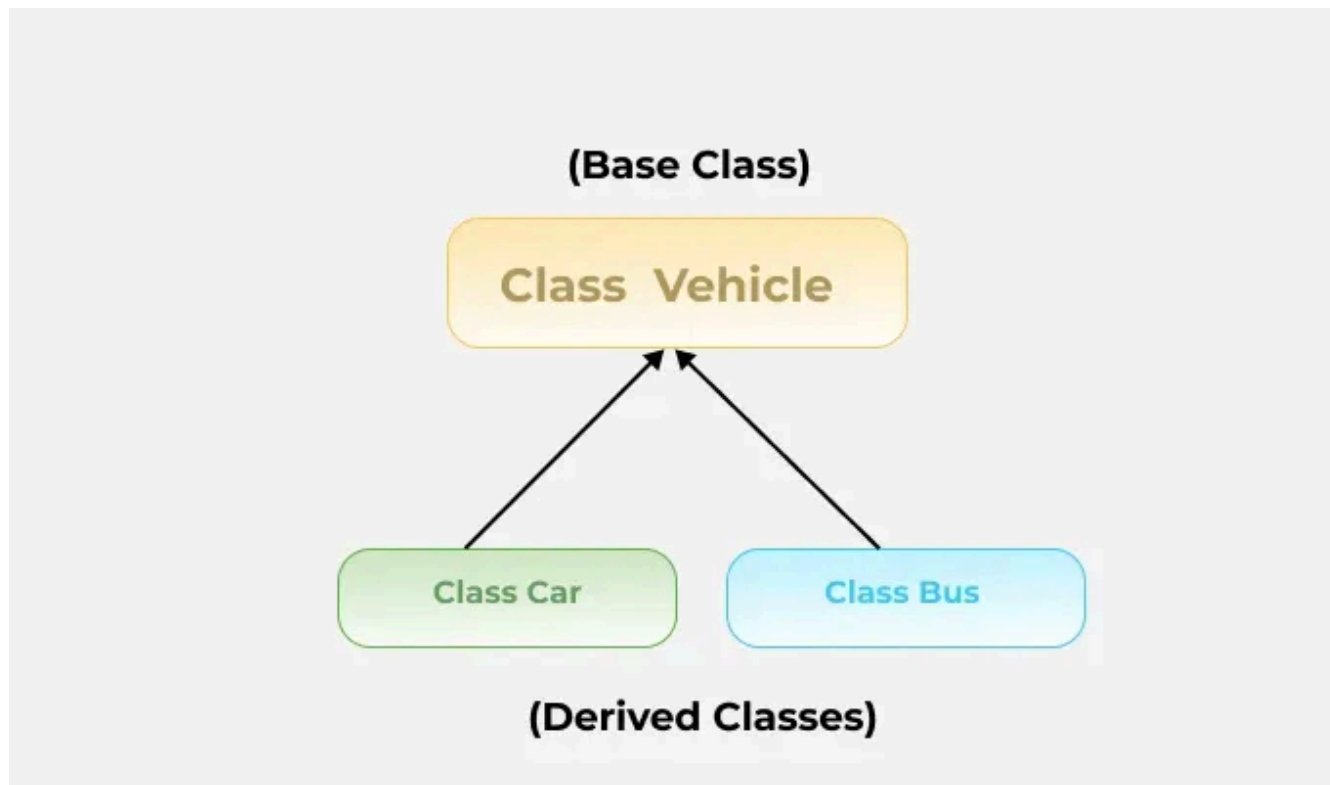
```
class Vehicle {
    Vehicle() {
        System.out.println("This is a Vehicle");
    }
}
class FourWheeler extends Vehicle {
    FourWheeler() {
        System.out.println("4 Wheeler Vehicles");
    }
}
class Car extends FourWheeler {
    Car() {
        System.out.println("This 4 Wheeler Vehicle is a Car");
    }
}
public class Geeks {
    public static void main(String[] args) {
        Car obj = new Car(); // Triggers all constructors in order
    }
}
```

Output:-

```
This is a Vehicle
4 Wheeler Vehicles
This 4 Wheeler Vehicle is a Car
```

7.4.4 Hierarchical Inheritance

In hierarchical inheritance, more than one subclass is inherited from a single base class. i.e. more than one derived class is created from a single base class. For example, cars and buses both are vehicle



Example Code:-

```
class Vehicle {
    Vehicle() {
        System.out.println("This is a Vehicle");
    }
}

class Car extends Vehicle {
    Car() {
        System.out.println("This Vehicle is Car");
    }
}

class Bus extends Vehicle {
    Bus() {
        System.out.println("This Vehicle is Bus");
    }
}

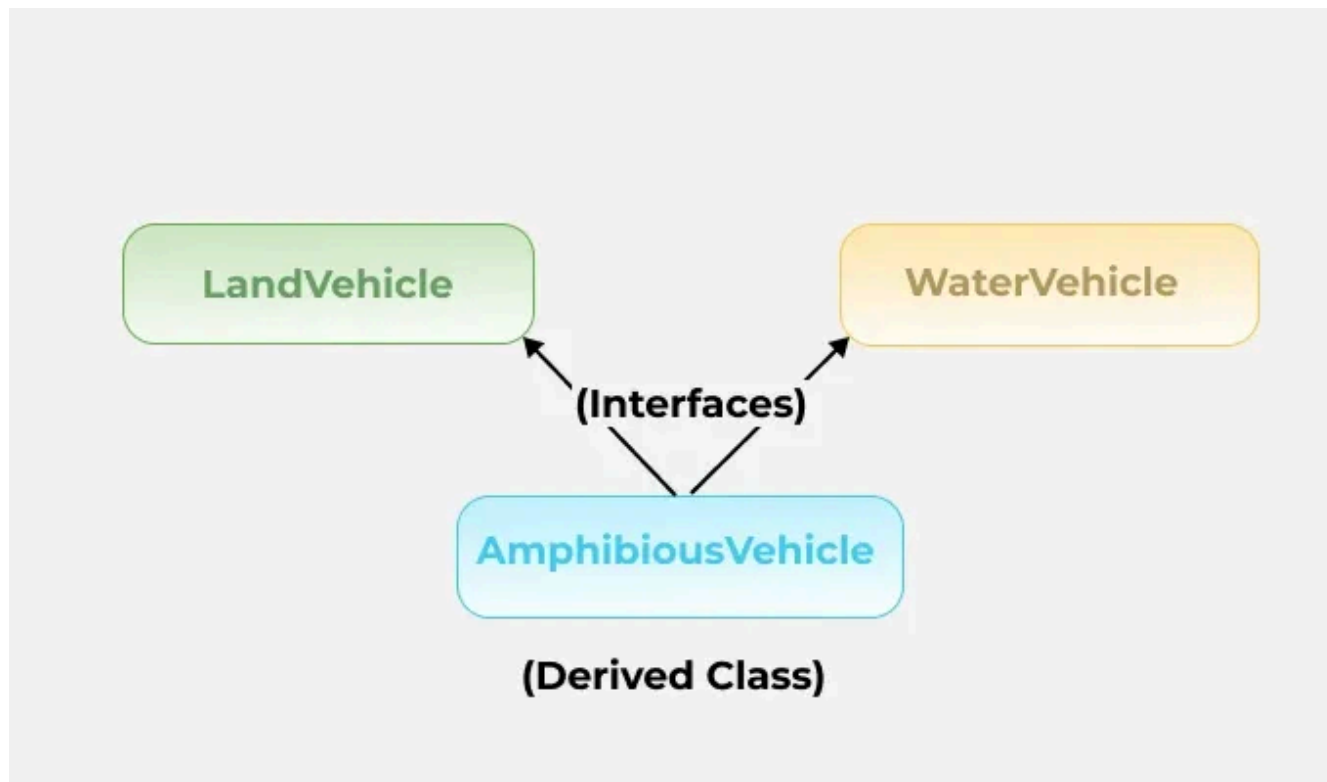
public class Test {
    public static void main(String[] args) {
        Car obj1 = new Car();
        Bus obj2 = new Bus();
    }
}
```

Output:-

```
This is a Vehicle
This Vehicle is Car
This is a Vehicle
This Vehicle is Bus
```

7.4.5 Multiple Inheritance (Through Interfaces)

In **Multiple inheritances**, one class can have more than one superclass and inherit features from all parent classes.



Note: that Java does not support multiple inheritances with classes. In Java, we can achieve multiple inheritances only through **Interfaces**.

Example Code:-

```
interface LandVehicle {
    default void landInfo() {
        System.out.println("This is a LandVehicle");
    }
}

interface WaterVehicle {
    default void waterInfo() {
        System.out.println("This is a WaterVehicle");
    }
}

// Subclass implementing both interfaces
class AmphibiousVehicle implements LandVehicle, WaterVehicle {
    AmphibiousVehicle() {
        System.out.println("This is an AmphibiousVehicle");
    }
}

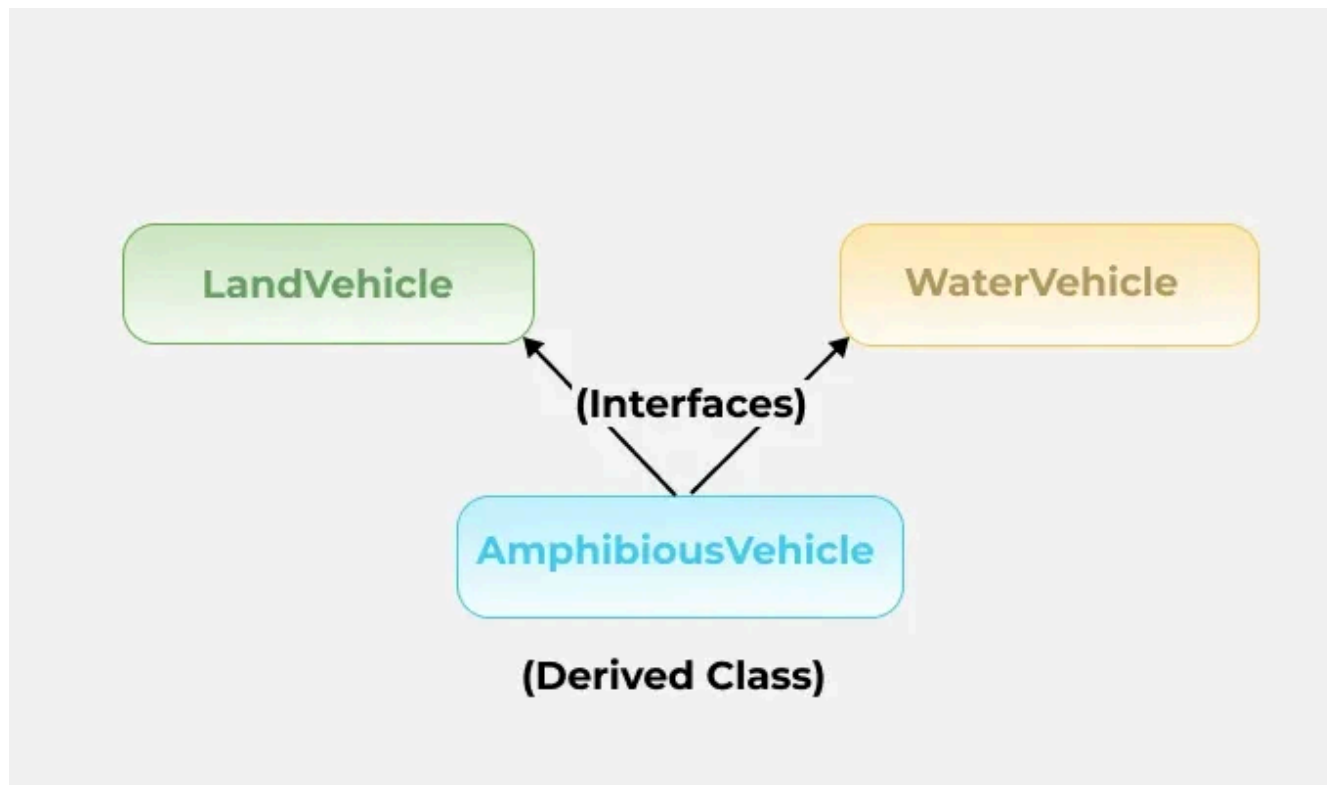
public class Test {
    public static void main(String[] args) {
        AmphibiousVehicle obj = new AmphibiousVehicle();
        obj.waterInfo();
        obj.landInfo();
    }
}
```

Output:-

```
This is an AmphibiousVehicle
This is a WaterVehicle
This is a LandVehicle
```

7.4.6 Hybrid Inheritance

It is a mix of two or more of the above types of inheritance. In Java, we can achieve hybrid inheritance only through **Interfaces** if we want to involve multiple inheritance to implement Hybrid inheritance.



Example Code:-

```
// Superclass
class Vehicle {
    void vehicleType() {
        System.out.println("This is a Vehicle");
    }
}

// Interface for fare
interface Fare {
    default void fareInfo() {
        System.out.println("Fare information");
    }
}

// Subclass 1: Single inheritance + multilevel
class Car extends Vehicle {
    void carType() {
        System.out.println("This is a Car");
    }
}

// Subclass 2: Hybrid inheritance (extends class + implements interface)
class Bus extends Vehicle implements Fare {
    void busType() {
        System.out.println("This is a Bus");
    }
}

public class GFG{
    public static void main(String[] args) {
        Car car = new Car();
        car.vehicleType(); // inherited from Vehicle
        car.carType();     // specific to Car

        Bus bus = new Bus();
        bus.vehicleType(); // inherited from Vehicle
        bus.busType();     // specific to Bus
        bus.fareInfo();    // from Fare interface
    }
}
```

```
}  
}
```

Output:-

```
This is a Vehicle  
This is a Car  
This is a Vehicle  
This is a Bus  
Fare information
```

7.4.7 What Can Be Done in a Subclass?

In sub-classes we can inherit members as is, replace them, hide them or supplement them with new members:

- The inherited fields can be used directly, just like any other fields.
- We can declare new fields in the subclass that are not in the superclass.
- The inherited methods can be used directly as they are.
- We can write a new instance method in the subclass that has the same signature as the one in the superclass, thus **overriding** it (as in the example above, toString() method is overridden).
- We can write a new static method in the subclass that has the same signature as the one in the superclass, thus hiding it.
- We can declare new methods in the subclass that are not in the superclass.
- We can write a subclass constructor that invokes the constructor of the superclass, either implicitly or by using the keyword **super**.

7.4.8 Advantages of Inheritance in Java

- **Code Reusability:** Inheritance allows for code reuse and reduces the amount of code that needs to be written. The subclass can reuse the properties and methods of the superclass, reducing duplication of code.
- **Abstraction:** Inheritance allows for the creation of abstract classes that define a common interface for a group of related classes. This promotes abstraction and encapsulation, making the code easier to maintain and extend.
- **Class Hierarchy:** Inheritance allows for the creation of a class hierarchy, which can be used to model real-world objects and their relationships.
- **Polymorphism:** Inheritance allows for polymorphism, which is the ability of an object to take on multiple forms. Subclasses can override the methods of the superclass, which allows them to change their behavior in different ways.

7.4.9 Limitations of Inheritance in Java

- **Complexity** → Deep inheritance hierarchies can make code hard to understand . Correction → keep hierarchy simple or use composition instead
- **Tight Coupling** → Changes in superclass can affect subclasses . Correction → use loose coupling (interfaces or composition)

7.5 Polymorphism

Polymorphism is one of the fundamental concepts of **Object-Oriented Programming** (OOP) that allows a single entity to take multiple forms. It enables the same method or interface to behave differently depending on the object involved. Polymorphism improves code flexibility, reusability, and maintainability.

- Polymorphism means "many forms".
- It allows the same method to exhibit different behaviors.
- It is achieved through method overloading and method overriding.

Example Code:-

```

class Person {

    // Method that displays the
    // role of a person
    void role() { System.out.println("I am a person."); }
}

// Derived class Father that
// overrides the role method
class Father extends Person {

    // Overridden method to show
    // the role of a father
    @Override void role()
    {
        System.out.println("I am a father.");
    }
}

public class Main {
    public static void main(String[] args)
    {

        // Creating a reference of type Person
        // but initializing it with Father class object
        Person p = new Father();

        // Calling the role method. It calls the
        // overridden version in Father class
        p.role();
    }
}

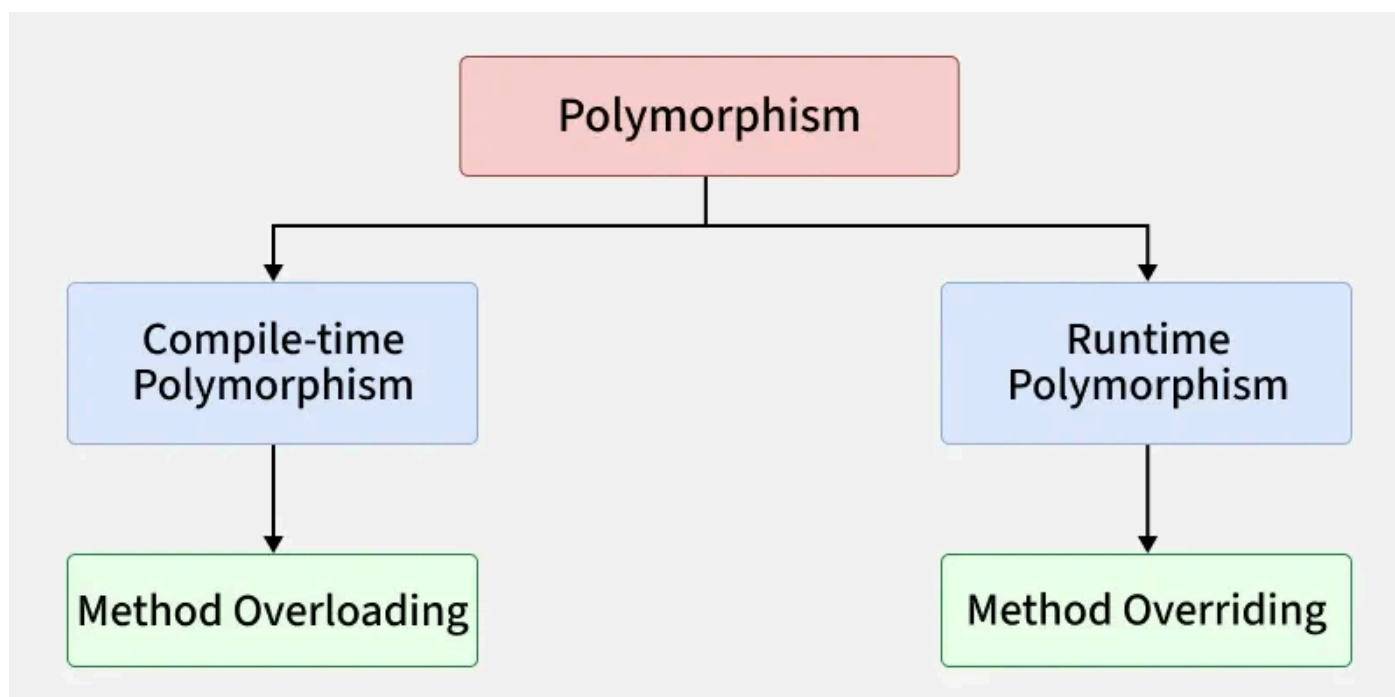
```

Output:-

```
I am a father.
```

7.5.1 Types of Polymorphism in Java

In Java Polymorphism is mainly divided into two types:



7.5.2 Compile-Time Polymorphism

Compile-time polymorphism also known as static polymorphism, occurs when the method to be executed is determined during compilation. It is primarily achieved through method overloading.

- Method selection occurs at compile time.
- Improves readability and flexibility.

Method Overloading

Method overloading allows multiple methods with the same name but different parameter lists within the same class. The compiler chooses the appropriate method based on the arguments passed. The parameters can differ in:

- Number of parameters
- Data types of parameters
- Order of parameters

Syntax:

```
class Demo {  
    void show(int a) {  
    }  
    void show(int a, int b) {  
    }  
}
```

Example Code:-

```
class Helper {  
  
    // Method with 2 integer parameters  
    static int Multiply(int a, int b)  
    {  
        // Returns product of integer numbers  
        return a * b;  
    }  
  
    // Method 2  
    // With same name but with 2 double parameters  
    static double Multiply(double a, double b)  
    {  
        // Returns product of double numbers  
        return a * b;  
    }  
}  
  
// Main class  
class Geeks {  
    // Main driver method  
    public static void main(String[] args)  
    {  
  
        // Calling method by passing  
        // input as in arguments  
        System.out.println(Helper.Multiply(2, 4));  
        System.out.println(Helper.Multiply(5.5, 6.3));  
    }  
}
```

Output:-

8
34.65

7.5.3 Runtime Polymorphism

Runtime polymorphism also known as **dynamic polymorphism**, occurs when the method call is resolved during program execution. It is achieved through method overriding.

- Method selection occurs at runtime.
- Depends on the actual object type.

Method Overriding

Method overriding occurs when a subclass provides its own implementation of a method already defined in its superclass. The overridden method is selected based on the actual object created.

Syntax:

```
class Parent {  
    void print() {  
    }  
}  
class Child extends Parent {  
    @Override  
    void print() {  
    }  
}
```

Note: At runtime, the method that gets executed depends on the actual object type, not the reference type.

Example Code:-

```
// Class 1  
// Helper class  
class Parent {  
  
    // Method of parent class  
    void Print() { System.out.println("parent class"); }  
}  
  
// Class 2  
// Helper class  
class Subclass1 extends Parent {  
  
    // Method  
    void Print() { System.out.println("subclass1"); }  
}  
  
// Class 3  
// Helper class  
class Subclass2 extends Parent {  
  
    // Method  
    void Print() { System.out.println("subclass2"); }  
}  
  
// Class 4  
// Main class  
class Geeks {
```

```
// Main driver method
public static void main(String[] args)
{

    // Creating object of class 1
    Parent a;

    // Now we will be calling print methods
    // inside main() method
    a = new Subclass1();
    a.Print();

    a = new Subclass2();
    a.Print();

}
}
```

Output:-

```
subclass1
subclass2
```

7.5.4 Compile-Time vs Runtime Polymorphism

Feature	Compile-Time Polymorphism	Runtime Polymorphism
Also called	Static polymorphism	Dynamic polymorphism
Mainly achieved using	Method overloading	Method overriding
Method selection	Compile time	Runtime
Inheritance required	No	Yes
Example	add(int, int) and add(double, double)	Dog.sound() overriding Animal.sound()

7.5.5 Advantages of Polymorphism

- Makes code flexible and reusable.
- Improves code readability.
- Makes programs easier to maintain.
- Allows the same method to perform different tasks.
- Makes it easier to add new classes and features.
- Supports abstraction and loose coupling.

7.5.6 Disadvantages of Polymorphism

- Can make code more complex to understand.
- Can make debugging difficult, especially with runtime polymorphism.
- May require more classes and methods, increasing code size.
- Runtime polymorphism can have a small performance overhead.
- Improper use can make the program harder to maintain.

7.6 This Keyword

The this keyword in Java is a reference that refers to the current object of a class. It is available in instance methods, constructors, and instance initializer blocks, and it is commonly used to access the current object's instance members.

- Return the current object from a method.
- Pass the current object as an argument to a method or constructor.

Example Code:-

```
public class Person {

    // Fields Declared
    String name;
    int age;

    // Constructor
    Person(String name, int age)
    {
        this.name = name;
        this.age = age;
    }

    // Getter for name
    public String get_name() { return name; }

    // Setter for name
    public void change_name(String name)
    {
        this.name = name;
    }

    // Method to Print the Details of the person
    public void printDetails()
    {
        System.out.println("Name: " + this.name);
        System.out.println("Age: " + this.age);
        System.out.println();
    }

    // main function
    public static void main(String[] args)
    {
        // Objects Declared
        Person first = new Person("ABC", 18);
        Person second = new Person("XYZ", 22);

        first.printDetails();
        second.printDetails();

        first.change_name("PQR");
        System.out.println("Name has been changed to: "
                           + first.get_name());
    }
}
```

Output:-

```
Name: ABC
Age: 18

Name: XYZ
Age: 22

Name has been changed to: PQR
```

7.6.1 Uses of `this` Keyword in Java

A. To Refer to Current Class Instance Variables

The `this` keyword is commonly used when a constructor or method parameter has the same name as an instance variable.

Example Code:-

```
class Geeks {
    int a;
    int b;

    // Parameterized constructor
    Geeks(int a, int b)
    {
        this.a = a;
        this.b = b;
    }

    void display()
    {
        // Displaying value of variables a and b
        System.out.println("a = " + a + " b = " + b);
    }

    public static void main(String[] args)
    {
        Geeks object = new Geeks(10, 20);
        object.display();
    }
}
```

Output:-

```
a = 10 b = 20
```

B. To Invoke the Current Class Constructor

The `this()` syntax is used for **constructor chaining**, where one constructor calls another constructor of the same class.

Example Code:-

```
class Geeks {
    int a;
    int b;

    // Default constructor
    Geeks()
    {
        this(10, 20);
        System.out.println(
            "Inside default constructor \n");
    }

    // Parameterized constructor
    Geeks(int a, int b)
    {
```

```

        this.a = a;
        this.b = b;
        System.out.println(
            "Inside parameterized constructor");
    }

    public static void main(String[] args)
    {
        Geeks object = new Geeks();
    }
}

```

Output:-

```

Inside parameterized constructor
Inside default constructor

```

C. To Return the Current Class Instance

The `this` keyword can be returned from an instance method to return the current object.

Example Code:-

```

class Geeks {
    int a;
    int b;

    // Default constructor
    Geeks()
    {
        a = 10;
        b = 20;
    }

    // Method that returns current class instance
    Geeks get() {
        return this;
    }

    // Displaying value of variables a and b
    void display()
    {
        System.out.println("a = " + a + " b = " + b);
    }

    public static void main(String[] args)
    {
        Geeks object = new Geeks();
        object.get().display();
    }
}

```

Output:-

```

a = 10 b = 20

```

D. To Pass the Current Object as a Method Argument

The `this` keyword can be passed as an argument to another method.

Example Code:-

```
class Geeks {
    int a;
    int b;

    // Default constructor
    Geeks()
    {
        a = 10;
        b = 20;
    }

    // Method that receives "this"
    // keyword as parameter
    void display(Geeks obj)
    {
        System.out.println("a = " + obj.a
                           + " b = " + obj.b);
    }

    // Method that returns current class instance
    void get() {
        display(this);
    }

    // main function
    public static void main(String[] args)
    {
        Geeks object = new Geeks();
        object.get();
    }
}
```

Output:-

```
a = 10 b = 20
```

E. To Invoke the Current Class Method

The `this` keyword can be used to explicitly invoke an instance method of the current object.

Example Code:-

```
class Geeks {

    void display()
    {
        // calling function show()
        this.show();

        System.out.println("Inside display function");
    }

    void show()
    {
```



```

        System.out.println("Inside show function");
    }

    public static void main(String args[])
    {
        Geeks g1 = new Geeks();
        g1.display();
    }
}

```

Output:-

```

Inside show function
Inside display function

```

F. To Pass the Current Object as a Constructor Argument

The `this` keyword can be passed to another class's constructor when the current object needs to be provided as an argument.

Example Code:-

```

class A {
    B obj;

    // Parameterized constructor with object of B
    // as a parameter
    A(B obj)
    {
        this.obj = obj;

        // calling display method of class B
        obj.display();
    }
}

class B {
    int x = 5;

    // Default Constructor that create an object of A
    // with passing this as an argument in the
    // constructor
    B() { A obj = new A(this); }

    // method to show value of x
    void display()
    {
        System.out.println("Value of x in Class B : " + x);
    }

    public static void main(String[] args)
    {
        B obj = new B();
    }
}

```

Output:-

Value of x in Class B : 5

7.6.2 Advantages of Using "this" Reference

There are many advantages of using "this" reference in Java as mentioned below:

- It helps to distinguish between instance variables and local variables with the same name.
- It can be used to pass the current object as an argument to another method.
- It can be used to return the current object from a method.
- It can be used to invoke a constructor from another overloaded constructor in the same class.

7.6.3 Disadvantages of Using "this" Reference

Although "this" reference comes with many advantages there are disadvantages of also:

- Overuse of this can make the code harder to read and understand.
- Using this unnecessarily can add unnecessary overhead to the program.
- Using this in a static context results in a compile-time error.
- Overall, this keyword is a useful tool for working with objects in Java, but it should be used judiciously and only when necessary.

7.7 Static keyword

The static keyword in Java is used for memory management and belongs to the class rather than any specific instance. It allows members (variables, methods, blocks, and nested classes) to be shared among all objects of a class.

- Memory is allocated only once when the class is loaded.
- No object creation is needed to access static members; use the class name directly.
- Static methods and variables can't access non-static members directly.
- Static methods can't be overridden because they belong to the class, not instances.

7.7.1 Static Variables

A **static variable** is also known as a class variable. It is shared among all instances of the class and is used to store data that should be common for all objects. Static fields belong to the class rather than individual objects. Their lifecycle is associated with the class, and the JVM manages their storage as part of the class's runtime representation.

Example Code:-

```
class Student {

    int rollNo;
    String name;

    // static variable
    static String Training_Center
        = "GFG";

    Student(int r, String n){

        rollNo = r;
        name = n;
    }

    void display(){

        System.out.println(rollNo + " " + name + " "
            + Training_Center);
    }
}
```

```

    }
}

public class GFG{

    public static void main(String[] args){

        Student s1 = new Student(101, "Ravi");
        Student s2 = new Student(102, "Amit");

        s1.display();
        s2.display();

    }
}

```

Output:-

```

101 Ravi GFG
102 Amit GFG

```

7.7.2 Static Blocks

A **static block** is executed only once when the class is first loaded into memory. It is often used to initialize static variables or perform configuration tasks before the main method executes.

Example Code:-

```

class Geeks{

    // static variable
    static int a = 10;
    static int b;

    // static block
    static{

        System.out.println("Static block initialized.");
        b = a * 4;
    }

    public static void main(String[] args)
    {
        System.out.println("from main");
        System.out.println("Value of a : "+a);
        System.out.println("Value of b : "+b);
    }
}

```

Output:-

```

Static block initialized.
from main
Value of a : 10
Value of b : 40

```

7.7.3 Static Methods

A **static method** belongs to the class rather than to any object. It can be called directly using the class name.

- Can access only static data directly.
- Cannot access instance variables or methods directly.
- Cannot use this or super keywords.

Example Code:-

```
class Geeks{

    // static variable
    static int a = m1();

    // static block
    static{

        System.out.println("Inside static block");
    }

    // static method
    static int m1(){

        System.out.println("From m1");
        return 20;
    }

    public static void main(String[] args){

        System.out.println("Value of a: " + a);
        System.out.println("From main");
    }
}
```

Output:-

```
From m1
Inside static block
Value of a: 20
From main
```

7.7.4 Static Nested Classes

A **static nested class** is a class declared as static inside another class. It can be accessed without creating an object of the outer class.

Example Code:-

```
class Outer {

    static class Inner{

        void show(){

            System.out.println(
                "Static Nested Class Method");
        }
    }

    public static void main(String[] args)
    {
```

```
        Outer.Inner obj = new Outer.Inner();
        obj.show();
    }
}
```

Output:-

```
Static Nested Class Method
```

Note: The Inner class is accessed using the outer class name (Outer.Inner).

7.7.5 Static vs Non-Static

The table below demonstrates the difference between Static and Non-Static

Aspect	Static	Non-Static
Belongs To	Class (shared by all objects)	Individual object (unique per instance)
Memory Allocation	Created once when class is loaded	Created separately for each object
Access Method	Accessed using class name	Accessed using object reference
Can Access	Only other static members directly	Both static and non-static members
Common Use	Utility methods, constants, shared data	Object-specific data and behavior